

## BYD Semiconductor

### MMU-720AE Technical Training

| Modules                              | Page |
|--------------------------------------|------|
| Armv9 Memory Management              | 2    |
| SMMUv3 programming                   | 32   |
| MMU S3 Introduction and topology     | 58   |
| MMU S3 Interfaces                    | 69   |
| DTI Protocol Overview                | 85   |
| MMU S3 Translation                   | 94   |
| MMU S3 Caching                       | 104  |
| MMU S3 Additional features           | 117  |
| MMU S3 Configuration and integration | 127  |
| MMU-720AE Protection Mechanism       | 144  |
| Appendix                             |      |
| Arm Glossary                         | 161  |
| Customer Support                     | 162  |



# Memory Management

AArch64

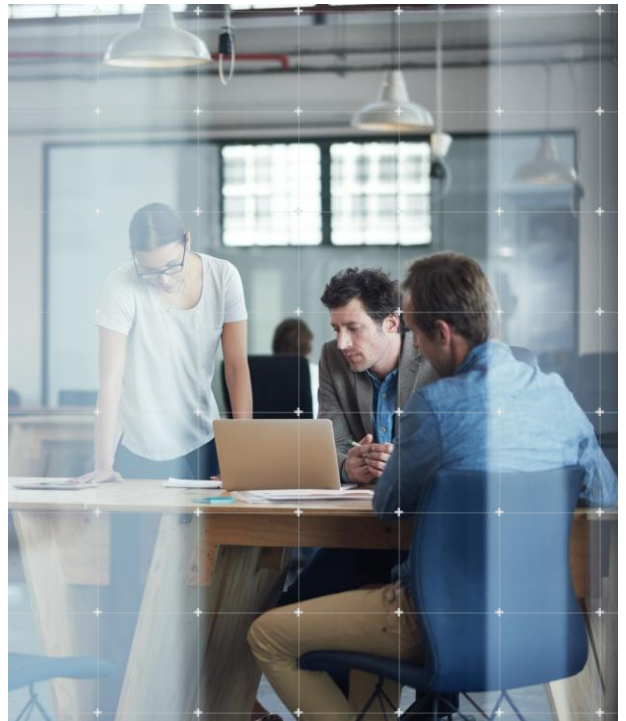
Confidential © 2025 Arm Limited

1

## Objectives

At the end of this topic you will be able to:

- Describe the operation of the Memory Management Unit (MMU)
- Summarize the differences between the memory types
- For a given set of system components, determine suitable Arm memory map properties
- For a given system definition create a memory map applying suitable Arm properties to each region
- Program a set of stage 1 translation tables that flat-map a given physical address space
- Perform a manual translation table walk, given a set of debug data



2 2011 rev 51692

2

# Memory Management Theory

Confidential © 2025 Arm Limited

arm

3

## The need for a Memory Management Unit (MMU)

The Arm A-Class architecture imposes no restrictions on where a system designer places its storage and peripheral resources

- Software is responsible for describing the physical memory map of the system to the PE

An MMU provides a mechanism to abstract memory from software, the storage and peripheral resources that are available in each system

- Hide fragmentation of physical memory
- Create the illusion of larger storage resources
- Separate address space among multiple processes
- Access Control (next slide)

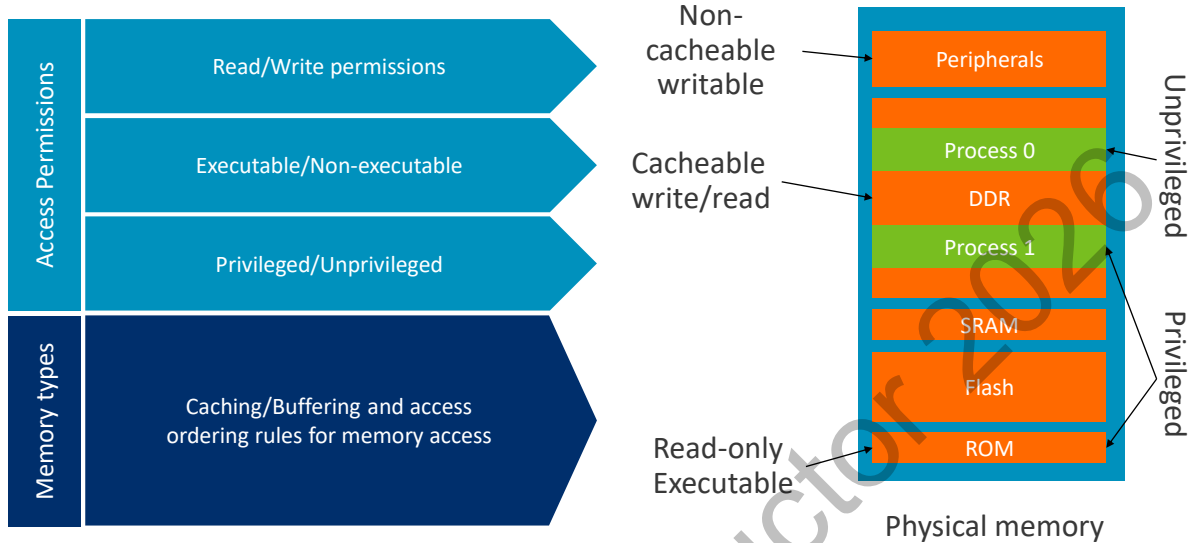
PE



|                 |                                             |
|-----------------|---------------------------------------------|
| 0x3FF_FFFF_FFFF | DRAM2                                       |
| 0x100_0000_0000 | DRAM1                                       |
| 0x80_8000_0000  | Expansion AXI2                              |
| 0x05_0000_0000  | CoreSight subsystem                         |
| 0x04_0000_0000  | Reserved                                    |
| 0x01_0000_0000  | DRAM0                                       |
| 0x00_8000_0000  | Expansion AXI1                              |
| 0x00_6000_0000  | CMN-600 GPV                                 |
| 0x00_5000_0000  | Subsystem peripherals                       |
| 0x00_2A00_0000  | Reserved                                    |
| 0x00_2000_0000  | Expansion AXI (IOFPGA TLX master interface) |
| 0x00_0800_0000  | Boot region                                 |
| 0x00_0000_0000  |                                             |

Application Processor memory map

## MMU and access control



5 2011 rev 51692

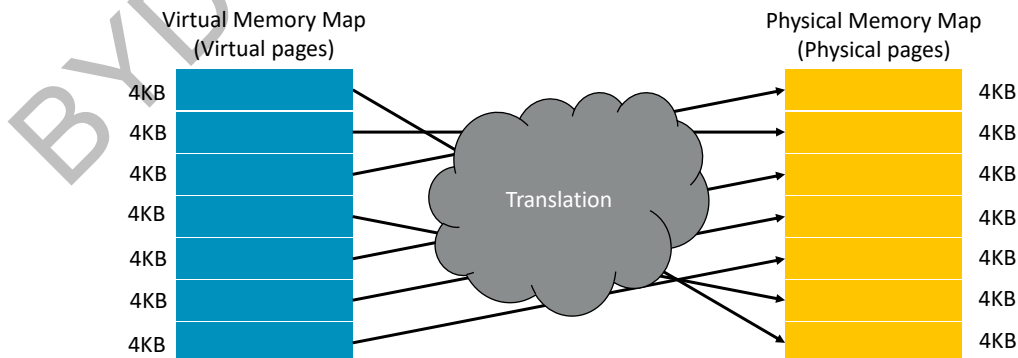
5

## What is virtual addressing?

A system will have a physical address map defined by the hardware

The core operates in a virtual address space defined by software

- Mapping between virtual and physical address defines by a translation tables



6 2011 rev 51692

6

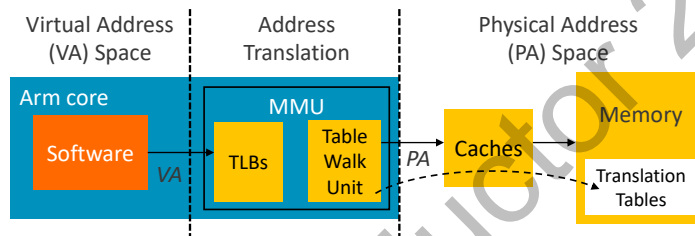
## What is a Memory Management Unit?

The Memory Management Unit is responsible for the translation of virtual addresses to physical addresses

- Provides hardware to read translation tables from memory (translation table walk)
- Translation Lookaside Buffer (TLB) caches recent translations

All memory addresses that are issued by software are virtual

- These memory addresses are passed to the MMU, which checks the TLBs for a recently cached translations. If the MMU does not find a recently cached translation, the table walk unit reads the translation tables from memory



7 2011 rev 51692

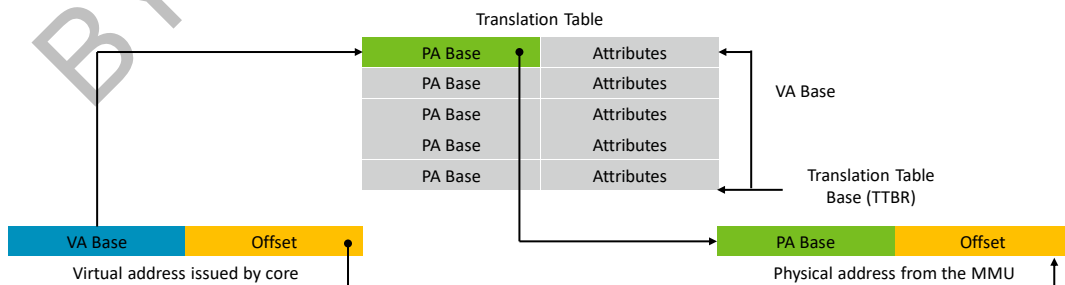
7

## How a physical address is formed

The core issues a 64-bit virtual address

- Top-bits identify which block is being accessed
  - Used as an index within the translation table
- Bottom bits give an offset within the physical block

The MMU combines the physical address bits from the block table entry with the bottom bits from the original virtual address



8 2011 rev 51692

8

## Multiple Levels of Translation Table

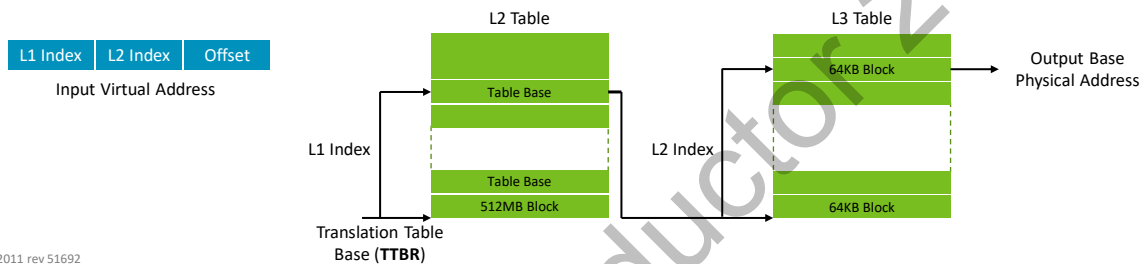
First-level tables divide large areas of virtual memory into coarse sections

- The first-level table is indexed using the first bitfield of the VA (L0 Index)
- In this example, table entries contain the physical base address of a 512MB block, or a pointer to a second-level table (configurable)

Second-level tables subdivide first-level sections into smaller blocks

- The second-level table is indexed using the second bitfield of the VA (L2 Index)
- Table entries contain the physical base address of a 64KB block

The last bitfield of the VA provides the offset of the final physical output address

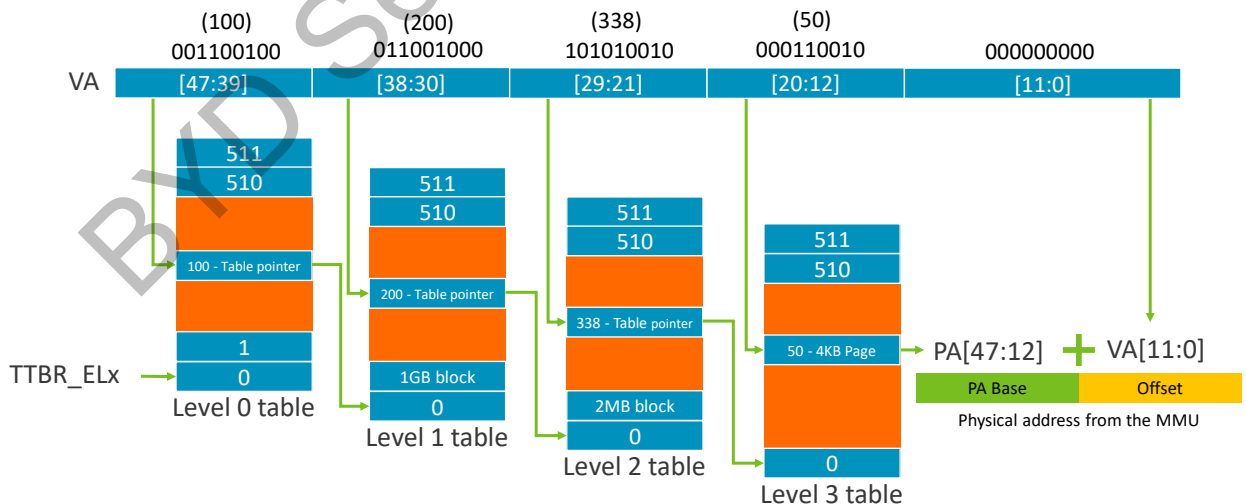


9 2011 rev 51692

9

## Translating a Virtual Address

Example translation table walk, translating a 48-bit VA to a 48-bit PA at a 4KB granularity in AArch64 for the virtual address 0x32322A432000



10 2011 rev 51692

10

# AArch64 Virtual Memory System Architecture VMSAv8-64

Confidential © 2025 Arm Limited

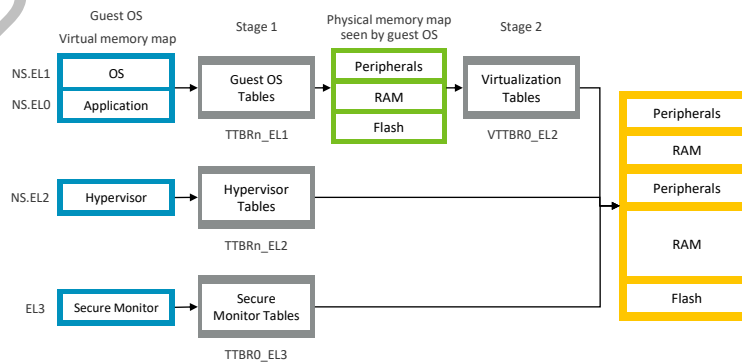
arm

11

## Translation regimes

There are several independent virtual address spaces in AArch64, these are referred to as translation regimes

- Translation regimes are defined independently for different Exception levels and Security states
- Translation regimes are configured by system registers (**TTBRn\_ELx**, **TCR\_ELx**, **SCTLR\_ELx**, **MAIR\_ELx** etc..)



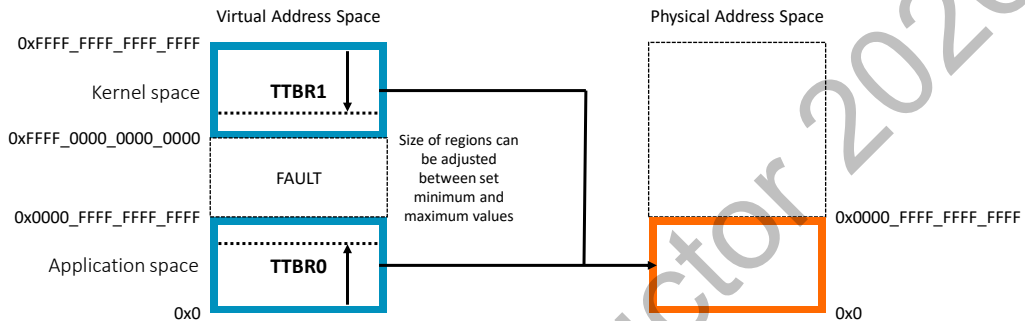
12 2011 rev 51692

12

## Separate tables for app/kernel space

AArch64 EL1/0 supports two translation tables

- Table base addresses are specified in **TTBR0\_EL1** and **TTBR1\_EL1**
- Input virtual address can be a full 64-bit VA
  - **TTBR1\_EL1** selected when upper 16 bits are all 1, **TTBR0\_EL1** selected when upper 16 bits are all 0
  - Size of each address region controlled by **TnSZ** fields in the Translation Control Register (**TCR\_EL1**)



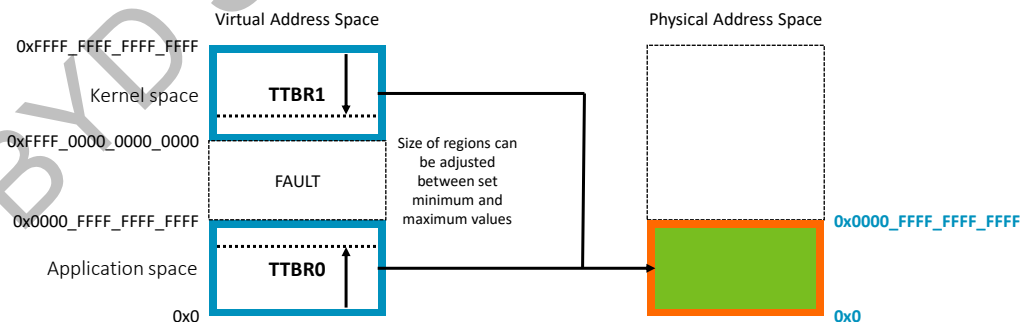
13 2011 rev 51692

13

## Separate tables for app/kernel space

AArch64 EL1/0 supports two translation tables

- But, both of these regions must map to within a single 48-bit physical address space



14 2011 rev 51692

14

# AArch64 Virtual Memory System Architecture features

Supports 3 different translation granules

- 4KB, 16KB, or 64KB
  - Defines the size of the lowest level of translation
- Configurable for each translation regime
- It is IMPLEMENTATION DEFINED which of the three are supported
  - [ID\\_AA64MMFR0\\_EL1](#) reports supported sizes

Support for up to a 52-bit Virtual Address and Physical Address space

- Limited to 48 bits maximum, when not using the 64KB translation granule (or if support for 52 bits is not implemented)
- Support for 52 bits is IMPLEMENTATION DEFINED
  - Support for VA size reported in [ID\\_AA64MMFR2\\_EL1.VARange](#)
  - Support for PA reported in [ID\\_AA64MMFR0\\_EL1.PARange](#)

15 2011 rev 51692

15

## Setting the first level of lookup

The level of translation table used for the first lookup is configurable

- No need to implement all levels of table if the full 52-bit virtual address range is not required
- The first level of lookup is implicitly controlled by the [TCR\\_ELx.TnSZ](#) fields and the specified translation granule

| Table Level | Address Range  |                |                |
|-------------|----------------|----------------|----------------|
|             | 4KB Granule    | 16KB Granule   | 64KB Granule   |
| 0           | 40 to 48 bits  | 48 bits        |                |
| 1           | 31 to 39 bits  | 37 to 47 bits  | 43 to 52 bits  |
| 2           | 25 to 30 bits  | 26 to 36 bits  | 30 to 42 bits  |
| 3           | 16 to 24 bits* | 16 to 25 bits* | 17 to 29 bits* |

- \*Support for smaller address spaces [ID\\_AA64MMFR2\\_EL1.ST](#)
  - OPTIONAL unless Secure EL2 is implemented, then mandatory
  - If not implemented, smallest VA space is 25 bits

16 2011 rev 51692

16

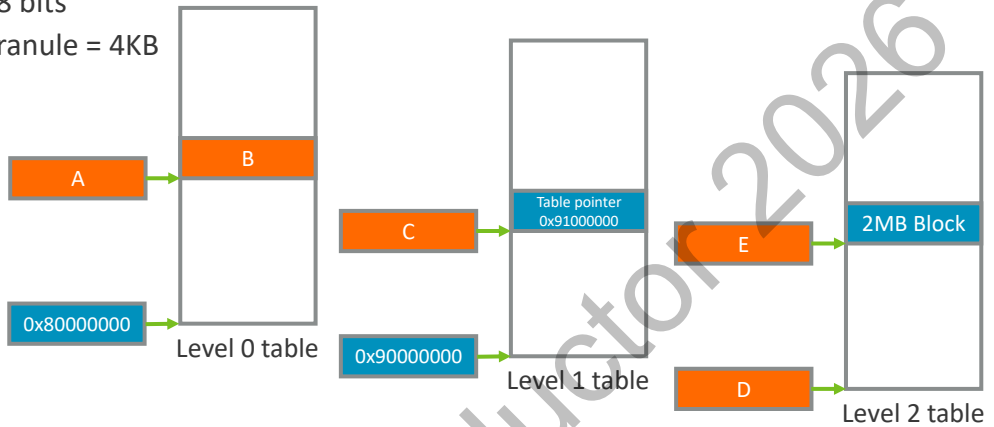
## Translation exercise

VA: : 0x180\_40E0\_0000

TTBR0\_EL3 = 0x80000000

VA space = 48 bits

Translation Granule = 4KB



17 2011 rev 51692

17

## Translation table descriptors

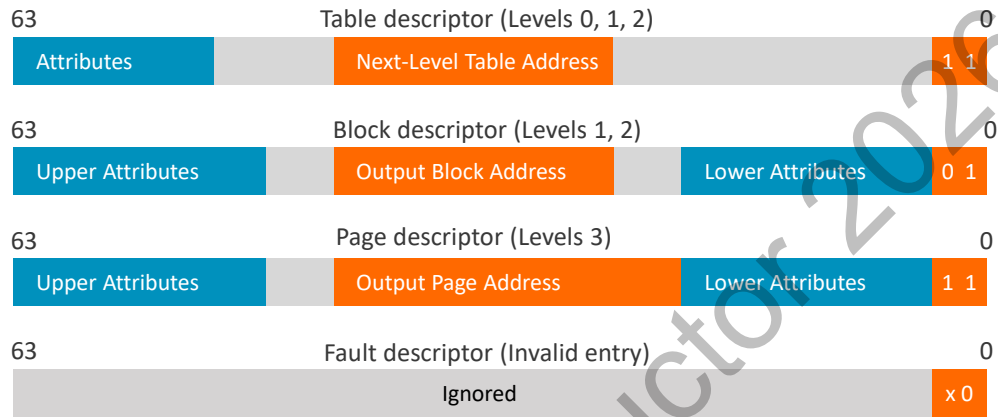
Confidential © 2025 Arm Limited

arm

18

# Translation table descriptors (TTDs)

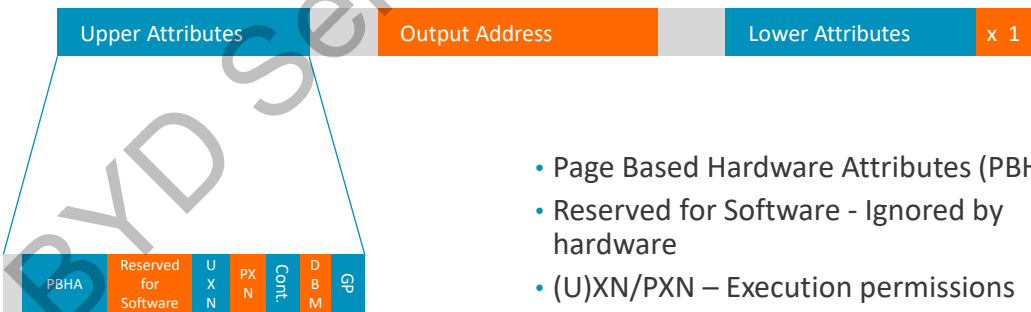
Each translation table descriptor entry is 64 bits (8 bytes) and the bottom two bits determine the type of entry



19 2011 rev 51692

19

## Blocks and Pages upper attributes



- Page Based Hardware Attributes (PBHA)
- Reserved for Software - Ignored by hardware
- (U)XN/PXN – Execution permissions
- Contiguous Bit
- Dirty Bit Modifier
- Guarded Page

20 2011 rev 51692

20

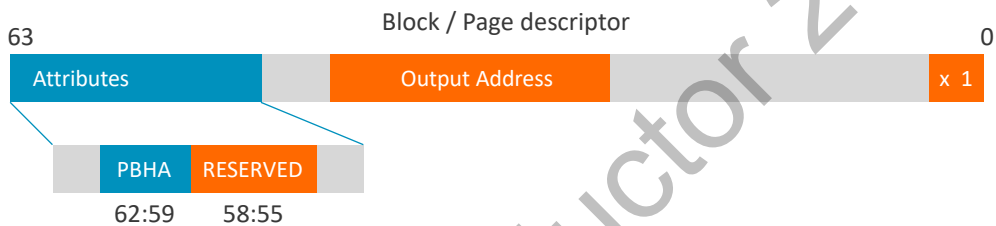
## PBHA and Reserved for Software Use

Top 4 bits of block/page descriptors can be made IMPLEMENTATION DEFINED

- Controlled by **TCR\_ELn/VTCTR\_ELx.HWn** bits
- Allows OS or hypervisor to hand over up to 4 bits of the last level page table for use by hardware for IMPLEMENTATION DEFINED usage

Bits within the block/page descriptors include bits reserved for use by the OS (SW)

- These bits are guaranteed to be ignored by the hardware
- So can be used to record OS specific information in the translation tables (for example a page dirty state)



21 2011 rev 51692

21

## Execute permissions

Blocks and pages can be marked as executable or non-executable (XN)

- Bit[54]: UXN** – Unprivileged Execute Never (EL0)
- Bit[53]: PXN** – Privileged Execute Never (EL1)
- Setting these attributes prevents speculative instruction fetches
- Device memory should always be XN

Processor can also be configured to treat writeable regions as Execute Never

- SCTLR\_EL1.WXN**
  - Regions writeable at EL0 treated as XN at EL0 and EL1
  - Regions writeable at EL1 treated as XN at EL1
- SCTLR\_EL2/3.WXN**
  - Regions writeable at ELn treated as XN at ELn

22 2011 rev 51692

22

## Contiguous Bit

Translation table page and block entries contain a Contiguous Bit (Bit[52])

- Setting this allows the TLB to cache a single-entry covering translations for multiple blocks or pages

To use the contiguous bit

- The pages / blocks must be adjacent (correspond to a contiguous range of VA)
- Have consistent attributes
- Start on an aligned boundary
- Point to a contiguous output address range at the same level of translation
- Number of contiguous pages must be:
  - 4KB: 16 adjacent blocks
  - 16KB: 32 or 128 adjacent blocks
  - 64KB: 32 adjacent blocks
- If these conditions are not met, setting the contiguous bit is considered a programming error which may result in TLB aborts or mismatched attributes

23 2011 rev 51692

23

## Dirty Bit Modifier (DBM)

The dirty state indicates whether a page or section of memory is modified

Bit[51]: DBM in block and page descriptors

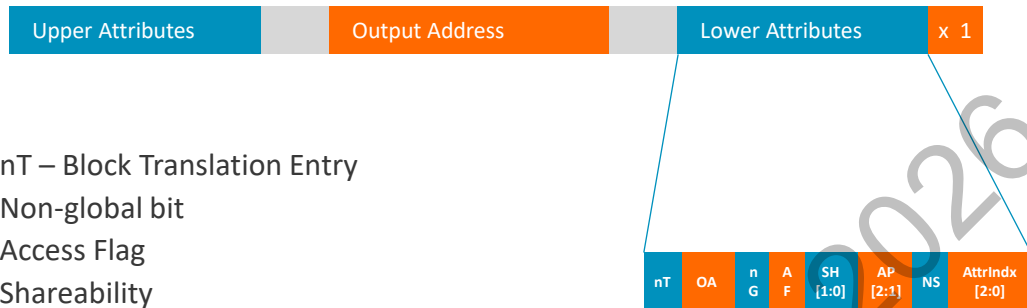
- If a page is marked as RO and **DBM==1**, write accesses to the page will cause the Access Permission bits to be updated to RW (no permission fault be generated)
- Allows software to easily check whether a page had been written to

This can be used in conjunction with the Access Flag (AF) to manage which pages are in use

24 2011 rev 51692

24

## Lower attributes



- nT – Block Translation Entry
- Non-global bit
- Access Flag
- Shareability
- Data Access Permissions
- Non-secure bit
- Memory type Attribute Index

25 2011 rev 51692

25

## nT bit

Armv8.4-TTReM adds a new bit in every block entry descriptor

- Referred to as the block translation **nT** bit
- When nT bit is 0, behavior is as described for Armv8.0-A

Upstream masters of SMMUs are sometimes not tolerant of seeing Invalid mappings

- The SMMU could signal an “Translation fault” to the upstream master
- The SMMU could stall while the translation is Invalid

When this bit is 1, the PE can either

- Generate a Translation fault when using a translation table entry that has the nT bit set
- Ensure that the hardware will not allow having multiple entries translating the same address within a TLB

Armv8.4-TTReM is OPTIONAL

- Support for changing block size without BBM is identified by [ID\\_AA64MMFR2\\_EL1](#)

26 2011 rev 51692

26

## ASIDs

ASID stands for Address Space Identifier

- Used to tag TLB entries to a process

ASID is 8 or 16-bit, controlled by **TCR\_EL1.AS** bit

Current ASID specified in **Bits[63:48]** of **TTBR0\_EL1** or **TTBR1\_EL1**

- TCR\_EL1** controls which **TTBR** holds the ASID
- Typically **TTBR0\_EL1**, as this corresponds to application space

User space context switching involves changing the active translation table

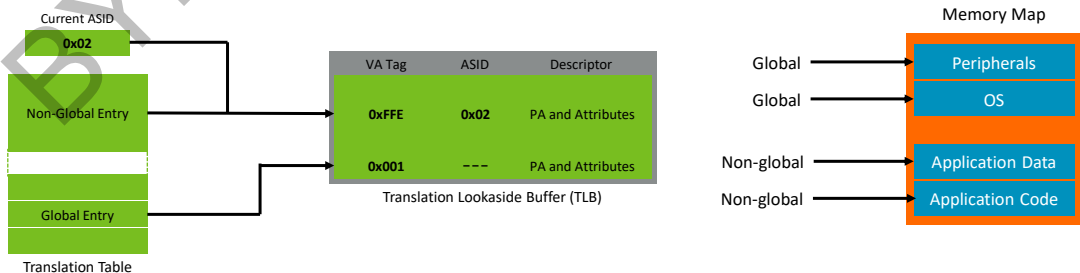
- As the ASID is specified in the TTBR it can be updated atomically

27 2011 rev 51692

27

## Global / Non-Global translations

- Translation table entries can be marked as Global (nG = 0) or Non-global (nG = 1)
  - Global entries apply to all tasks (e.g. Kernel space entries)
  - Non-Global entries only apply to the current task
- For Non-Global entries the TLBs record the current ASID
  - TLB hit only occurs if the ASID in TLB entry matches current ASID



28 2011 rev 51692

28

## Access flag

The Access Flag (AF) is a bit in each Block and Page descriptor

- Indicates whether the entry has been referenced yet
- **AF = 0**: This block entry has not yet been used
- **AF = 1**: This block entry has been used

Entries with AF = 0 trigger an MMU fault

- Abort handler must manually set the **AF** bit in the table entry
- Translation tables can be written with the **AF** bits set if this feature is not being used

Support added for hardware updating of the Access Flag in translation table entries

- Controlled by the **TCR\_ELn.HA** bit for stage 1 translations, and **VTCCR\_ELn.HA** for stage 2

When **HA==1**:

- Accesses to a page/block will cause Access Flag bit to be set atomically by hardware without generating exception
- Can be triggered by speculative accesses

29 2011 rev 51692

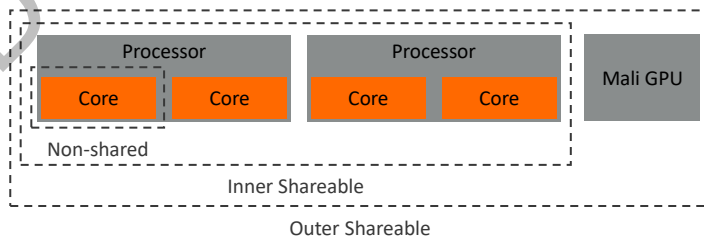
29

## Shareability

The Shareable attribute is used to define whether a location is shared with multiple processors

- Non-shareable (00b) – only used by this observer
- Inner Shareable (11b)/ Outer Shareable (10b) – shared with other observers

Example:



These attributes can define sets of observers for which the Shareability attributes make the data/unified caches transparent for data accesses

- Requires system to provide coherency management

30 2011 rev 51692

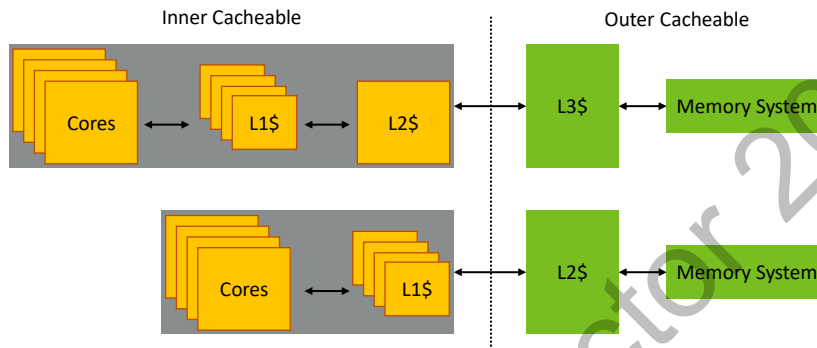
30

## Cacheability

The Arm Architecture describes the caches to software by their behaviours

- Rather than exposing physical topologies

The MAIR types allow a cache policy to be set individually for the **inner** and **outer** caches



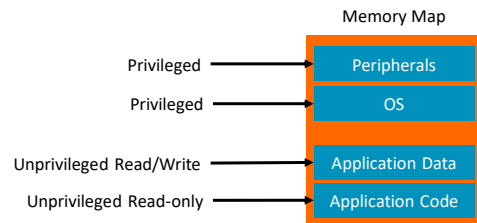
31 2011 rev 51692

31

## Access permissions

Stage one TTDs have access permissions for privileged (EL1+) and un-privileged (EL0) code

| AP | Un-privileged (EL0) | Privileged (EL1/2/3) |
|----|---------------------|----------------------|
| 00 | No access           | Read/write           |
| 01 | Read/write          | Read/write           |
| 10 | No access           | Read-only            |
| 11 | Read-only           | Read-only            |



The EL2 and EL3 Translation regimes never execute un-privileged code

Privileged Access Never (PAN) disables privileged access to any memory accessible by EL0

- Part of PSTATE

32 2011 rev 51692

32

## NS bit

Armv9-A defines two security state:  
Secure and Non-secure (“Normal”)

It also defines two physical address spaces

- Secure and Non-secure

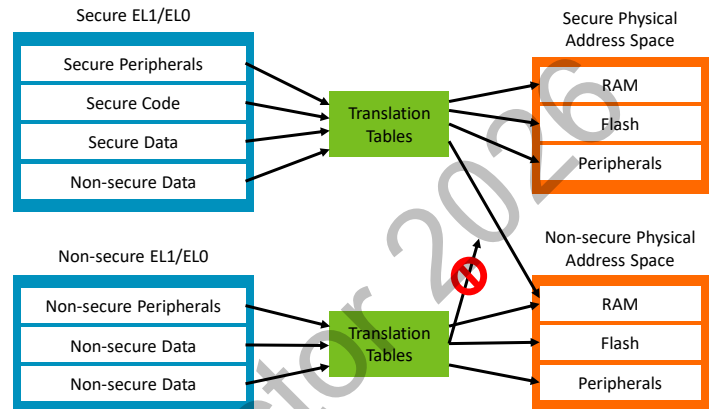
These are in theory completely separate

- SP:0x8000 != NP:0x8000
- But most systems instead treat Secure and Non-secure as an attribute for access control

The Normal world can **only** access the Non-secure physical address space

The Secure world can access **both** physical address spaces

- Controlled through the translation tables NS bit



33 2011 rev 51692

33

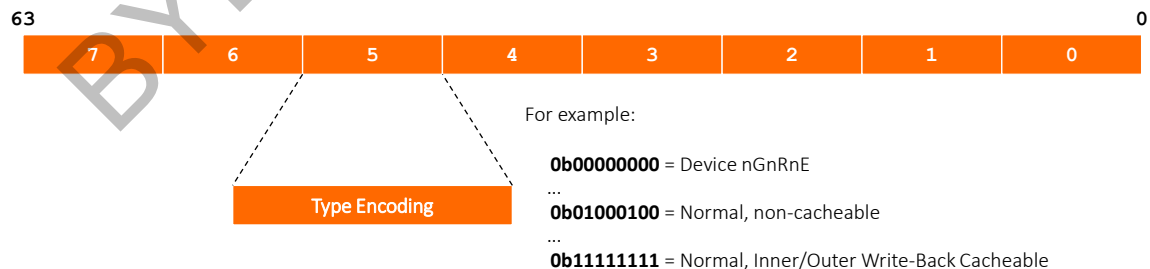
## Specifying the type

Translation table entries do not directly encode the type

- Each block entry specifies a 3-bit index into a table of types

The table of types is held in the Memory Attribute Indirection Register (**MAIR\_ELn**)

- Eight entry table, each entry is 8 bits



34 2011 rev 51692

34

# Memory types

Confidential © 2025 Arm Limited

arm

35

## The Armv8-A Memory Model

The Arm Architecture is a **weakly ordered** memory architecture

- Memory accesses can be **observed** out of program order
- Memory accesses can **complete** out of program order

This allows for implementations to optimize memory accesses and improve performance

- But it does mean extra work for software to ensure correct operation

The Arm Architecture uses memory types to control how much reordering is allowed

# Normal memory

The Normal memory type is used for most memory in the system

Normal memory allows:

- Instruction fetches
- Speculative data reads
- Repeated read accesses to the same location (with no side affects)
- Reads to fetch additional data
- Writes to be repeated (with no side affects)
- Access merging
- Unaligned accesses

Normal memory is not suitable for peripheral/MMIO areas

Normal memory **can** be cached

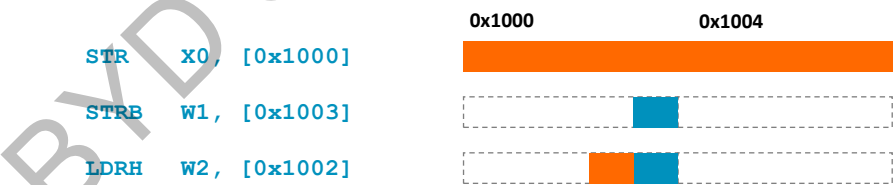
37 2011 rev 51692

37

# Normal memory ordering

As Normal memory implements a weakly ordered memory model

- There is no requirement for Normal accesses to complete in order with respect to other memory accesses
- However, a processor must handle address dependencies



In the example, the accesses are to overlapping addresses

- Processor must ensure the memory is updated as if the **STR** and **STRB** occurred in order
- The **LDRH** must return the most up-to-date value

Memory that is accessed by multiple PEs may require extra ordering

38 2011 rev 51692

38

## Peripheral memory vs Normal memory

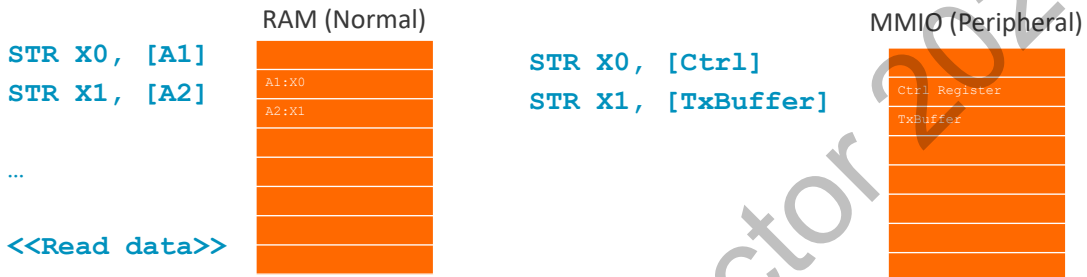
Peripheral memory is often read (or write) sensitive

- E.g. the Tx register of a UART

Peripherals often require particular access size and alignment

- E.g. Registers only accept 32bit writes

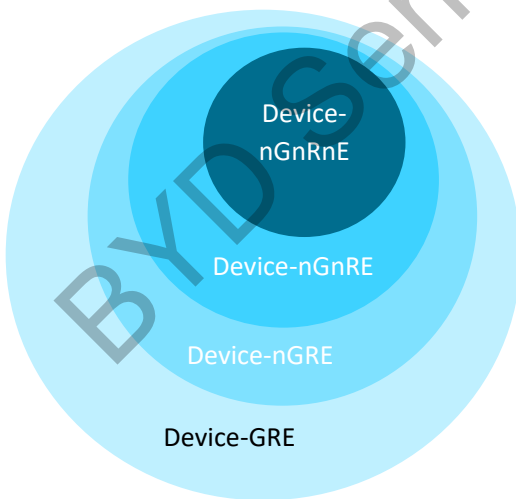
The order of accesses at the end point is very important



39 2011 rev 51692

39

## Device memory



### Gathering (G, nG)

- Determines whether multiple accesses can be merged into a single bus transaction
- nG: number/size of accesses on the bus = number/size of accesses in code

### Re-ordering (R, nR)

- Determines whether accesses to same device can be re-ordered
- nR: accesses to the same IMPLEMENTATION DEFINED block size will appear on the bus in program order

### Early Write Acknowledgement (E, nE)

- Indicates to the memory system whether a buffer can send acknowledgements
- nE: The response should come from the end slave, not buffering in interconnect

40 2011 rev 51692

40

## Which memory type is most suitable?

### Data type

1. Executable code?
2. GIC control registers?
3. Stack and heap?
4. DMA buffer?

### Memory type

1. Normal
2. Device(-nGnRnE)
3. Normal
4. Normal
  - Control registers will be device

41 2011 rev 51692

41

## Configuring a translation regime

Confidential © 2025 Arm Limited

arm

42

## Enabling a translation regime

Program **TTBR\_ELx** to point to the base address of the first level translation table for this translation regime

Program **MAIR\_ELx** with the memory types required in this translation regime

- Memory types will be discussed further in the Memory Model topic

Program **TCR\_ELx** to define the translation regime (VA size, PA size, Translation granule, etc..)

Create a set of valid translation tables which cover the entire VA space (as defined by TCR\_ELx above)

- See next slides

Invalidate the TLBs

- Architecturally, the TLBs are in an UNKNOWN state after reset

Enable the MMU (and caches) using **SCTLR\_ELx**

```
// Set the Base address
// -----
LDR x0, =tt_l1_base // Get address of level 1 for TTBR0_EL3
MSR TTBR0_EL3, x0 // Set TTBR0_EL3 (NOTE: There is no TTBR1
// at EL3)
```

```
// Set up memory attributes
// -----
// This equates to:
// 0 = b01000100 = Normal, Inner/Outer Non-Cacheable
// 1 = b11111111 = Normal, Inner/Outer WB/WA/RA
// 2 = b00000000 = Device-nGnRE
MOV x0, #0x0000000000000FF44
MSR MAIR_EL3, x0
```

```
// Set up TCR_EL3
// -----
MOV x0, #0x19 // T0SZ=0b011001 Limits VA space to 39 bit
ORR x0, x0, #(0x1 << 8) // IGRN0=0b01 Walks to TTBR0 are Inner WB/WA
ORR x0, x0, #(0x1 << 10) // OGRN0=0b01 Walks to TTBR0 are Outer WB/WA
ORR x0, x0, #(0x3 << 12) // SH0=0b11 Inner Shareable
// TBI0=0b0 Top byte not ignored
// T0=0b00 4KB granule
// IPS=0 32-bit PA space
MSR TCR_EL3, x0
```

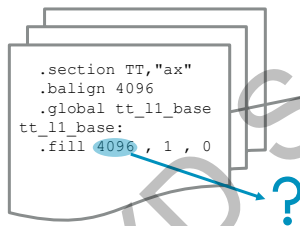
```
// Invalidate TLBs
// -----
TLBI ALLE3
DSB SY
ISB
```

```
// Enable MMU
// -----
MOV x0, #(1 << 0) // M=1 Enable the stage 1 MMU
ORR x0, x0, #(1 << 2) // C=1 Enable data and unified caches
ORR x0, x0, #(1 << 12) // I=1 Enable instruction caching
MSR SCTLR_EL3, x0
ISB
```

43 2011 rev 51692

43

## Writing a set of translation tables

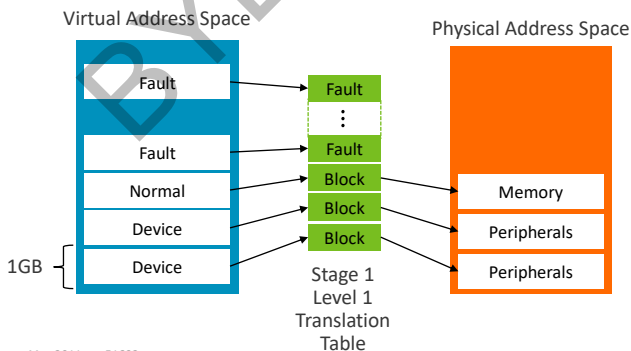


```
LDR x1, =tt_l1_base // Address of L1 table
```

```
// [0]: 0x0000,0000 - 0x3FFF,FFFF
LDR x0, =TT_S1_DEVICE_nGnRnE // Entry template
// Do not need to OR in
// address, as it is 0x0
STR x0, [x1]
```

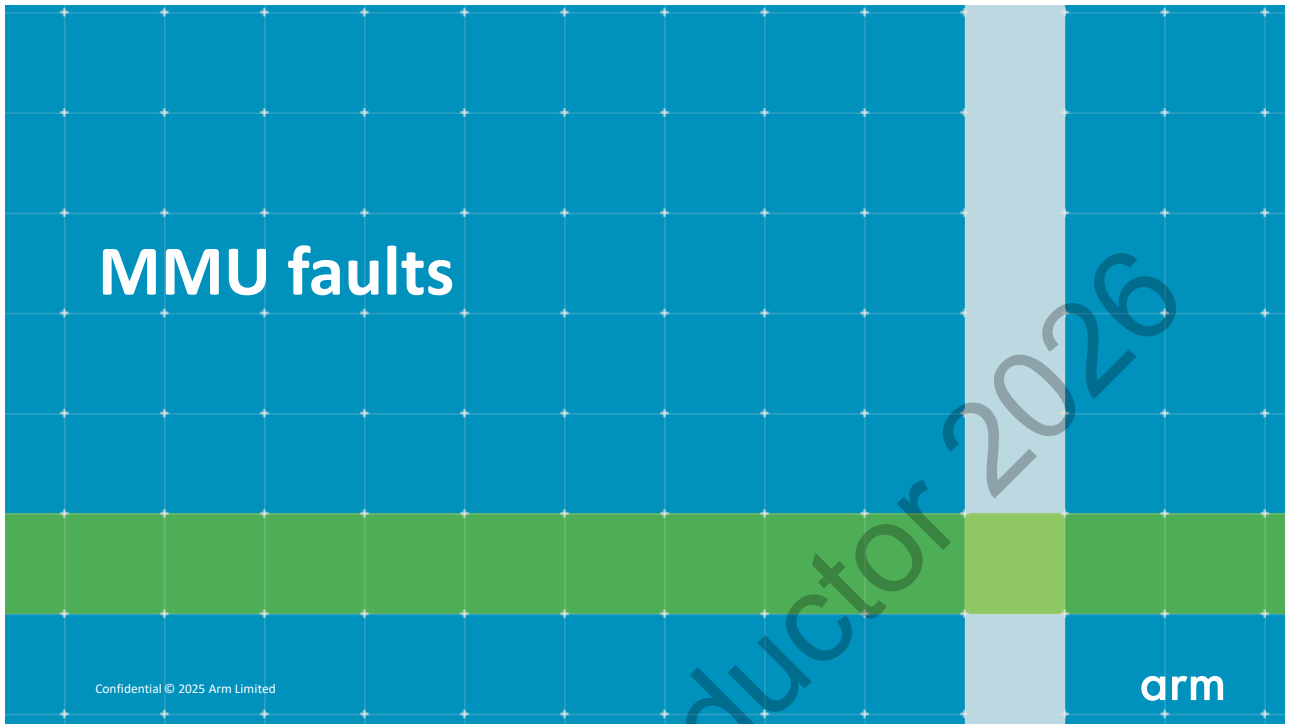
```
// [1]: 0x4000,0000 - 0x7FFF,FFFF
LDR x0, =TT_S1_DEVICE_nGnRnE // Entry template
ORR x0, x0, #0x40000000 // 'OR' with base PA
STR x0, [x1, #8]
```

```
// [2]: 0x8000,0000 - 0xBFFF,FFFF
LDR x0, =TT_S1_NORMAL_WBWA // Entry template
ORR x0, x0, #TT_S1_INNER_SHARED // 'OR' with IS attr
ORR x0, x0, #0x80000000 // 'OR' with base PA
STR x0, [x1, #16]
DSB SY
```



44 2011 rev 51692

44



45

## MMU faults (1)

If MMU is unable to perform the translation for one of a number of reasons, and therefore causes an exception to be generated. This exception called an MMU fault.

MMU faults caused during the translation process are Synchronous

- Permission faults
- Translation faults
- Address size faults
- Access flag fault
- TLB conflict abort

The Exception level that an MMU fault is taken to depends on the translation regime and stage that generated the fault. The fault context saved in the appropriate ESR\_ELx

46

## MMU faults (2)

- Data address authentication failure
- Address is out of range (as the configured address size)
- Invalid configuration
  - TCR\_ELx.TnSZ is configured to the value which is less than minimum supported
- Errors in translation tables
  - Translation table descriptor is invalid
  - ARMv8.2-LPA is not implemented, but a block descriptor is found in a Level 1 table
- Specific configuration for a translation table walks (EPDn = 1)
- Misuse of a contiguous bit

A lot of ways to get a fault. Good thing that it is synchronous.

47 2011 rev 51692

47

## Permission fault

An attempt to perform a memory access that the translation table access permission bits do not permit generates a Permission fault.

There are some other conditions (for example secure instruction fetch) which can generate this type of fault.

Can be generated at any level of lookup and the reported fault code identifies the lookup level.

A TLB might hold a translation table entry that causes a Permission fault. If while handling this fault SW updates translation tables, it also must invalidate the appropriate TLB entry.

48 2011 rev 51692

48

## Address size fault

Nonzero address bits above the most significant bit of the maximum output address size

Architecture guaranties that the translation entry is not cached, so no need to invalidate TLB

49 2011 rev 51692

49

## Access flag fault

Access flag bit is 0 in a translation table descriptor

The Access flag mechanism expects that, when an Access flag fault occurs, SW resets the Access flag to 1 in the translation entry that causes the fault.

50 2011 rev 51692

50

## TLB conflict abort

An address can hit multiple entries in the TLB if the TLB has been invalidated inappropriately, for example if TLB invalidation required by the architecture has not been performed.

If the address matches multiple entries in the TLB, it is IMPLEMENTATION DEFINED whether a TLB conflict abort is generated.

51 2011 rev 51692

51

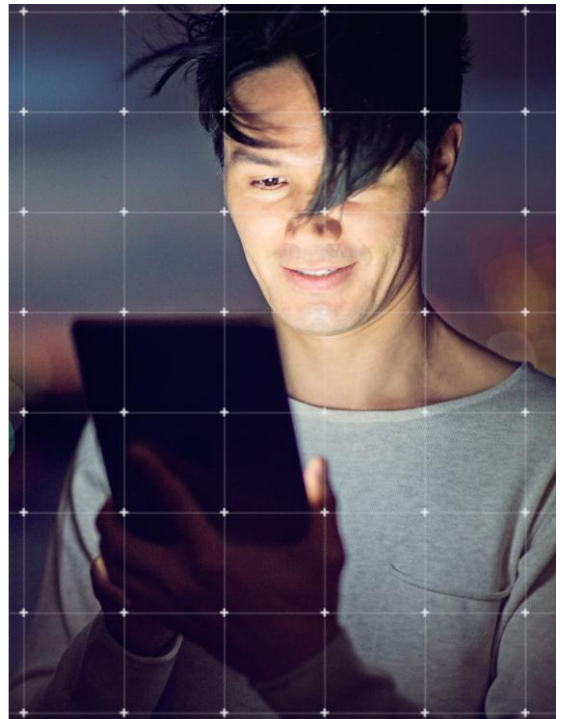
## Module summary

In this module we have covered:

- The processes of translating a virtual address
- The AArch64 VMSAv8-64 memory architecture
- How to write static translation tables
- How to dynamically modify translation tables
- The format of the translation table descriptors
- The contents of the translation table descriptors
- The Arm memory types

52 2011 rev 51692

52



Thank You!  
Danke!  
Merci!  
谢谢!  
ありがとう!  
Gracias!  
Kiitos!  
감사합니다  
धन्यवाद

arm

53

Confidential © 2025 Arm Limited

53

## AArch64 TLBI instructions

AArch64 TLB maintenance operations are performed via instructions

TLBI <type><level>{IS} {, <Xt>}

### Type

- **All** - All TLB entries
- **VMALL** - All TLB entries (stage 1, for current guest OS)
- **VMALLS12** - All TLB entries (stage 1 & 2 for current guest OS)
- **ASID** - Entries that match ASID in Xt
- **VA** - Entry for virtual address and ASID specified in Xt
- **VAA** - Entries for virtual address specified in Xt, with any ASID
- There are more...

### Level

- **En** = ELn virtual address space (n can be 3, 2, or 1)

### IS

- Inner-shareable operation

Examples:

```
TLBI VAE1, X0 ; Invalidate address/ASID in x0, for EL1 virtual address space
TLBI ALLE3    ; Invalidate entries for the EL3 virtual address space
```

54 2011 rev 51692

54

## Translation granules

### 4KB granule:

- 4-level look up, 48-bit address, 9 address bits per level (512 entries)

| [47:39]                                                             | [38:30]                                                                 | [29:21]                                                                 | [20:12]                                                            | [11:0]       |
|---------------------------------------------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------|--------------|
| Level 0 Table Index<br>• Point to an L1 Table<br>(No Block entries) | Level 1 Table Index<br>• Point to an L2 Table<br>• Point to a 1GB Block | Level 2 Table Index<br>• Point to an L3 Table<br>• Point to a 2MB Block | Level 3 Table Index<br>• Point to a 4KB Page<br>(No Table entries) | Block Offset |

### 16KB granule:

- 4-level look up, 48-bit address, 11 address bits per level (2048 entries)

| [47]                                                                | [46:36]                                                                  | [35:25]                                                                  | [24:14]                                                             | [13:0]       |
|---------------------------------------------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------|---------------------------------------------------------------------|--------------|
| Level 0 Table Index<br>• Point to an L1 Table<br>(No Block entries) | Level 1 Table Index<br>• Point to an L2 Table<br>• Point to a 64GB Block | Level 2 Table Index<br>• Point to an L3 Table<br>• Point to a 32MB Block | Level 3 Table Index<br>• Point to a 16KB Page<br>(No Table entries) | Block Offset |

55 2011 rev 51692

55

## Translation granules cont.

### 64KB granule (48-bits):

- 3-level look up, 48-bit address, 13 address bits per level (8192 entries)

| [47:42]                                                             | [41:29]                                                                   | [28:16]                                                             | [15:0]       |
|---------------------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------|--------------|
| Level 1 Table Index<br>• Point to an L1 Table<br>(No Block entries) | Level 2 Table Index<br>• Point to an L3 Table<br>• Point to a 512MB Block | Level 3 Table Index<br>• Point to a 64KB Page<br>(No Table entries) | Block Offset |

### 64KB granule (52-bits):

- 3-level look up, 52-bit address, 13 address bits per level (2048 entries)

| [51:42]                                                                 | [41:29]                                                                   | [28:16]                                                             | [15:0]       |
|-------------------------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------------|--------------|
| Level 1 Table Index<br>• Point to an L2 Table<br>• Point to a 4TB Block | Level 2 Table Index<br>• Point to an L3 Table<br>• Point to a 512MB Block | Level 3 Table Index<br>• Point to a 64KB Page<br>(No Table entries) | Block Offset |

56 2011 rev 51692

56

## Translation Lookaside Buffers (TLBs)

Translation Lookaside Buffers (TLBs) store the result of recent translations

Dynamically changing translation table entries requires maintenance of the TLB

- Whenever the translation tables are modified the TLBs must be manually invalidated
- TLBs will not cache table entries which result in a Translation fault, Address size fault or an Access flag fault

TLB invalidations can severely affect performance

- Desirable to limit the scope of the invalidation to specific entries if possible (VA, EL, ASID, VMID, PE, ect)
- See appendix for available TLBI qualifiers

```
<< Write to Translation Tables >>
DSB ISH          ; Ensure write has completed
TLBI ???
DSB ISH          ; Ensure completion of TLB invalidation
ISB              ; Synchronize context on this processor
```

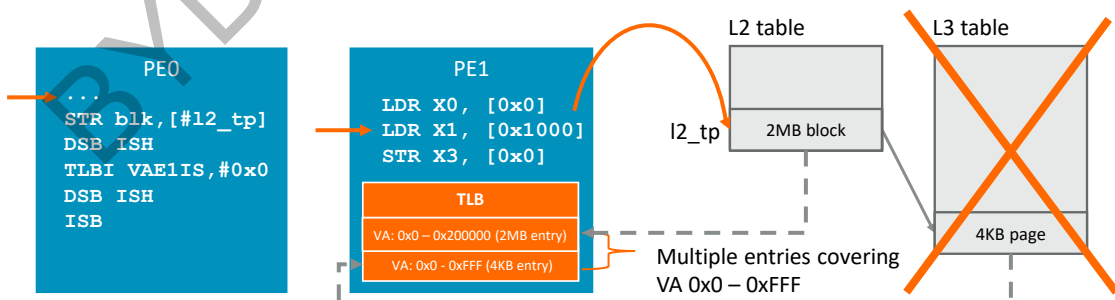
57 2011 rev 51692

57

## Changing the size of translation table mappings

Changing the block size used for translating an address may lead to the possibility of having multiple entries translating the same address within a TLB

- Creates a window in which multiple TLB entries translating the same address can be present prior to the TLBI
- Possibly breaking coherency and ordering guarantees



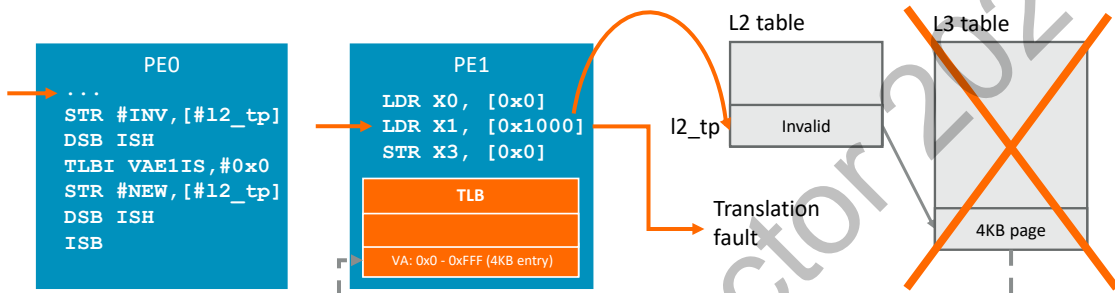
58 2011 rev 51692

58

## Break-Before-Make (BBM)

AArch64 requires that when changing the block size used for translating an address the software uses a Break-Before-Make approach (BBM)

- BBM prevents the possibility of having multiple entries translating the same address within a TLB
- The sequence involves “breaking” the translation by writing an Invalid descriptor
- Generates a Translation fault, if a translation is made prior to the TLBI



59 2011 rev 51692

59

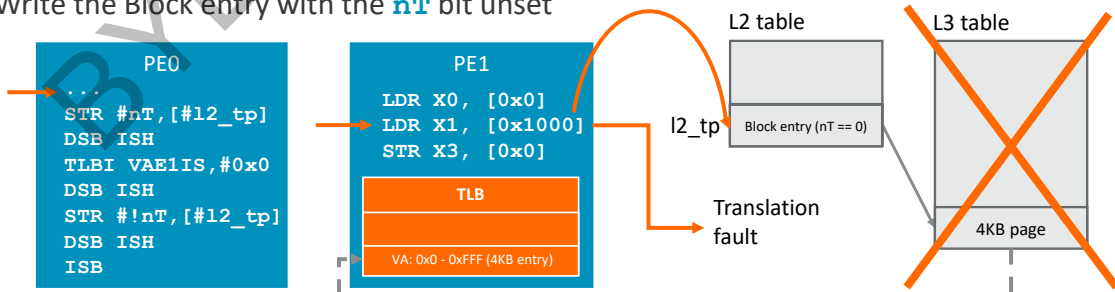
## Block translation entry bit

Write the new Block entry with the **nT** bit set

- Not an Invalid descriptor, no longer blocking to upstream SMMU master

PE1 can either generate a Translation fault or it could atomically invalidate the old entry

Write the Block entry with the **nT** bit unset



61 2011 rev 51692

61

# arm

## Programming the SMMU

SMMUv3

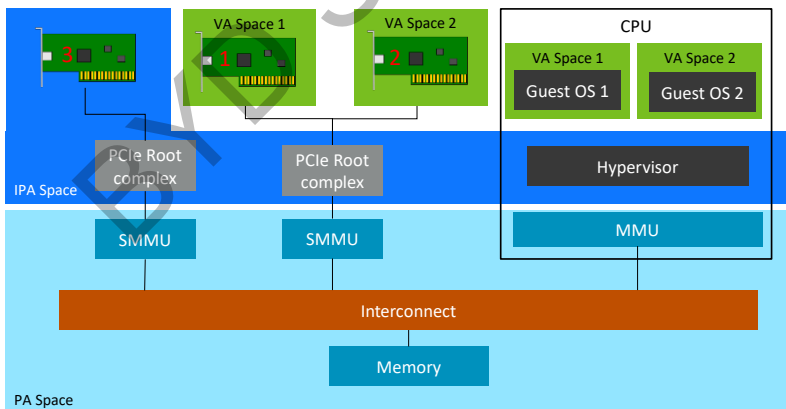
Confidential © 2025 Arm

1

### What is a SMMU?

#### A System MMU provides address translation for non-processor masters

- Uses same translation table format as the processor
- An SMMU can perform stage 1 only, stage 2 only or both stages of translation



2 1462 rev 47356

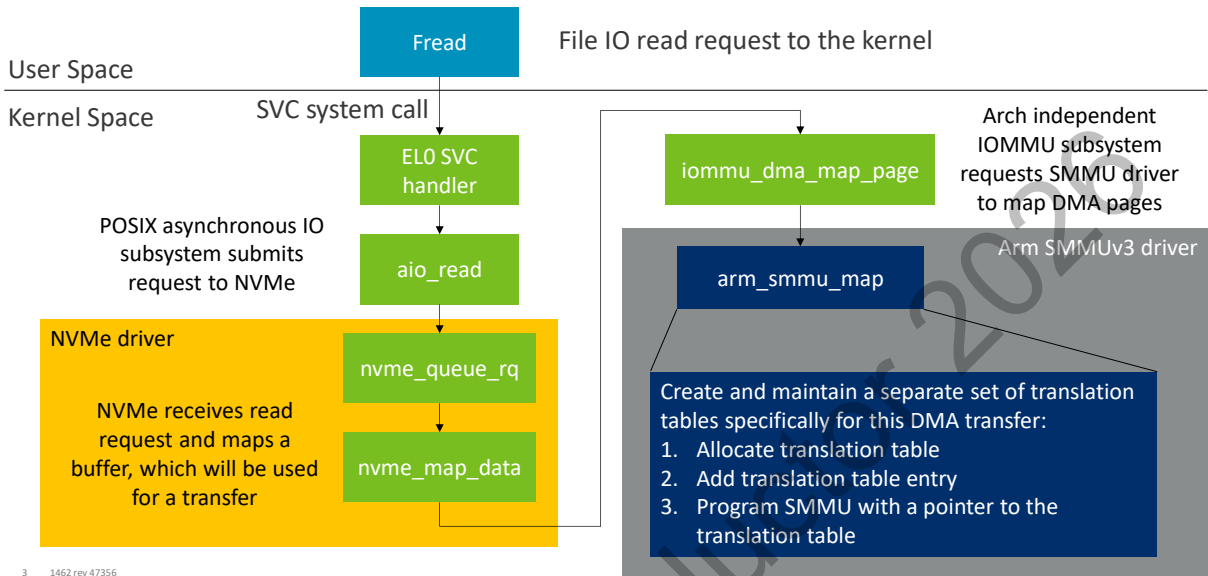
#### Virtualization example:

- Stage 1 and Stage 2 translations are enabled
  - Device 1 is assigned to the Guest OS 1
  - Device 2 is assigned to the Guest OS 2
- Stage 1 translation is enabled
  - Device 3 is assigned to the hypervisor

Stage 2 only is not shown. That would be used when a device is assigned to a Guest OS and the Guest OS wants to program the device using physical addresses.

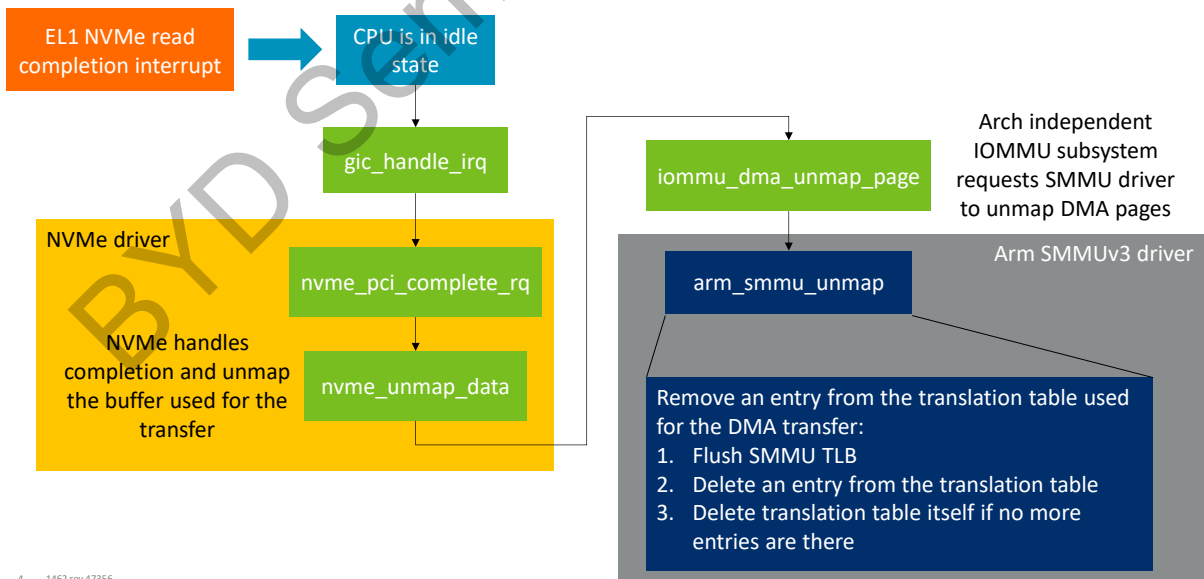
2

## NVMe SSD read request example



3

## NVMe SSD read completion example



4

# Agenda

## Development of the SMMU architecture (a brief history)

Overview of the operation of an SMMU

Programming the SMMU

5 1462 rev 47356

5

## Development of the SMMU Architecture

### SMMUv1

**Features:**

- Support for Armv7-A
- Supports up to 128 contexts
  - Configuration stored in registers
- Optional support for TrustZone

**Implemented by:**

- MMU-40x
  - Stage 2 only

### SMMUv2

**Adds:**

- Support for Armv8-A
- Support for Armv8.1-A

**Implemented by:**

- MMU-500

### SMMUv3

**Adds:**

- Supports larger number of supported contexts
- Configuration stored in memory
- Support for Substreams
- Support for PCIe address translation requests
- Message based interrupts, to integrate with GICv3

**Implemented by:**

- MMU-600
- MMU-700

6 1462 rev 47356

6

## SMMUv3.1 Updates

### Adds support for Armv8.2-A

- Support for 52-bit VAs, IPAs and PAs
- Hardware update of the Access Flag and Dirty State
- ELO vs EL1 execute permissions in stage 2 tables

### Support for additional transaction types: cache-stash and destructive reads

7 1462 rev 47356

7

## SMMUv3.2 Updates

### SMMUv3.2 adds support for the Arm v8.4 architecture

- Support for Memory Partitioning And Monitoring (MPAM).
- Secure EL2 and Secure stage 2 translation (where supported by CPUs)
  - All previous rules about Secure streams being stage-1 only are removed.
- Stage 2 control of memory types and Cacheability.
  - Basically, stage 2 attributes can override stage 1 attributes
- Small translation tables support
  - For limited amount of memory being translated, smaller page tables can be used
- TLB range maintenance instructions
  - Allows invalidation of TLB entries over a range of entries (based on virtual address)
- Translation table updates without break-before-make.
  - By setting nT bit in page table, allows old page descriptors to be invalidated if memory block size has changed

*Note that not all CPUs support all features of the v8.4 Architecture*

8 1462 rev 47356

8

## SMMUv3.3 Updates

### SMMUv3.3 adds support for the Arm v8.5-A VMSA

- Restrict ELO access to halves of the memory map
- Protected Table Walk

### Enhanced Command Queue interface

9 1462 rev 47356

## SMMU for RME Updates

### Support for interacting with Realm Management Extension

- Granule Protection Checks (GPCs) for SMMU-originated and client-originated accesses
- Root PAS registers for GPC
- Support for Realm Management Extensions
- Reporting of GPC faults
- Support for GPC invalidations

10 1462 rev 47356

## RME for Device Assignment

### Support for interacting with Realm Management Extension

- Realm translation regime
- Realm queues
- Realm programmers model
- Memory Encryption Context (MEC) support

11 1462 rev 47356

11

## Agenda

Development of the SMMU architecture (a brief history)

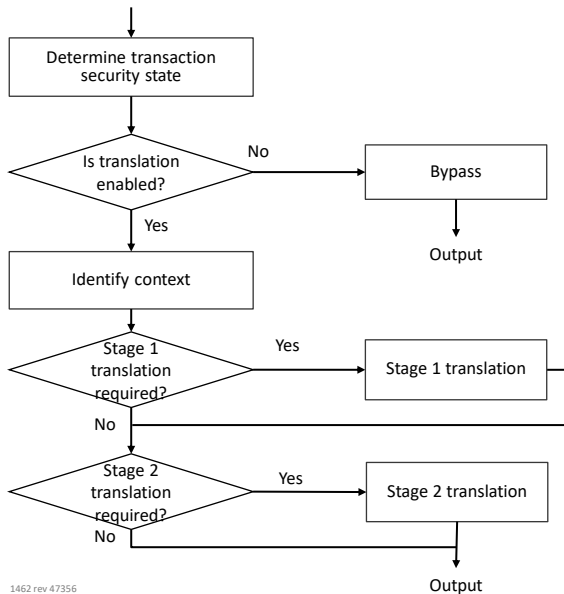
**Overview of the operation of an SMMU**

Programming the SMMU

12 1462 rev 47356

12

## Translation Process



**Translation can be enabled separately for Secure and Non-secure masters**

- Identified by SSD (Secure State Determination)

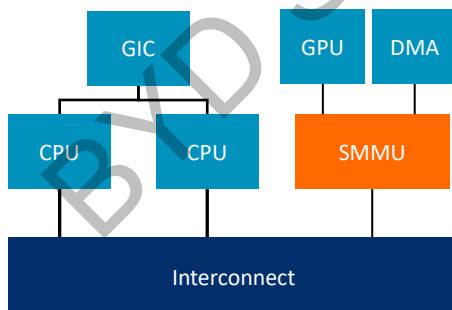
**A StreamID is used to determine which context should be used for translation**

- A transactions might require both stage 1 and stage 2 translations

**What happens if a translation faults?**

13

## Contexts and Streams



**A SMMU can be aware of multiple different sets of translation tables**

- A set of tables, and the associated settings, are called a context

**Incoming transactions are tagged with a StreamID**

- The StreamID is used to identify which context should be used for translating a transaction
- StreamID to context mapping is run time configurable

**Example:**

- GPU uses StreamID 3, allocated to virtual machine 3
- DMA channel 1 uses StreamID 4, allocated to virtual machine 2
- DMA channel 2 uses StreamID 5, allocated to virtual machine 1

**How the StreamID is formed is IMPLEMENTATION DEFINED**

- Typically a combination of side-band signals

14 1462 rev 47356

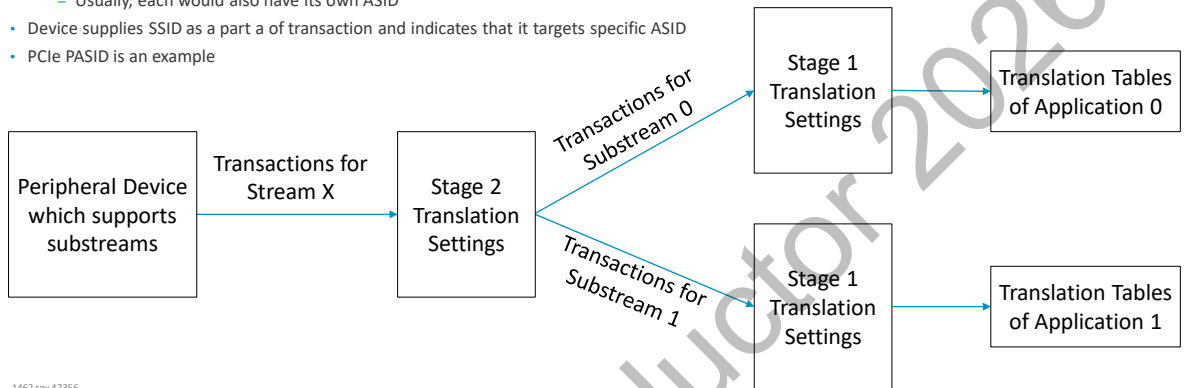
14

## Substreams

Typically, all the transactions for a given Stream use the same translation settings

### SMMUv3 allows each Stream to have multiple Substreams

- All the substreams share the same stage 2 translation settings
- Each substream has its own stage 1 translation
  - Usually, each would also have its own ASID
- Device supplies SSID as a part of a transaction and indicates that it targets specific ASID
- PCIe PASID is an example



15 1462 rev 47356

15

## TrustZone (1)

### The SMMU architecture has optional support for two Security states (TrustZone)

- Reported by SMMU\_S\_IDR1.SECURE\_IMPL
- When not implemented, SMMU is always in Non-secure state

#### When TrustZone support is implemented:

- There are separate Secure and Non-secure Stream Tables
- All transactions have an SEC\_SID

**SEC\_SID==0:** this is a Non-secure stream and uses the Non-secure Stream Table

**SEC\_SID==1:** this is a Secure stream and uses the Secure Stream Table

The checking of SEC\_SID is sometimes referred to as Secure State Determination or SSD

16 1462 rev 47356

16

# TrustZone (2)

**Secure and Non-secure are different StreamID namespaces**

- Secure StreamID 0 is a different StreamID to Non-secure StreamID 0

**A Secure stream can generate both Secure and Non-secure downstream transactions**

**A Non-secure stream can only generate Non-secure downstream transactions**

- Incoming NS attribute ignored

17 1462 rev 47356

# Realm Security

When Realm Device Assignment is supported:

SEC\_SID is extended to 2 bits

| SEC_SID | Security State |
|---------|----------------|
| 0b00    | Non-secure     |
| 0b01    | Secure         |
| 0b10    | Realm          |
| 0b11    | Reserved       |

Realm transactions are associated with the Realm Programming Interface  
Realm transactions use the incoming NS bit to select between Non-secure and Realm

18 1462 rev 47356



19

## Bypass

- Attribute override settings taken from **SMMU\_(S\_)GBPA**
- Configured separately for each Security state

20

## Agenda

Development of the SMMU architecture (a brief history)

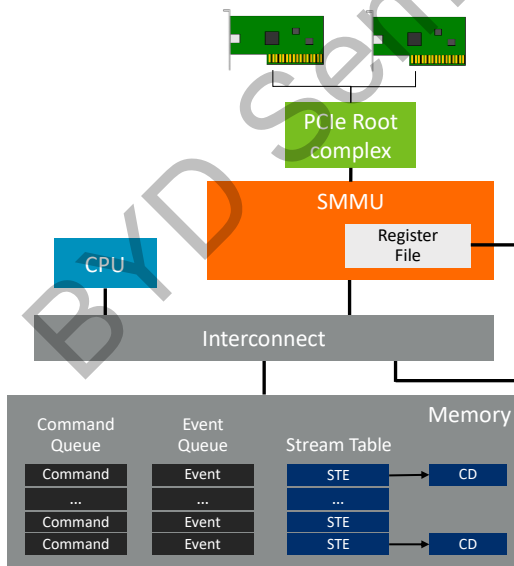
Overview of the operation of an SMMU

### Programming the SMMU

21 1462 rev 47356

21

## SMMUv3 Programming Interface



22 1462 rev 47356

### Most configuration stored in memory structures

- Software must allocate the memory for these structures
- The SMMU provides registers to set the location and sizes of these structures

### A circular command queue is used to control the SMMU

- SW writes commands to the queue

### A circular event queue is used to notify SW

- SMMU writes events to the queue

### The StreamID of incoming transactions used to index Stream Table

- Stream Table Entry (STE) contains details of the Stage 2 translation
- STE can optionally point at a Context Descriptor (CD) which contains the details of the Stage 1 translation

22

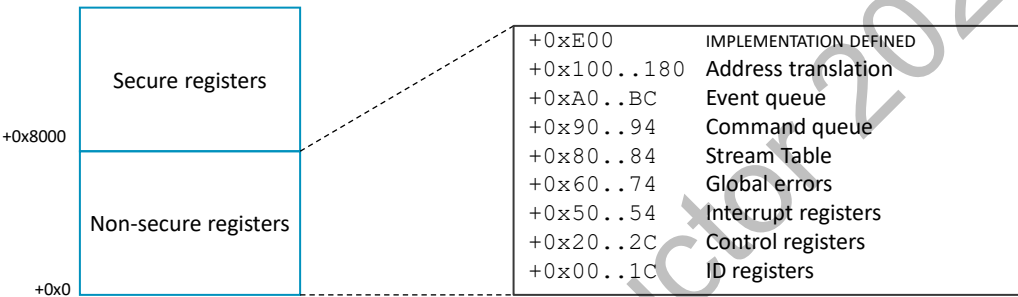
# SMMU Registers

The SMMU registers occupy two consecutive 64K pages

- Additional pages might be present for optional or IMP DEF features

There are separate registers for Secure and Non-secure configuration

- SMMU\_S\_\* Secure register (RES0 to Non-secure software)
- SMMU\_\* Non-secure register
- The Secure and Non-secure register sets are very similar to each other



23 1462 rev 47356

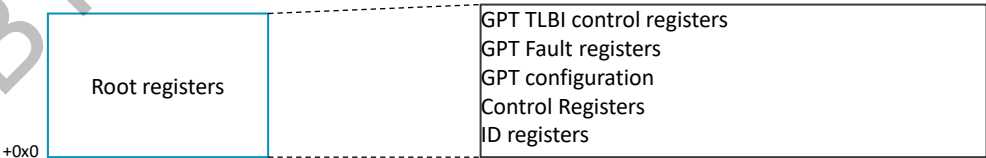
# Root Control Page

Only present if RME is implemented

- Base address is IMPLEMENTATION DEFINED

Only accessible from Root PAS

- Accesses from other PAS as RAZ/WI



24 1462 rev 47356

# Realm Register Pages

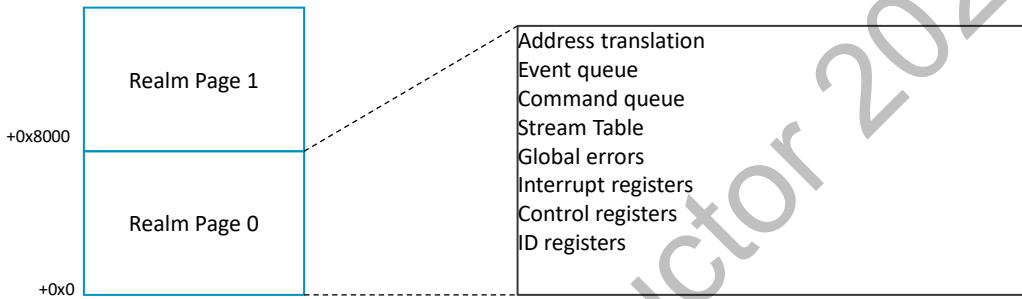
Only present if RME is implemented

The Realm registers occupy two consecutive 64K pages

- Accessible to Root and Realm PAS

Realm Page 0 has similar contents to the Secure/Non-secure configuration pages

Realm Page 1 holds Realm Event and PRI queue pointers

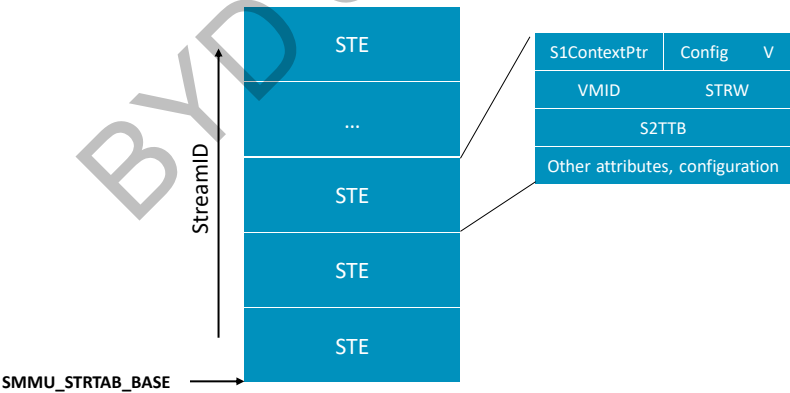


25 1462 rev 47356

# Stream Table

The StreamID is used to index the Stream Table

- A Stream Table Entry (STE) stores the context information for that Stream
- 64 Bytes per STE
- Contiguous memory required



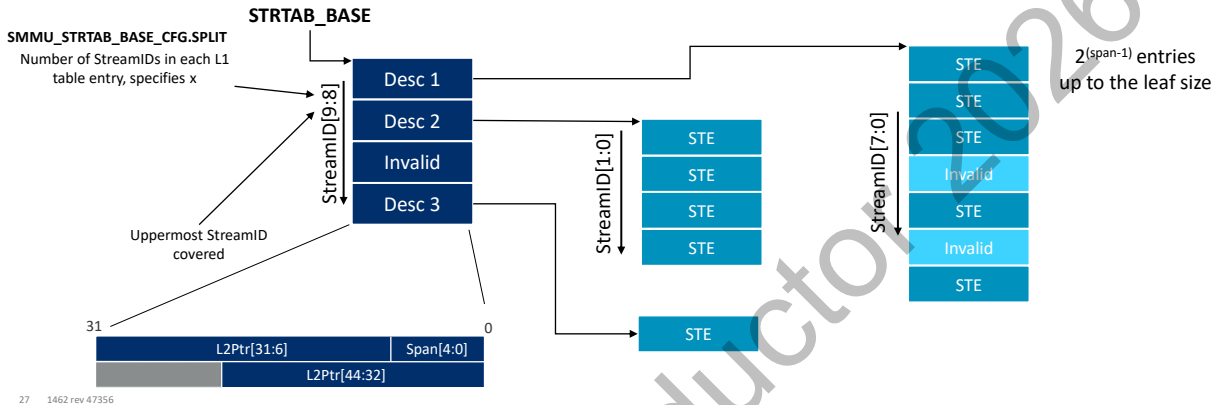
- **Valid** - Whether this StreamID is valid
- **Config** – Whether stage 1 and/or 2 translation enabled
- **STRW** – Stream World (e.g. AArch64 EL2)
- Stage 2 translation settings
- Pointer to Context Descriptor (stage 1 settings)
- Whether Substreams are used and default Substream
- ....

26 1462 rev 47356

## 2-Level Tables

## SMMUv3 allows for the Stream Table to be 2-level

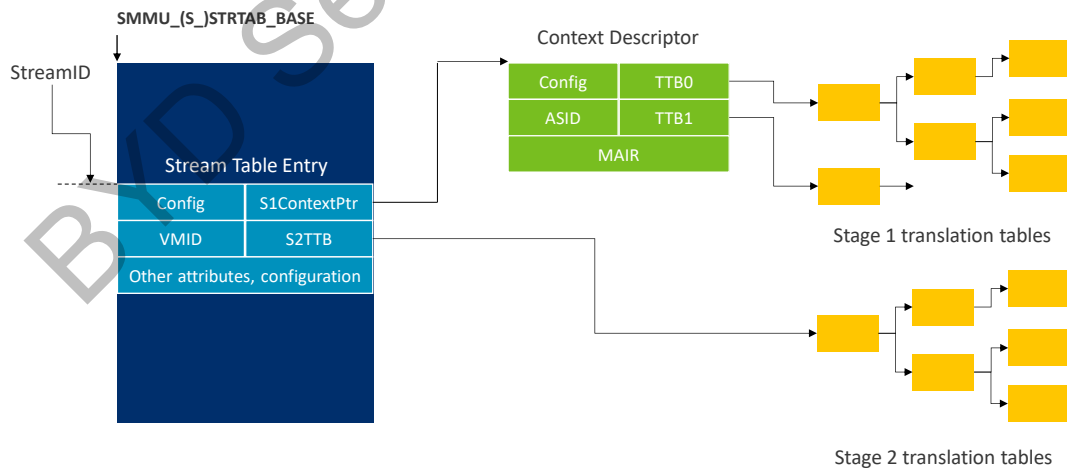
- L1 table entries are either invalid or pointer to L2 table
  - The number of StreamIDs covered by an L1 table entry set via a register
- L2 tables contain STEs



27

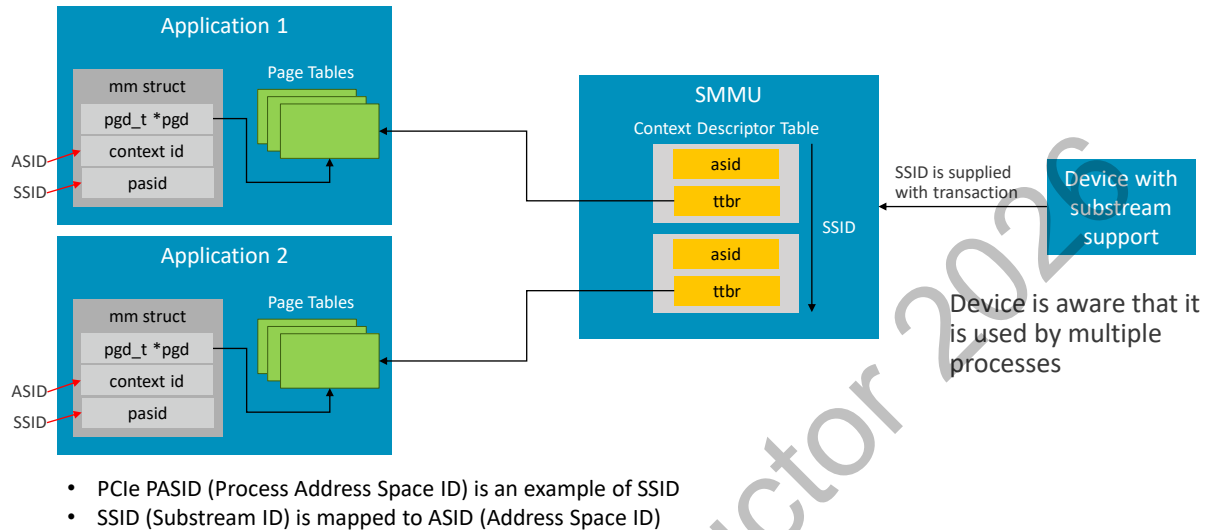
## Context Descriptor

**A Context Descriptor (CD) stores the settings related to stage 1 translation**



28 1462 rev 47356

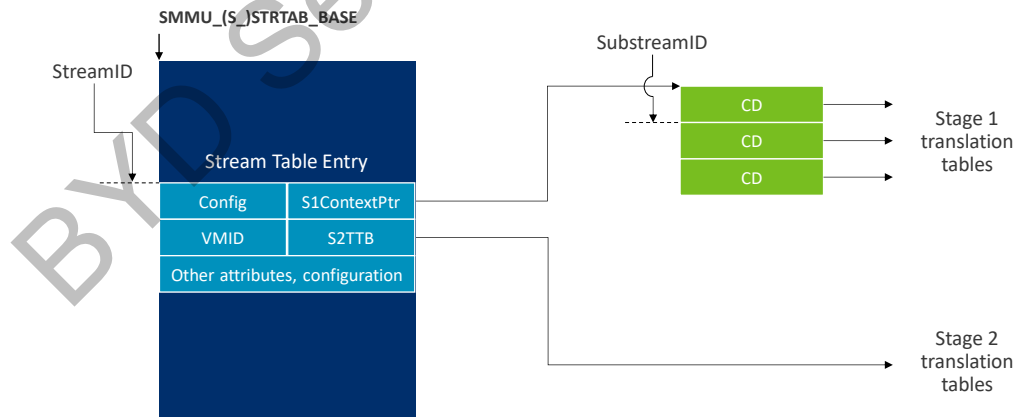
## Substream usecase



29 1462 rev 47356

29

## Context Descriptor with substreams enabled

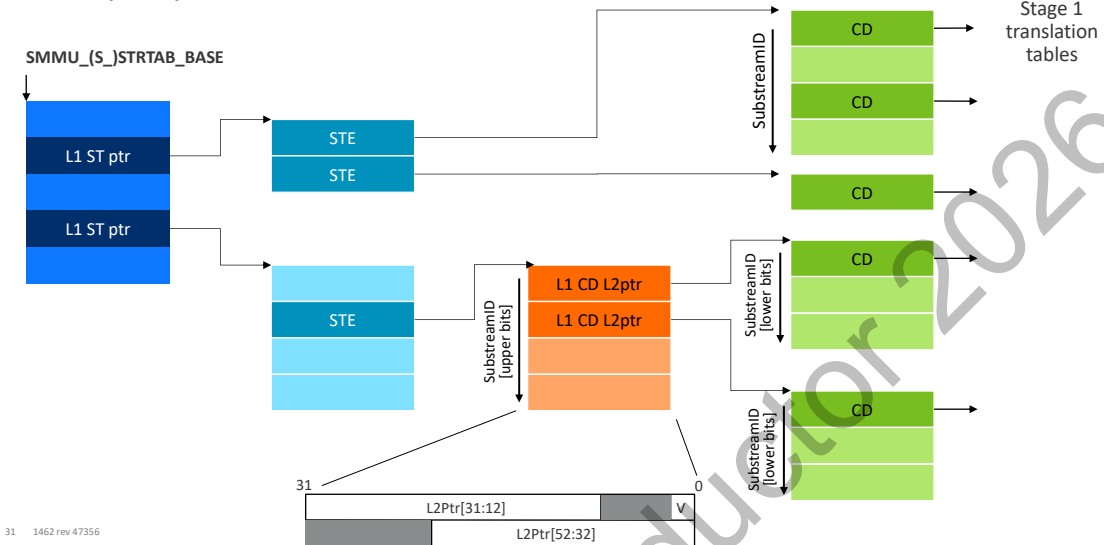


30 1462 rev 47356

30

## Multi-level Stream and CD tables

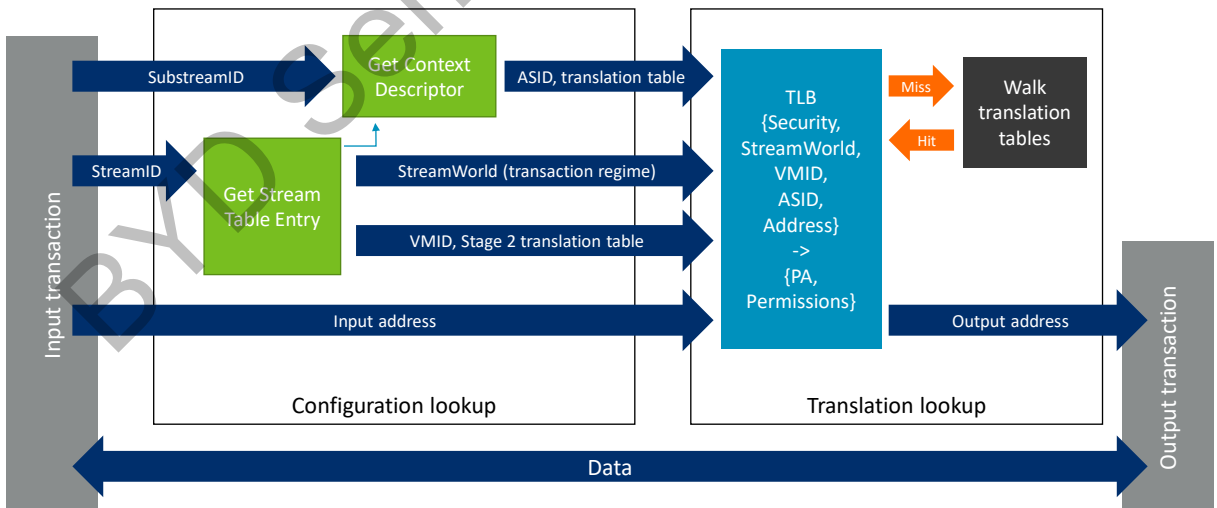
More complex layout: multi-level Stream table + multi-level CD table



31 1462 rev 47356

31

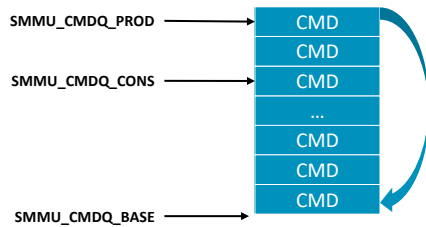
## Configuration and Translation lookup



32 1462 rev 47356

32

## Command Queue



The SMMU is controlled via a circular command queue in memory

- Separate Secure and Non-secure queues

### SMMU\_CMDQ\_BASE

- The base address and size of the command queue

### SMMU\_CMDQ\_PROD and SMMU\_CMDQ\_CONS

- The header and tail of the queue
- Software updates PROD after adding commands
- SMMU updates CONS when it has processed a command
- Software must check for space before adding new command

CONS being update indicates that a command has been consumed

- Does not necessarily mean the effects are visible
- The **CMD\_SYNC** command provides a synchronization mechanism
- WFE wake-up events can be generated. Used on the PE to wait for **CMD\_SYNC**

33 1462 rev 47356

33

## Command Queue - commands

### Type of commands:

#### TLB Invalidation commands

- **CMD\_TLBI\_<exception level>\_<tag>** : The command invalidates all the TLB entries at the given Exception level that match the tag (VMID, ASID, VA or All).

#### Configuration cache invalidation commands

- **CMD\_CFGI\_<STE/CD>\_<range>** : The command invalidates all the configuration cache entries within the specified range <All, RANGE> for a given StreamID (in case of STE) or both StreamID and SubstreamID (in case of CD).

#### Synchronization command

- **CMD\_SYNC** : The command provides a synchronization mechanism for preceding commands that were issued to same Command Queue as the **CMD\_SYNC**.

34 1462 rev 47356

34

# Caching of STEs, CDs and Translations

The SMMU is permitted to cache the settings in the STEs and CDs as well as translations

If software updates on these structures, it must invalidate any caching

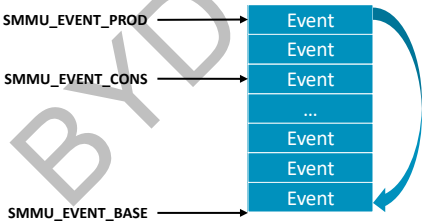
- For STE/CD changes, this is done via the Command Queue
- For translations, either the Command Queue or could receive TLB invalidates via broadcast

Example:

|                                                         |                                             |
|---------------------------------------------------------|---------------------------------------------|
| <code>CMD_CFGI_STE(&lt;StreamID&gt;, SSec, Leaf)</code> | Invalidate cache of STE                     |
| <code>CMD_SYNC(SIG_IRQ, &lt;MSI details&gt;)</code>     | Generate interrupt when invalidate complete |

35 1462 rev 47356

## Event Queue



The SMMU uses an Event queue to report information back to software

- Similar structure to the Command queue

Example events:

- CERROR\_ILL: Illegal or unrecognised command
- C\_BAD\_STREAMID: Transaction with an out of range StreamID
- C\_BAD\_CD: CD had Valid==0, or had invalid configuration

If the event queue overflows, records could be lost

- SMMU can generate an interrupt on new events being added to the queue

Merging

- Events for the same Stream can be merged into a single record
- Merging can be disabled using STE.MEV

36 1462 rev 47356

## Global Errors

The Global Error registers (SMMU\_(S\_)GERROR) are used to report some errors

- Command queue errors
  - Stops processing further commands until error is cleared
  - Event queue entry created as well, this gives the cause
- External aborts on Event queue accesses
- External aborts on writes due to Message Signalled Interrupts (MSI)

SMMU can be configured to generate an interrupt on detecting a global error

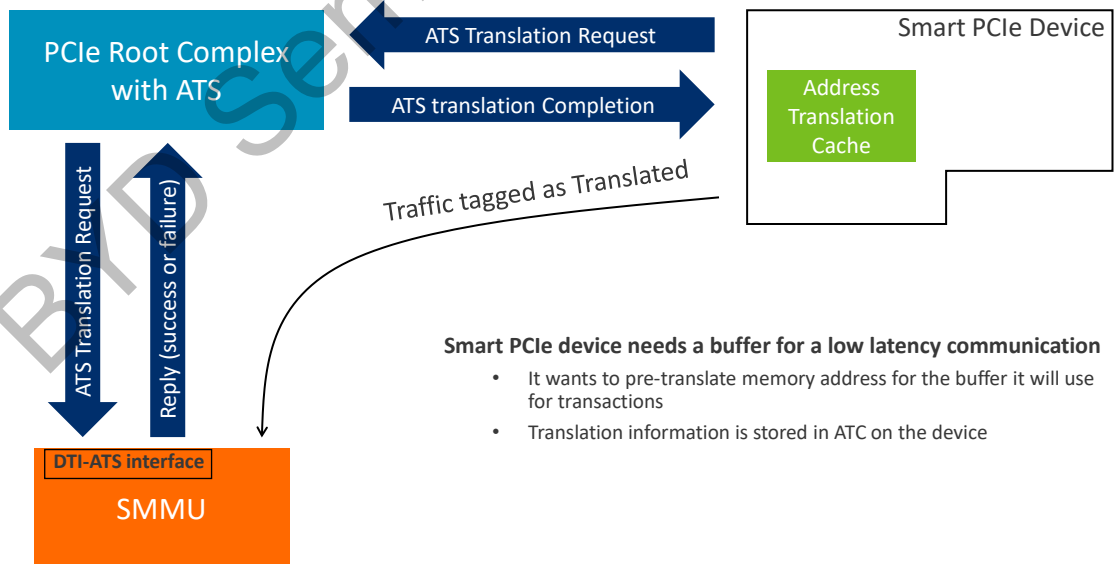
- Apart from if the error was due to an aborted MSI and the global error interrupt is configured to be sent to same address

Software clears an error by toggling the correspond bit in SMMU\_(S\_)GERRORN

37 1462 rev 47356

37

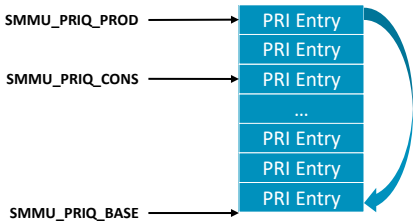
## PCIe ATS overview



38 1462 rev 47356

38

# PRI Queue



## Optional PRI queue to store the PRI messages received on the DTI-ATS interface

- Similar structure to the Event queue

### PRI Flow:

- PCIe master can issue a PRI request when an ATS request fails due to page not being present.
- SMMU stores the request in the PRI queue and increments SMMU\_PRIQ\_PROD.
- Software reads the PRI entry, makes the pages available and increments SMMU\_PRIQ\_CONS .
- Software can issue CMD\_PRI\_RESP (using the command queue) to send a PRI response to the endpoint to indicate success or failure of the PRI request.
- DTI\_ATS\_PAGE\_RESP is not guaranteed by the DTI protocol. It must be guaranteed by the software operation by issuing CMD\_PRI\_RESP command.

39 1462 rev 47356

arm

Thank You  
Danke  
Gracias  
謝謝  
ありがとう  
Asante  
Merci  
감사합니다  
धन्यवाद  
Kiitos  
شكراً  
ধন্যবাদ  
תודה

# Appendix

Confidential © 2025 Arm

arm

41

## Check-List for Minimum Configuration

### Allocate Stream Table

- Allocate memory for Stream Table, and initialize the memory (at least the Valid bits)
- SMMU\_STRTAB\_BASE\_CFG and SMMU\_STRTAB\_BASE

### Allocate Command and Event queues

- Allocate memory for the command queue and event queue
- SMMU\_CMDQ\_BASE, SMMU\_CMDQ\_PROD and SMMU\_CMDQ\_CONS
- SMMU\_EVENT\_BASE, SMMU\_EVENT\_PROD and SMMU\_EVENT\_CONS

### Set memory attributes for accessing Stream Table, Command Queue and Event Queue

- SMMU\_CR1

### Enable IRQs for Event, PRI and GERROR

### Setup Global Bypass Attribute register (SMMU\_GBPA) by writing all zeroes

### Enable SMMU

- SMMU\_CR0.SMMUEN, SMMU\_CR0.EVENTQEN and SMMU\_CR0.CMDQEN

42 1462 rev 47356

42



## Appendix: Embedded Configuration

The SMMUv3 specification allows for the registers that point to the data structures in memory to be read-only

- In this such a design, the address pointed at might actually be registers instead of memory

45 1462 rev 47356

45

## Appendix: ATOS

The SMMU has an optional translation operations, ATOS

- Allows software to query translations in the SMMU, similar to AT instruction in Armv8-A
- Supported reported by SMMU\_IDR0.ATOS

When two Security states are implemented, there are separate Secure and Non-secure ATOS registers

- Operate independently of each other
- Secure ATOS registers can operate on Secure or Non-secure streams
- Non-secure ATOS registers can only operate on Non-secure streams

An optional Virtual ATOS (VATOS) can perform stage 1 only translations

- Typically mapped into a VM's address space, and used by VM like the regular ATOS

46 1462 rev 47356

46

## Using ATOS

### The Secure, Non-secure and Virtual ATOS all have the same registers

- The specification often generically refers to **ATOS\_\*** instead of **SMMU\_S\_GATOS\_\***, **SMMU\_VATOS\_\***, etc...

### To perform a translation:

- Set **ATOS\_ADDR** to the input address to be translated and the type of translation
- Set **ATOS\_SID** to the StreamID of the translation to be used
  - For Secure ATOS, also sets whether the StreamID is Secure or Non-secure
- Set **ATOS\_CTLR.RUN** to 1 to start the translation
- Poll **ATOS\_CTLR.RUN** until reads 0, this signals the translation has completed
- The result of the translation is returned in **ATOS\_PAR**

For VATOS, **SMMU\_VATOS\_SEL.VMID** is set by the hypervisor to limit VM address translations operations to only its own address space

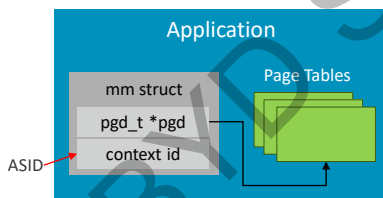
47 1462 rev 47356

47

## Shared Virtual Addressing for the IOMMU

Shared Virtual addressing (SVA) – share process address spaces with devices

Application can instruct a device to perform DMA on buffers obtained with malloc



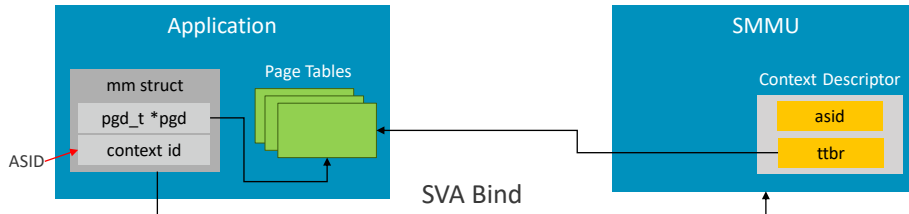
48 1462 rev 47356

48

## Shared Virtual Addressing for the IOMMU

Shared Virtual addressing (SVA) – share process address spaces with devices

Application can instruct a device to perform DMA on buffers obtained with malloc



Arm SMMU driver implements `iommu_ops.sva_bind` with `arm_smmu_sva_bind()` method:

```
cd->ttbr = virt_to_phys(mm->pgd);  
asid = arm64_mm_context_get(mm);  
cd->asid = asid;
```

By doing SVA bind we also register a callback for SMMU TLB invalidate in Linux `mm_notifier`

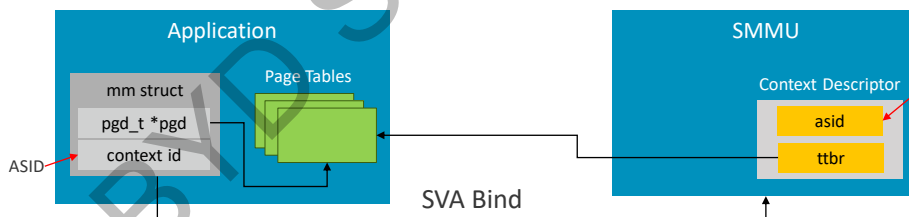
49 1462 rev 47356

49

## Shared Virtual Addressing for the IOMMU

Shared Virtual addressing (SVA) – share process address spaces with devices

Application can instruct a device to perform DMA on buffers obtained with malloc



Peripheral device  
doesn't know ASID.  
ASID is not a part of the  
transaction. Substream  
support is a must!

Arm SMMU driver implements `iommu_ops.sva_bind` with `arm_smmu_sva_bind()` method:

```
cd->ttbr = virt_to_phys(mm->pgd);  
asid = arm64_mm_context_get(mm);  
cd->asid = asid;
```

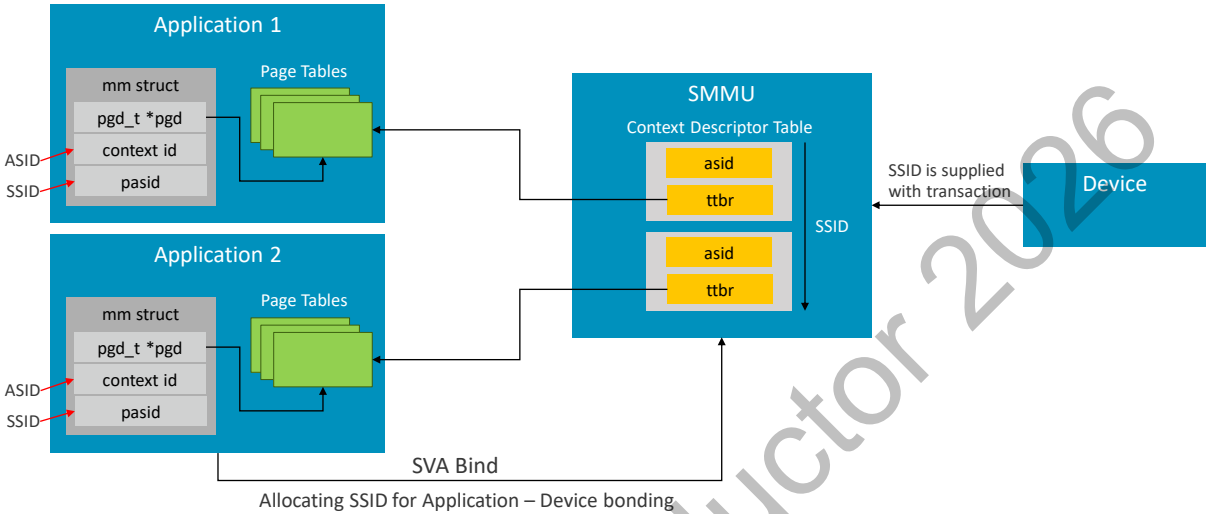
By doing SVA bind we also register a callback for SMMU TLB invalidate in Linux `mm_notifier`

50 1462 rev 47356

50

# SVA and SSID

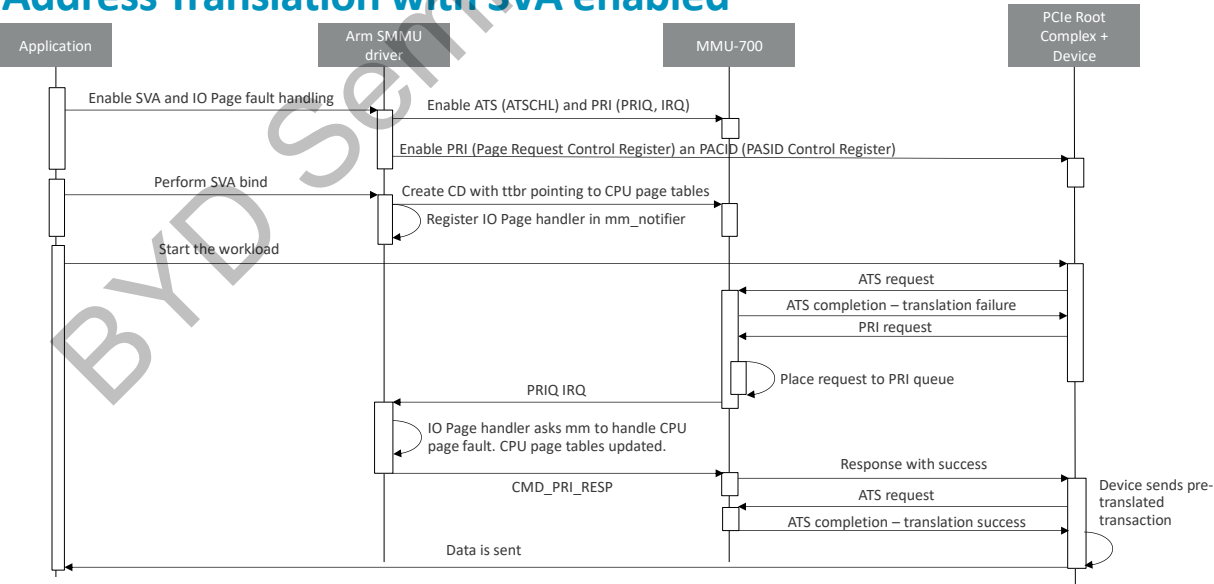
SSID (PASID) hardware support is required for SVA



51 1462 rev 47356

51

# Address Translation with SVA enabled



52 1462 rev 47356

52

# arm

## MMU S3

Introduction and topology

Confidential © 2024 Arm

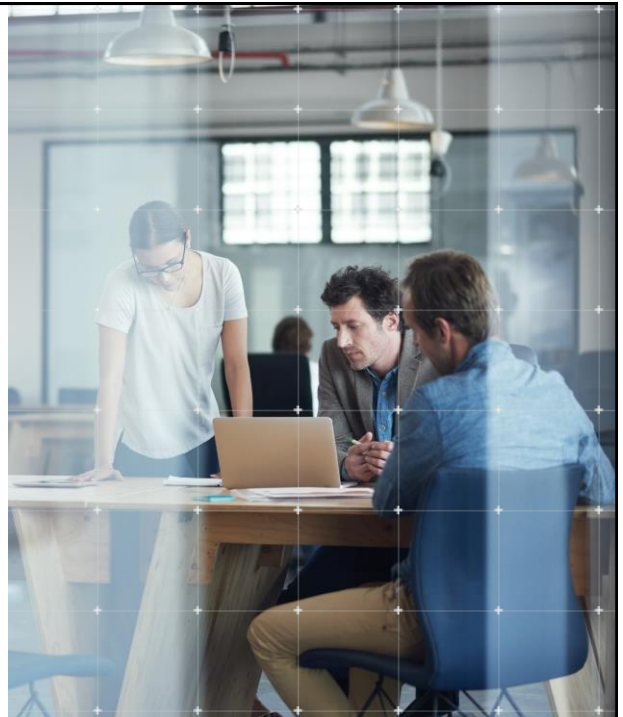
3

### Objectives

At the end of this module you will be able to:

- Explain the difference between a processor MMU and a System MMU
- Recall the key architectural features implemented by MMU S3
- Demonstrate where MMU S3 could be located in an example system
- Explain the meaning of key SMMU terms and acronyms
- Demonstrate the topological options available for MMU S3 integration
- Describe MMU S3 support for PCIe ATS
- Explain the security advantages of DPT
- Understand how a system can be designed to support PCIe bifurcation

4 xxxx rev 00000



4

## Agenda

### Introduction

- What is a Memory Management Unit?
- What is a System Memory Management Unit?
- MMU S3 overview
- MMU S3 example system

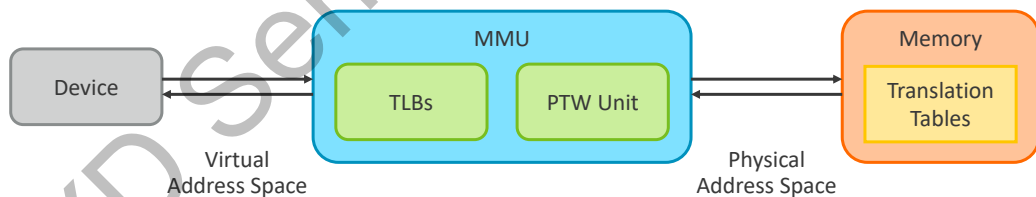
### Topology

- MMU S3 terminology
- MMU S3 components
- Integration
  - Inline integration
  - Cache integration
  - Device Permission Table (DPT)
  - Lookaside integration
- ACE-Lite bifurcation example
- LTI bifurcation example

5 xxxx rev 00000

5

## What is a Memory Management Unit?



### A Memory Management Unit (MMU) controls how memory is accessed

- Address translation
  - Reads **translation tables** from memory (page table walks) to map virtual addresses to physical addresses
  - Uses translation lookaside buffers (TLBs) to cache recent translations
- Access permissions
  - Checks **memory access permissions** to provide memory protection
  - Achieved with translation tables, completer-side filtering or granule protection checks (GPCs)
- Memory attribute conversion
  - Converts **memory attributes** to determine how memory accesses are handled by the rest of the system

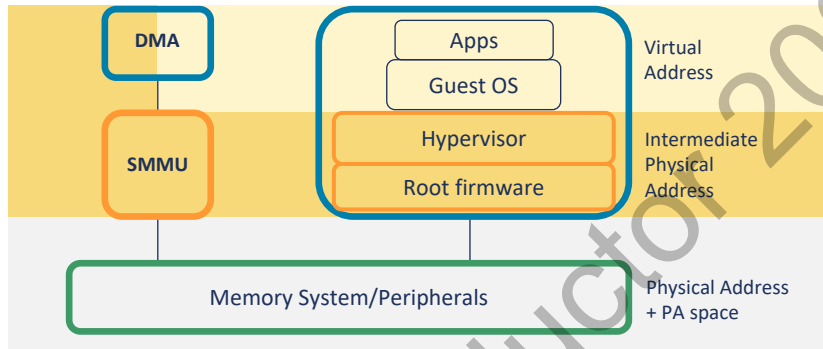
6 xxxx rev 00000

6

## What is a System Memory Management Unit?

### A System MMU (SMMU) performs address translation for system managers

- Stand-alone MMU that can be placed downstream of another device
  - Uses the same translation table formats as the applications processor (VMSA)
- An SMMU can perform **stage 1** translation only, **stage 2** translation only, or **both stages** of translation (nested)
- An SMMU can optionally perform **granule protection checks (GPCs)** on translated addresses
  - Controlled by Root firmware



7 xxxxx rev 00000

7

## MMU S3 overview

### MMU S3 is an implementation of three architectures defined by the SMMUv3 Architecture Specification

- SMMUv3.3
- SMMU for RME
- SMMU for RME DA

### Supports stage 1 and stage 2 address translations

- Stage 1: **Virtual Address (VA)** -> **Intermediate Physical Address (IPA)**
  - Operating system creates a page table set for each process, giving each process a unique virtual address space
  - IPA to PA mapping is flat if only a single stage of translation is enabled (no hypervisor is present)
- Stage 2: **Intermediate Physical Address (IPA)** -> **Physical Address (PA)**
  - Hypervisor creates page tables for each Guest Operating System
  - Each OS is unaware of the underlying hypervisor and acts as if it has direct access to hardware (transparent virtualization)

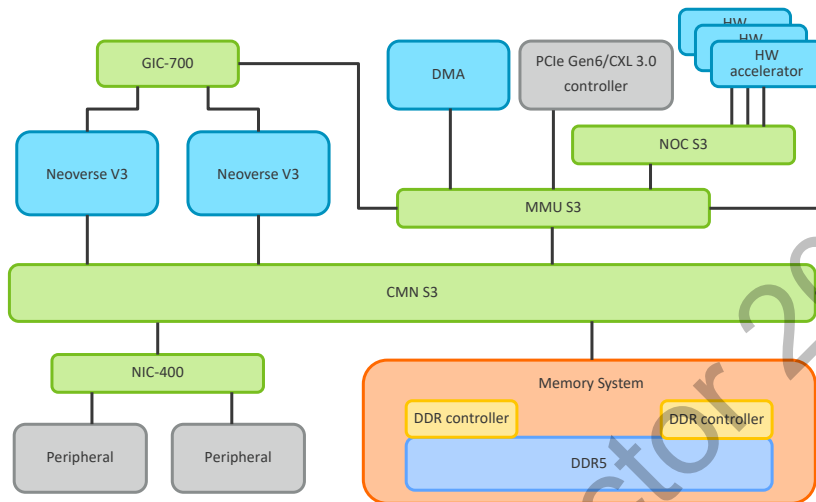
### Enables the Arm Confidential Compute Architecture (CCA)

- Armv9.2 VMSA compliant
- Granule protection checks (GPCs)
- Realm translations (including Device Permission Table (DPT) support from MMU S3 r1)
- Memory encryption contexts (MEC)

8 xxxxx rev 00000

8

## MMU S3 example system



9 xxxx rev 00000

9

## Recap question

1. Which of these is NOT a function of a Memory Management Unit?

- a) Address translation
- b) Control of access permissions
- c) Interrupt handling
- d) Memory attribute conversion

10 xxxx rev 00000

10

## Agenda

### Introduction

- What is a Memory Management Unit?
- What is a System Memory Management Unit?
- MMU S3 overview
- MMU S3 example system

### Topology

- MMU S3 terminology
- MMU S3 components
- Integration
  - Inline integration
  - Cache integration
  - Device Permission Table (DPT)
  - Lookaside integration
- ACE-Lite bifurcation example
- LTI bifurcation example

11 xxxx rev 00000

11

## MMU S3 terminology

### Translation Buffer Unit (TBU)

- Contains **Translation Lookaside Buffers (TLBs)** for caching translation information

### Translation Control Unit (TCU)

- Performs **Configuration and Translation Table Walks (CTW/PTW)** to memory to retrieve translation information
- Performs **Granule Protection Table (GPT) Walks** to memory to retrieve granule protection information

### Bidirectional AXI5-Stream (BAS) interconnect

- Connects multiple TBUs and PCIe Root Ports to a single TCU

### Distributed Translation Interface (DTI)

- Message-based protocol for communication between TBUs, TCUs, and PCIe Root Ports
- DTI messages are carried over the physical **BAS protocol** in MMU S3

### Local Translation Interface (LTI)

- Channel-based protocol for sending translation requests between tightly-coupled SMMU components
- Translation responses must be used immediately and cannot be cached

12 xxxx rev 00000

12

## MMU S3 components

### Translation Buffer Unit (TBU)

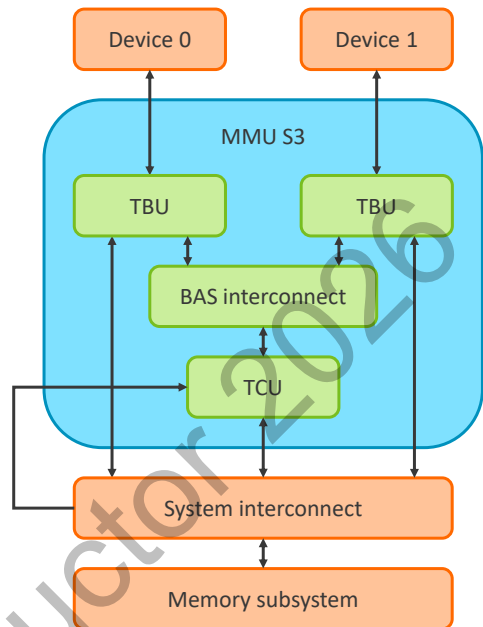
- Bus Interface Unit (BIU)
- Translation Lookaside Buffer Unit (TLBU)

### Translation Control Unit (TCU)

- Translation Management Unit (TMU)
- Programming Interface Unit (PIU)

### Bidirectional AXIS-Stream (BAS) interconnect

- BAS sizer
- BAS switch
- BAS register slice



13 xxxx rev 00000

13

## Integration

### MMU S3 supports three options for integration with an I/O device

- Inline integration
  - All transactions from the device pass through the SMMU and are translated **on the fly**
- Cached integration
  - The device can request translations in advance and store them in an **address translation cache (ATC)**
  - All transactions pass through the SMMU, but translated transactions do not need to be translated again
- Lookaside integration
  - The device requests translations from the SMMU, and the SMMU responds with translation information
  - The device uses translations from the SMMU to issue translated accesses directly to the system
  - Also known as **Translation-as-a-Service**

### All options require each SMMU component to be configured and integrated separately

- Different integration options can coexist within the same system for different devices

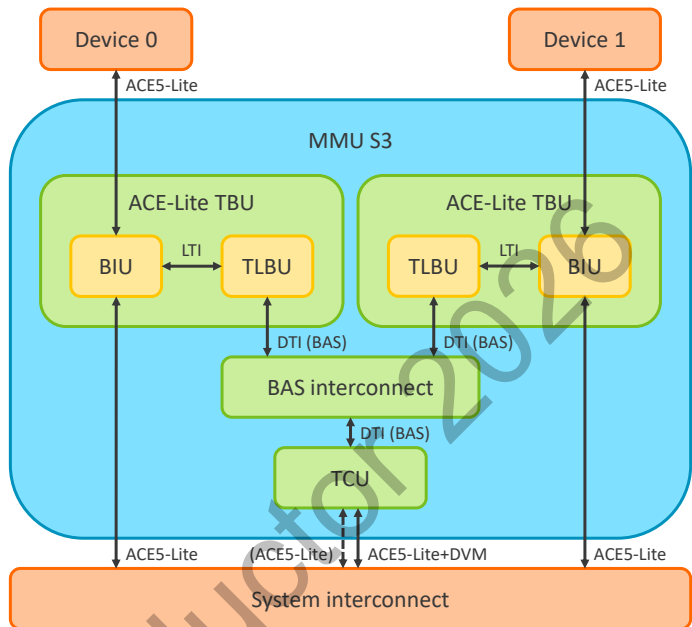
14 xxxx rev 00000

14

## Inline integration

### Simplest integration option

- All transactions pass through the SMMU
- ACE-Lite TBUs contain a **BIU**
- Devices are not aware of underlying translation



15 xxxx rev 00000

15

## Cached integration

### Provides support for PCIe ATS

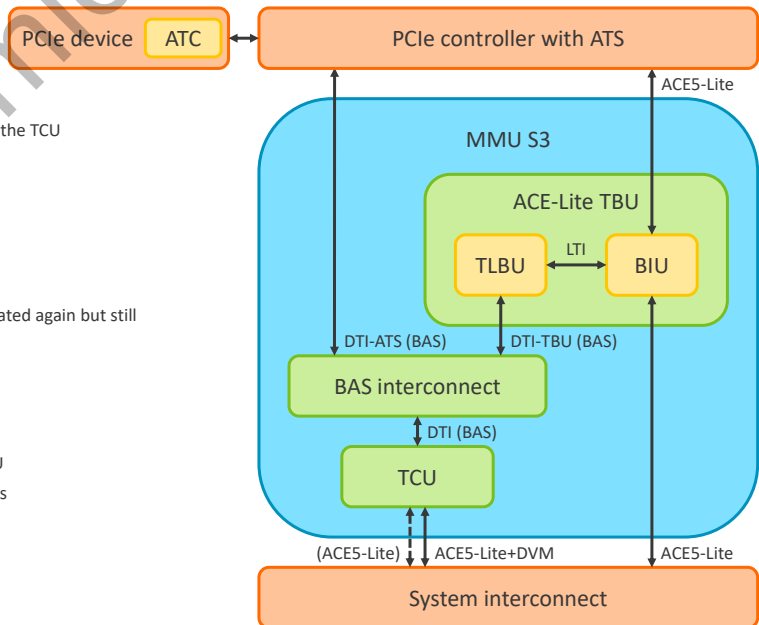
- PCIe device can request translations directly from the TCU
- Translations are cached locally within an ATC
- All transactions pass through the SMMU

### Full ATS

- Stage 1 + Stage 2 translation cached in ATC
- Translated transactions from device are not translated again but still undergo security checks

### Split-stage ATS

- Only Stage 1 translation cached in ATC
- Stage 2 translation performed on the fly by SMMU
- Provides greater security against untrusted devices



16 xxxx rev 00000

16

## Device Permission Table (DPT)

A future MMU S3 r1 feature to enhance PCIe ATS security for Realms

### Full ATS has security limitations

- GPC enforces the correct physical address space
- Realms are protected from a rogue hypervisor
- Realms are not protected **from one another**

### Split-stage ATS has functional limitations

- Devices cannot be **fully coherent**
- Peer-to-peer PCIe ACS traffic still requires **stage 2 translation**

### DPT combines the security of split-stage ATS with the flexibility of full ATS

- Translated transactions are subject to **DPT checks**, but do not need to undergo a whole stage of translation
- DPT checks ensure the StreamID of the translated transaction is permitted to access the target address
- A DPT is a lookup table in memory that specifies the **permitted VMID** for each physical memory region
- DPT entries can be cached

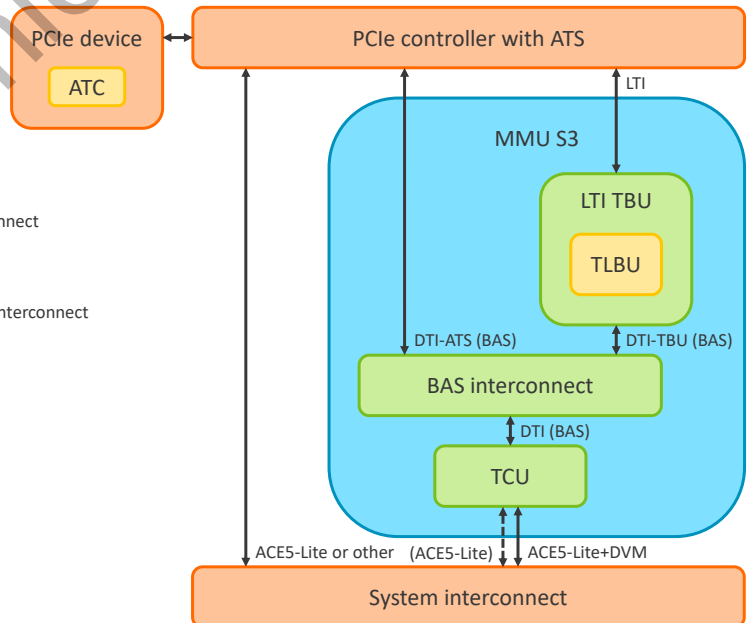
17 xxxx rev 00000

17

## Lookaside integration

### Translation-as-a-Service

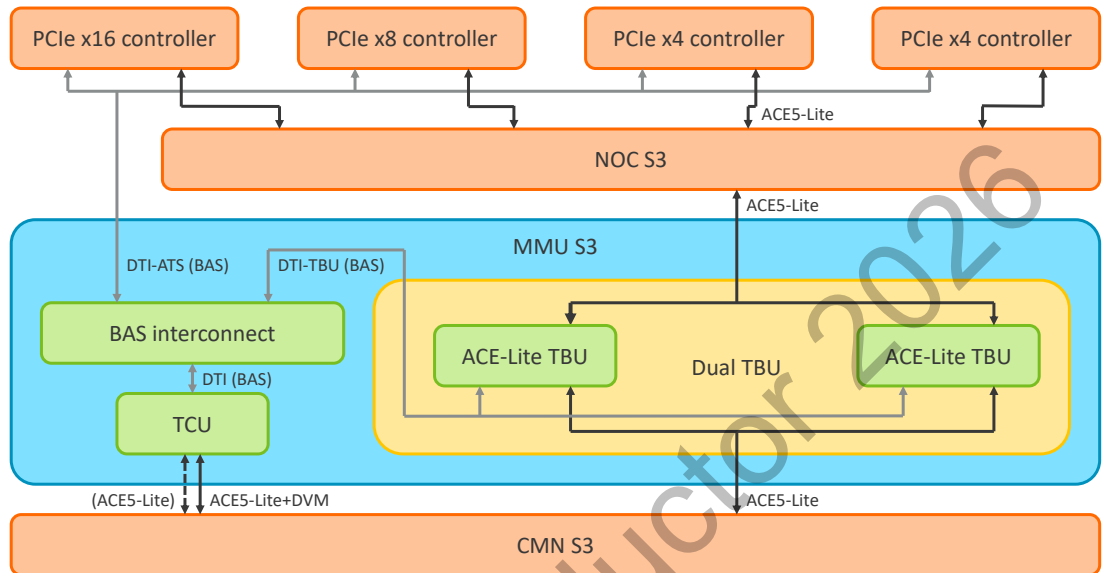
- Devices request translations directly from the SMMU
- The SMMU responds with translation information
- Transactions do not pass through the SMMU
- PCIe controller can connect directly to system interconnect
- PCIe controller can maintain its own ordering rules
- No write data buffer provides area savings
- Peer-to-peer PCIe ACS traffic does not need to enter interconnect



18 xxxx rev 00000

18

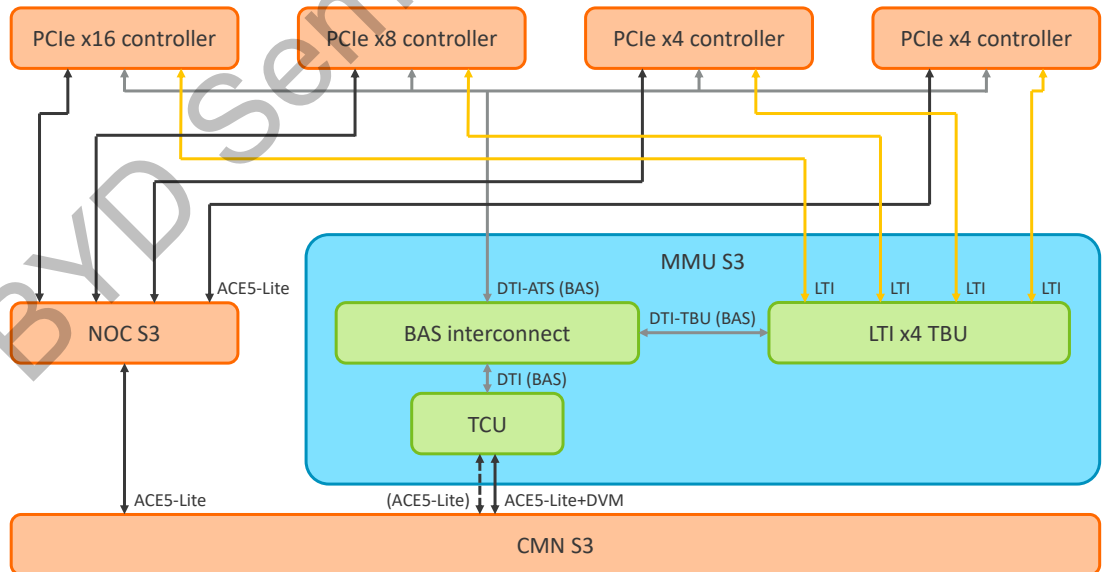
## ACE-Lite bifurcation example



19 xxxx rev 00000

19

## LTI bifurcation example



20 xxxx rev 00000

20

## Recap questions

1. Which of these performs Granule Protection Table (GPT) walks to memory to retrieve granule protection information?
  - a) TBU
  - b) TCU
  - c) BAS interconnect
2. Which integration option uses the LTI-TBU instead of the ACE-Lite TBU?
  - a) Inline integration
  - b) Cached integration
  - c) Lookaside integration

21 xxxx rev 00000

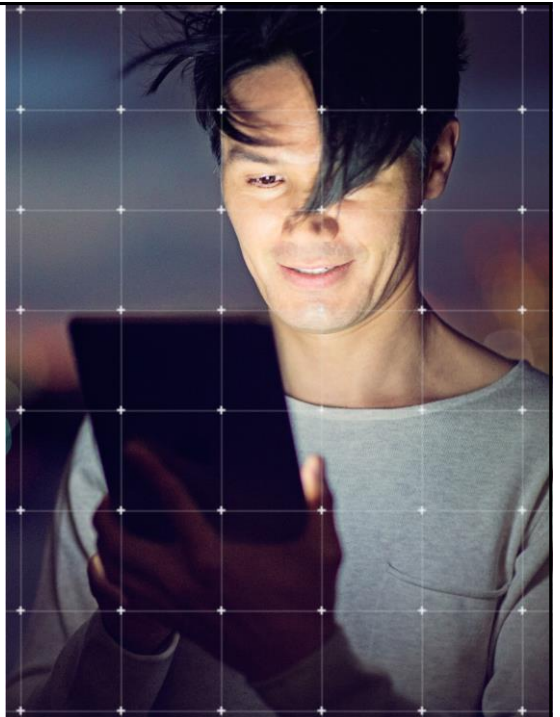
21

## Module summary

In this module we have covered

Introduction  
Topology

22 xxxx rev 00000



22

arm

Thank You

Danke

Gracias

谢谢

ありがとう

Asante

Merci

감사합니다

धन्यवाद

Kiitos

شكراً

ধন্যবাদ

תודה

Confidential © 2024 Arm

# arm

## MMU S3

Interfaces

Confidential © 2024 Arm

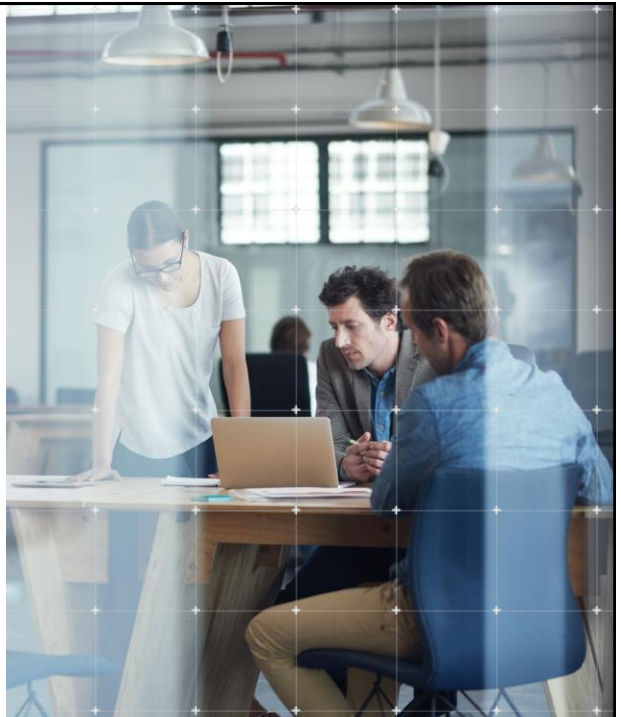
2

### Objectives

At the end of this module you will be able to:

- Describe the global interfaces present on all MMU S3 components
- Recall the interfaces of the ACE-Lite TBU, LTI TBU and TCU
- Understand the use case for AMBA properties implemented by MMU S3
- Explain the MMU S3 power and clock control requirements
- Describe the MMU S3 TBU and TCU interrupts
- Explain the difference between interrupts and events
- Explain how a PMU snapshot can be performed
- Recall the DTI message groups
- Describe which DTI versions are supported by MMU S3
- Demonstrate how BAS interconnect components could be integrated
- Explain how to configure the BAS switch component

3 xxxxx rev 00000



3

## Agenda

### Interfaces

- Global interfaces
- ACE-Lite TBU
  - ACE-Lite TBU interfaces
  - ACE-Lite TBU interface properties
  - ACE-Lite TBU AxiMMUVALID signal
  - ACE-Lite TBU AxiUSER signals
- LTI TBU
  - LTI TBU interfaces
  - LTI TBU interface properties
- TCU
  - TCU memory system interfaces
  - TCU other interfaces
  - TCU interface properties
- LPI\_PD interface considerations

- Interrupts
  - TCU interrupts
  - TCU interrupts
  - Interrupt signalling
- Event signalling
- Performance monitoring interfaces
- AMBA interface summary

### DTI protocol and BAS interconnect

- DTI protocol
- DTI flow control
- BAS interconnect
- BAS switch configuration

4 xxxxx rev 00000

4

## Global interfaces

### Clocks

- All MMU S3 components have individual **clk** inputs
- ADB-400 can act as a bridge between components in different **clock** or **power domains**

### Resets

- All MMU S3 components have individual **resetsn** inputs
- TBUs and TCU have internal **reset synchronization**
- BAS interconnect components must have their resets synchronized externally

### Low Power Interfaces (LPIs)

- All MMU S3 components except the BAS switch have Q-Channel LPIs
  - LPI\_CG interface for clock gating
  - LPI\_PD interface for power down/up (TBU and TCU only)
- LPD-500 can connect a single clock or power controller to multiple MMU S3 components

5 xxxxx rev 00000

5

## ACE-Lite TBU interfaces

### ACE5-Lite TBS interface

- For incoming untranslated transactions

### ACE5-Lite TBM interface

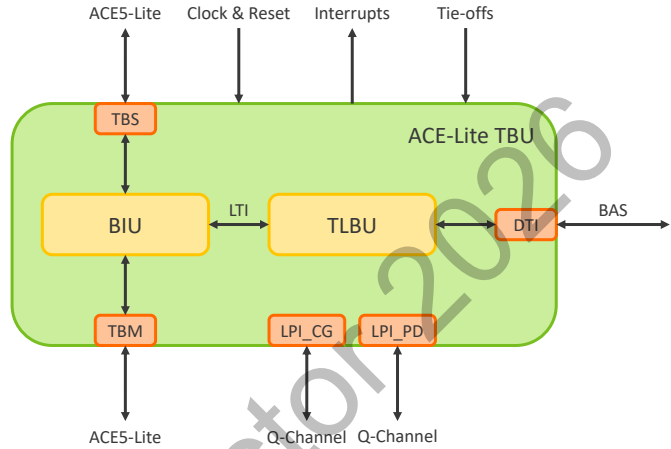
- For outgoing translated transactions

### DTI interface (BAS)

- For communication with the TCU

### Q-Channel LPIs

- For controlling clock gating
- For controlling power down/up



6 xxxx rev 00000

6

## ACE-Lite TBU interface properties (1/3)

The TBS and TBM interfaces include support for the following ACE5-Lite properties

| Property                                                                    | Signal                       | Description                                                |
|-----------------------------------------------------------------------------|------------------------------|------------------------------------------------------------|
| Untranslated_Transactions v3                                                | <b>AxMMUSECSID</b>           | Security state of the stream                               |
|                                                                             | <b>AxMMUSID</b>              | StreamID -> Indexes STEs (PCIe Requester ID)               |
|                                                                             | <b>AxMMUSSID (TBS only)</b>  | SubstreamID -> Indexes CDs (PCIe PASID)                    |
|                                                                             | <b>AxMMUSSIDV (TBS only)</b> | SubstreamID is valid                                       |
|                                                                             | <b>AxMMUFLOW (TBS only)</b>  | Flow for managing translation faults                       |
|                                                                             | <b>RRESP[2] / BRESP[2]</b>   | Additional bit for signalling a translation fault response |
|                                                                             | <b>AxMMUVALID</b>            | Controls per-transaction physical bypass of the SMMU       |
| Cache_Stash_Transactions (enabled by <b>TBUCFG_STASH_SUPPORT</b> parameter) | <b>AWSNOP[3]</b>             | Encodes the type of cache stash transaction                |
|                                                                             | <b>AWSTASHNID</b>            | Node ID of cache stash target                              |
|                                                                             | <b>AWSTASHNIDEN</b>          | Node ID is valid                                           |
|                                                                             | <b>AWSTASHLPID</b>           | Logical Processor ID of cache stash target                 |
|                                                                             | <b>AWSTASHLPIDEN</b>         | Logical Processor ID is valid                              |

7 xxxx rev 00000

7

## ACE-Lite TBU interface properties (2/3)

The TBS and TBM interfaces include support for the following ACE5-Lite properties

| Property                  | Signal                         | Description                                                                               |
|---------------------------|--------------------------------|-------------------------------------------------------------------------------------------|
| DeAllocation_Transactions | (No new signals)               | Support for deallocating read transactions                                                |
| Atomic_Transactions       | <b>AWATOP</b>                  | Encodes the type of atomic transaction                                                    |
| Loopback_Signals          | <b>AxLOOP</b>                  | Loopback value for transaction tracking                                                   |
| Poison                    | <b>xPOISON</b>                 | Indicates the data in this transfer has been corrupted                                    |
| Unique_ID_Support         | <b>AxIDUNQ</b> / <b>xIDUNQ</b> | Indicates that no other transactions are currently in progress with the same AXI ID value |
| Read_Data_Chunking        | <b>ARCHUNKEN</b>               | Read data chunking enabled                                                                |
|                           | <b>RCHUNKV</b>                 | Read data chunk number and strobe valid                                                   |
|                           | <b>RCHUNKNUM</b>               | Read data chunk number                                                                    |
|                           | <b>RCHUNKSTRB</b>              | Read data chunk strobe                                                                    |
| CMO_On_Read, Persist_CMO  | (No new signals)               | Support for cache maintenance operations (CMOs)                                           |
| Wakeup_Signals            | <b>AWAKEUP</b>                 | Activity present on the TBM interface – drive QACTIVE                                     |

8 xxxxx rev 00000

8

## ACE-Lite TBU interface properties (3/3)

The TBS and TBM interfaces include support for the following ACE5-Lite properties

| Property                   | Signal                   | Description                                                                    |
|----------------------------|--------------------------|--------------------------------------------------------------------------------|
| MPAM_Support               | <b>AxMPAM</b> (TBM only) | Memory partitioning and monitoring (MPAM) information                          |
| MTE_Support (Basic)        | <b>AxTAGOP</b>           | Tag operation                                                                  |
|                            | <b>xTAG</b>              | Tag value                                                                      |
|                            | <b>WTAGUPDATE</b>        | Strobe to indicate which tags must be written                                  |
| InvalidateHint_Transaction | <b>AWSNOOP</b> [ 4 ]     | AWSNOOP extended to encode the InvalidateHint opcode                           |
| PBHA_Support               | <b>AxPBHA</b>            | Page-based hardware attributes                                                 |
| RME_Support                | <b>AxNSE</b>             | Combines with <b>AxPROT</b> [ 1 ] to encode Realm and Root security attributes |
| MEC_Support                | <b>AxMECID</b>           | MECID value                                                                    |

9 xxxxx rev 00000

9

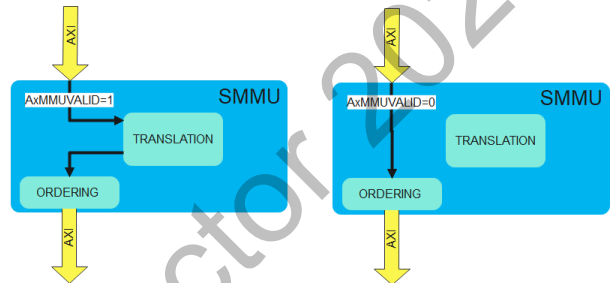
## ACE-Lite TBU AxMMUVALID signal

Allows trusted devices to issue transactions through the SMMU without translation

- Physical bypass of the architectural SMMU translation process
- No attribute conversion (unlike Global Bypass and Stream Bypass)
- GPC is still performed
- Path to memory can be shared with agents that do not need translation
- This is called a **NoStreamID** transaction

### Operation

- If **AxMMUVALID** == 1, translation is possible
- If **AxMMUVALID** == 0, all SMMUv3 translation is bypassed
  - The incoming transaction must have a physical address
  - All other **AxMMU\*** signals are invalid
- If **LAMMUV** == 1, translation is possible
- If **LAMMUV** == 0, all SMMUv3 translation is bypassed
  - The incoming transaction must have a physical address
  - Only **LA\*** transport and control signals are valid



10 xxxx rev 00000

10

## ACE-Lite TBU AxUSER signals

### TBS interface AxUSER bits (MSB to LSB)

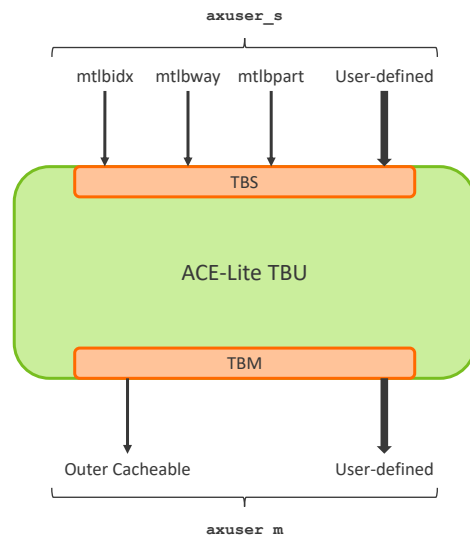
- TLBLOC attributes
  - mtlbidx (MTLB index for MTLB direct indexing)
  - mtlbway (MTLB way for MTLB direct indexing)
  - mtlbpart (MTLB partition for MTLB partitioning)
- User-defined bits

### TBM interface AxUSER bits (MSB to LSB)

- Outer Cacheable bit – can be used as a hint for which caches to allocate into
- User-defined bits

### AxUSER signal widths are variable

- TLBLOC attributes depend on **TBUCFG\_DIRECT\_IDX** and **TBUCFG\_MTLB\_PARTS** parameters
- User-defined bits depend on **TBUCFG\_AxUSER\_WIDTH** parameter



11 xxxx rev 00000

11

## LTI TBU interfaces

### LTI interface

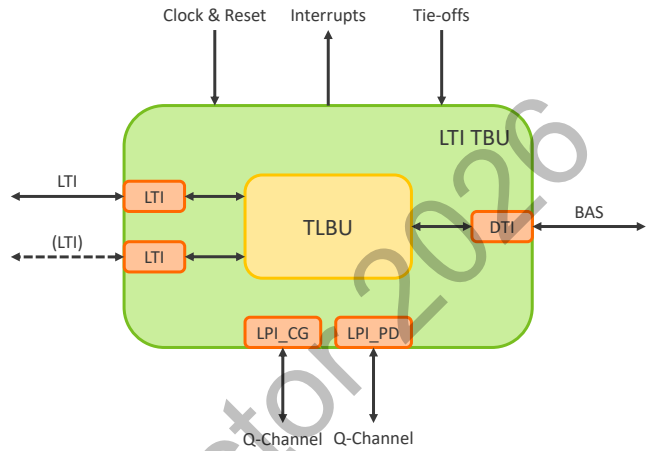
- For communication with an LTI device
  - PCIe Root Port
  - Other LTI I/O
- Up to 8 LTI interfaces are supported

### DTI interface (BAS)

- For communication with the TCU

### Q-Channel LPIs

- For controlling clock gating
- For controlling power down/up



12 xxxx rev 00000

12

## LTI TBU interface properties

### Two virtual channels supported

- One virtual channel for non-posted transactions (reads)
- One virtual channel for posted transactions (writes)

### User signals are not implemented

Other LTI interface parameters are determined by LTI TBU configuration parameters

| LTI interface parameter | LTI TBU configuration parameter         |
|-------------------------|-----------------------------------------|
| LTI_ID_WIDTH            | TBUCFG_ID_WIDTH                         |
| LTI_SID_WIDTH           | TBUCFG_SID_WIDTH                        |
| LTI_OG_WIDTH            | TBUCFG_LTI_OG_WIDTH                     |
| LTI_TLBLOC_WIDTH        | TBUCFG_DIRECT_IDX and TBUCFG_MTLB_PARTS |
| LTI_LOOP_WIDTH          | TBUCFG_LTI_LOOP_WIDTH                   |
| LTI_MECID_WIDTH         | TBUCFG_MECID_WIDTH                      |

13 xxxx rev 00000

13

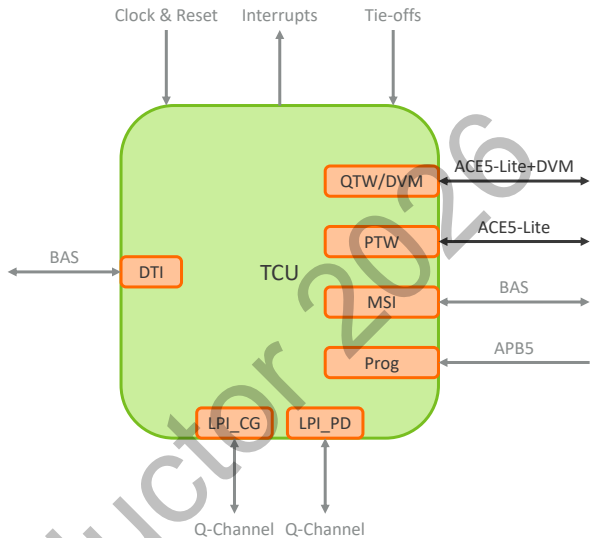
## TCU memory system interfaces

### ACE5-Lite+DVM QTW/DVM interface

- Configuration table walks (CTWs)
- Translation/page table walks (PTWs)
- Hardware translation table updates (HTTU)
- Virtual machine structure (VMS) fetches
- Granule protection table (GPT) walks
- Device permission table (DPT) walks
- Command queue reads (Secure/Non-secure/Realm)
- Event queue writes (Secure/Non-secure/Realm)
- PRI queue writes (Non-secure/Realm)
- DVM TLB invalidate messages
- Message-signalled interrupts (MSIs)

### ACE5-Lite PTW interface

- Optional dedicated interface for **translation table walks** and **HTTU**
- Only for systems that have very high PTW bandwidth requirements



14 xxxx rev 00000

14

## TCU other interfaces

### DTI interface (BAS)

- For communication with TBUs and PCIe Root Ports

### MSI interface (BAS)

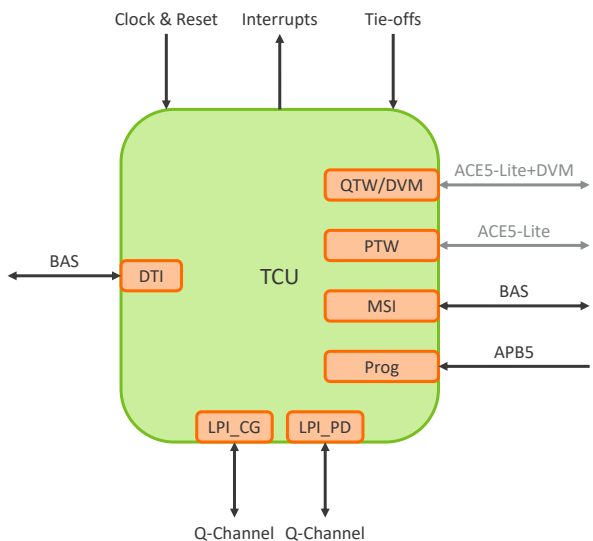
- For **message-signalled interrupts (MSIs)** generated by the TCU

### APB5 programming interface

- For programming the TBU and TCU registers
- Access to Root Control Page for GPT control and invalidation

### Q-Channel LPIs

- For controlling clock gating
- For controlling power down/up



15 xxxx rev 00000

15

## TCU interface properties (1/2)

The QTW/DVM and PTW interfaces include support for the following ACE5-Lite properties

| Property            | Signal                | Description                                                                               |
|---------------------|-----------------------|-------------------------------------------------------------------------------------------|
| Atomic_Transactions | <b>AWATOP</b>         | Encodes the type of atomic transaction                                                    |
| Poison              | <b>xPOISON</b>        | Indicates the data in this transfer has been corrupted                                    |
| Unique_ID_Support   | <b>AxIDUNQ/xIDUNQ</b> | Indicates that no other transactions are currently in progress with the same AXI ID value |
| Wakeup_Signals      | <b>AWAKEUP</b>        | Activity present on the QTW/DVM or PTW interface                                          |
| MPAM_Support        | <b>AxMPAM</b>         | Memory partitioning and monitoring (MPAM) information                                     |
| PBHA_Support        | <b>AxPBHA</b>         | Page-based hardware attributes                                                            |
| RME_Support         | <b>AxNSE</b>          | Combines with <b>AxPROT</b> [1] to encode Realm and Root security attributes              |
| MEC_Support         | <b>AxMECID</b>        | MECID value                                                                               |

16 xxxx rev 00000

## TCU interface properties (2/2)

The QTW/DVM interface additionally includes support for the following ACE5-Lite+DVM properties

| Property                     | Signal           | Description                                                        |
|------------------------------|------------------|--------------------------------------------------------------------|
| DVM_v8.4                     | <b>ACVMIDEXT</b> | Extension for 16-bit VMID on snoop address channel                 |
| DVM_v9.2                     | (No new signals) | Support for DVM invalidation of Realm translations and GPT entries |
| Coherency_Connection_Signals | <b>SYSOREQ</b>   | Request to enter (HIGH) or exit (LOW) coherency domain             |
|                              | <b>SYSOACK</b>   | Coherency connection acknowledge                                   |
| Wakeup_Signals               | <b>ACWAKEUP</b>  | Activity present on the AC channel – drive QACTIVE                 |

17 xxxx rev 00000

## LPI\_PD interface considerations

### TBU power down

- Each TBU must disconnect from the TCU before the TBU can be powered down

### TCU power down

- All TBUs and PCIe Root Ports must be disconnected before the TCU can be powered down

### Power up

- The TCU must be powered up before any TBU or PCIe Root Port can be powered up and can connect

### Q-Channel requirements

- The interface must be in the Q\_STOPPED state before the clock can be gated or power removed

### Power control requires the clock to be active

- `qactive_cg` signal is asserted and LPI\_CG clock gating requests are denied while power control is in progress

### TBU and TCU state not retained during power down

- SMMU registers need to be reprogrammed after every power up

18 xxxx rev 00000

18

## TBU interrupts

| Edge-triggered signal   | Level-sensitive signal     | Description                                                                            |
|-------------------------|----------------------------|----------------------------------------------------------------------------------------|
| <code>ras_et_fhi</code> | <code>ras_lt_fhi</code>    | Fault Handling Interrupt (corrected error)                                             |
| <code>ras_et_eri</code> | <code>ras_lt_eri</code>    | Error Recovery Interrupt (recoverable uncorrected error)                               |
| <code>ras_et_cri</code> | <code>ras_lt_cri</code>    | Critical Error Interrupt (uncontainable uncorrected error)                             |
| -                       | <code>ras_lt_irpt_v</code> | Validity of at least one level-sensitive RAS interrupt (FHI, ERI or CRI)               |
| <code>pmu_irpt</code>   | -                          | Performance monitoring unit counter overflow                                           |
| -                       | <code>crit_err</code>      | Critical error – cannot connect to TCU due to connection denial or insufficient tokens |

The TBU cannot generate MSIs, but the ACE-Lite TBU can propagate ACE-Lite MSIs from TBS to TBM interfaces

19 xxxx rev 00000

19

## TCU interrupts (1/2)

| Edge-triggered signal | MSI | Description                                                                                |
|-----------------------|-----|--------------------------------------------------------------------------------------------|
| event_q_irpt_ns       | ✓   | Non-secure Event queue transitions from empty to non-empty                                 |
| event_q_irpt_s        | ✓   | Secure Event queue transitions from empty to non-empty                                     |
| event_q_irpt_r        | ✓   | Realm Event queue transitions from empty to non-empty                                      |
| cmd_sync_irpt_ns      | ✓   | Completion of <b>CMD_SYNC</b> consumed by Non-secure Command queue                         |
| cmd_sync_irpt_s       | ✓   | Completion of <b>CMD_SYNC</b> consumed by Secure Command queue                             |
| cmd_sync_irpt_r       | ✓   | Completion of <b>CMD_SYNC</b> consumed by Realm Command queue                              |
| global_irpt_ns        | ✓   | Non-secure Global Error – behavior determined by <b>SMMU_GERROR</b> register               |
| global_irpt_s         | ✓   | Secure Global Error – behavior determined by <b>SMMU_S_GERROR</b> register                 |
| global_irpt_r         | ✓   | Realm Global Error – behavior determined by <b>SMMU_R_GERROR</b> register                  |
| pri_q_irpt_ns         | ✓   | Non-secure PRI queue interrupt – behavior determined by <b>SMMU_PRIQ_IRQ_CFG2</b> register |
| pri_q_irpt_r          | ✓   | Realm PRI queue interrupt – behavior determined by <b>SMMU_R_PRIQ_IRQ_CFG2</b> register    |
| pmu_irpt              | ✗   | Performance monitoring unit counter overflow                                               |

20 xxxx rev 00000

20

## TCU interrupts (2/2)

| Edge-triggered signal | Level-sensitive signal | MSI | Description                                                              |
|-----------------------|------------------------|-----|--------------------------------------------------------------------------|
| gpf_far               | -                      | ✗   | A granule protection fault (GPF) has occurred (PAS mismatch)             |
| gpt_cfg_far           | -                      | ✗   | A GPT lookup error has occurred                                          |
| ras_et_fhi            | ras_lt_fhi             | ✗   | Fault Handling Interrupt (corrected error)                               |
| ras_et_eri            | ras_lt_eri             | ✗   | Error Recovery Interrupt (recoverable uncorrected error)                 |
| ras_et_cri            | ras_lt_cri             | ✗   | Critical Error Interrupt (uncontainable uncorrected error)               |
| -                     | ras_lt_irpt_v          | ✗   | Validity of at least one level-sensitive RAS interrupt (FHI, ERI or CRI) |

21 xxxx rev 00000

21

## Interrupt signalling

### TBU

- TBU are **rising edge triggered** or **level-triggered** wired interrupts only
- MSIs from connected ACE-Lite devices can be passed through the ACE-Lite TBU from the TBS to the TBM interface
- LTI devices are expected to have their own MSI interface for direct connection to the GIC ITS

### TCU

- Interrupts are always available as rising edge triggered or level-triggered wired interrupts
  - MSIs are available for queue interrupts and Global error interrupts
  - Wired interrupts do not need to be connected if MSIs are used instead
- Interrupts can be sent as MSIs from the dedicated MSI delivery interface
  - **AXIS-Stream MSIs** supported by GIC-700
  - MSIs are sent from this interface when interrupt address matches **msi\_addr** tie-off value
- Also possible to send MSIs from the QTW/DVM interface
  - **ACE5-Lite MSIs** supported by GIC-600
  - Interconnect should route these transactions to a downstream ITS

22 xxxx rev 00000

## Event signalling

**Software may place a core in a wait for event (WFE) low-power state while waiting for SMMU TLB invalidation to complete**

- Saves power compared to polling on **SMMU\_\*\_CMDQ\_CONS** register value
- Requires SMMU to notify core upon **CMD\_SYNC** completion
- Completion signal determined by **CMD\_SYNC.CS** field

### Interrupt mechanism

- **cmd\_sync\_irqt\_{s/ns/r}** wired interrupt or MSI is triggered and sent to GIC
- Core wakes up and **takes an exception** to handle interrupt

### Event mechanism

- Event is sent to core via TCU **Event interface** and Event network
- Four-phase handshake using **eventoreq** and **eventoack** signals
  - **eventoreq** is held HIGH until **eventoack** acknowledgement is received
- Core wakes up and resumes execution **without switching context**

23 xxxx rev 00000

## Performance monitoring interfaces

### MMU S3 implements the SMMUv3 Performance Monitor Extension

- Can count architectural and microarchitectural events to help with testing, debugging and performance analysis
- Each TBU and TCU has a separate **Performance Monitor Counter Group (PMCG)**
- Events are counted by performance counters and can be filtered by StreamID
- Functionality controlled by **SMMU\_PMCG\_\*** registers

### PMU snapshot

- Each TBU and TCU has **pmusnapshot\_req** and **pmusnapshot\_ack** signals for capturing event counter values
- Four-phase asynchronous handshake, typically controlled by debug infrastructure (CoreSight Cross Trigger Interface (CTI))
- Counter values at time of snapshot are saved to **SMMU\_PMCG\_SVRn** shadow registers
- Snapshot can also be initiated with software by setting **SMMU\_PMCG\_CFGR.CAPTURE**
- Counter values are also available in real-time by reading from **SMMU\_PMCG\_EVCNTRn** registers

24 xxxx rev 00000

24

## AMBA interface summary (r0 only)

### ACE-Lite TBU

- AXI5/ACE5-Lite (Issue K)
- AXI5-Stream (Issue B)
- DTI-TBUv3 (Issue F)

### LTI TBU

- LTI (Issue B)
- AXI5-Stream (Issue B)
- DTI-TBUv3 (Issue F)

### TCU

- AXI5/ACE5-Lite (Issue K)
- AXI5-Stream (Issue B)
- DTI-TBUv3, DTI-ATS{v1-v3} (Issue F)
- APB5 (Issue E)

### BAS interconnect components

- AXI5-Stream (Issue B)
- DTI-TBUv3, DTI-ATS{v1-v3} (Issue F)

25 xxxx rev 00000

25

## Agenda

### Interfaces

- Global interfaces
- ACE-Lite TBU
  - ACE-Lite TBU interfaces
  - ACE-Lite TBU interface properties
  - ACE-Lite TBU AxiMMUVALID signal
  - ACE-Lite TBU AxiUSER signals
- LTI TBU
  - LTI TBU interfaces
  - LTI TBU interface properties
- TCU
  - TCU memory system interfaces
  - TCU other interfaces
  - TCU interface properties
- LPI\_PD interface considerations

- Interrupts
  - TCU interrupts
  - TCU interrupts
  - Interrupt signalling
- Event signalling
- Performance monitoring interfaces
- AMBA interface summary

### DTI protocol and BAS interconnect

- DTI protocol
- DTI flow control
- BAS interconnect
- BAS switch configuration

26 xxxx rev 00000

26

## DTI protocol

### Message-based protocol for internal communication and communication with external translation agents

- DTI messages are carried over the physical BAS protocol in Arm SMMUs
- Two sub-protocols
  - DTI-TBU – for communication between a TBU and TCU
  - DTI-ATS – for communication between a PCIe Root Port and TCU

### Version support

- MMU S3 r0 implements **DTI-TBUv3** internally between TBUs and TCUs
- DTI-ATSV1, DTI-ATSV2 and DTI-ATSV3 are supported externally by ATS managers such as PCIe Root Ports
- DTI-TBUv1 and DTI-TBUv2 TBUs are **not supported** by the MMU S3 TCU

### DTI messages are grouped according to function

- Connection and disconnection, translation request/response, invalidation and synchronization, page request, register access

27 xxxx rev 00000

27

## DTI flow control

### Tokens provide flow control

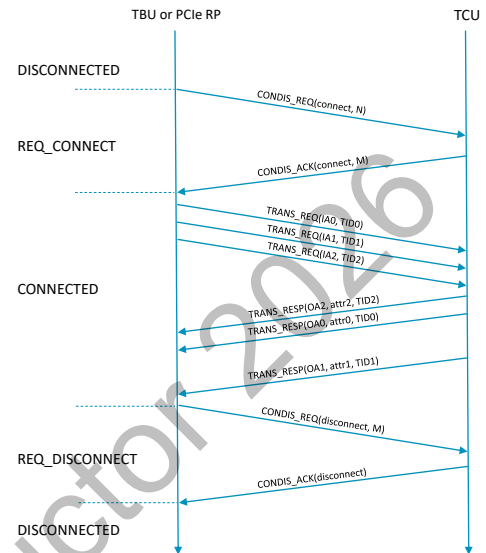
- Translation tokens – to limit outstanding translation requests
- Invalidation tokens – to limit outstanding invalidations

### Transmitters need tokens to send requests

- Request messages consume tokens
- Response/acknowledgement messages return tokens

### Token allocation

- Translation tokens are allocated by the TCU
- **max\_tok\_trans** tie-off controls number of translation tokens requested by TBU in connection request message
- TBUs must not request more translation tokens than TCU translation slots
- Invalidation tokens are granted by TBU or PCIe Root Port
- Tokens are returned when TBU or PCIe Root Port requests disconnection



28 xxxx rev 00000

28

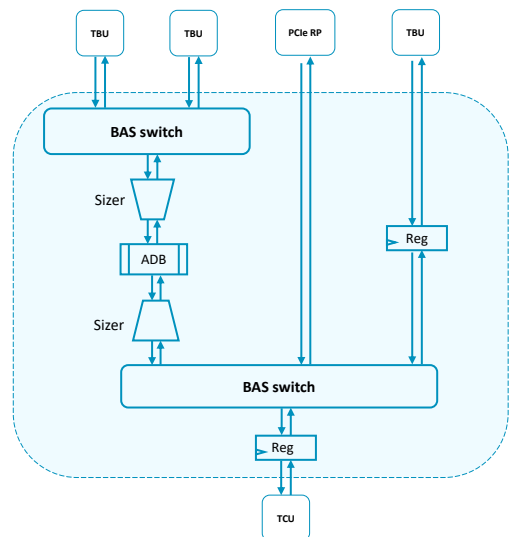
## BAS interconnect

### Set of composable elements

- Internal SMMU topology is **user-defined**
- Each component is instantiated and connected separately

### BAS interconnect components

- BAS sizer
  - Upsizes/downsizes data width to reduce number of wires
  - TBUs and TCU have **fixed data width** of 160 bits in the downstream direction
  - TBUs and TCU have fixed data width of **160 or 192 bits** in the upstream direction depending on the MECID width
- BAS switch
  - Connects multiple DTI subordinate interfaces (TBUs or PCIe Root Ports) to one DTI manager interface (TCU)
- BAS register slice
  - To improve timing by breaking long paths
- ADB-400
  - Asynchronous bridge between clock or power domains



29 xxxx rev 00000

29

# BAS switch configuration

Each subordinate interface has a unique namespace

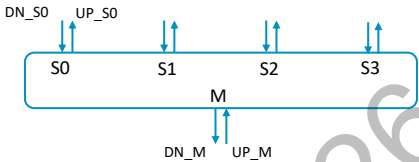
- Extends TIDs from DN\_Sn to DN\_M
- Reduces TDESTs from UP\_M to UP\_Sn

Configurable parameters

- ID\_WIDTH
- DECMIN\_SIn
- DECMAX\_SIn

Constraints

- Each {DECMIN\_SIn, DECMAX\_SIn} range must be unique
- ID\_WIDTH must be at least  $\log_2(\text{DECMAX\_SIn})$
- What ID\_WIDTH does this example require?



| Interface | TID/TDEST @ SIn | DECMIN_SIn | DECMAX_SIn |
|-----------|-----------------|------------|------------|
| S0        | 0-5             | 0          | 5          |
| S1        | 0-3             | 6          | 9          |
| S2        | 0-3             | 10         | 13         |
| S3        | 0-7             | 14         | 21         |

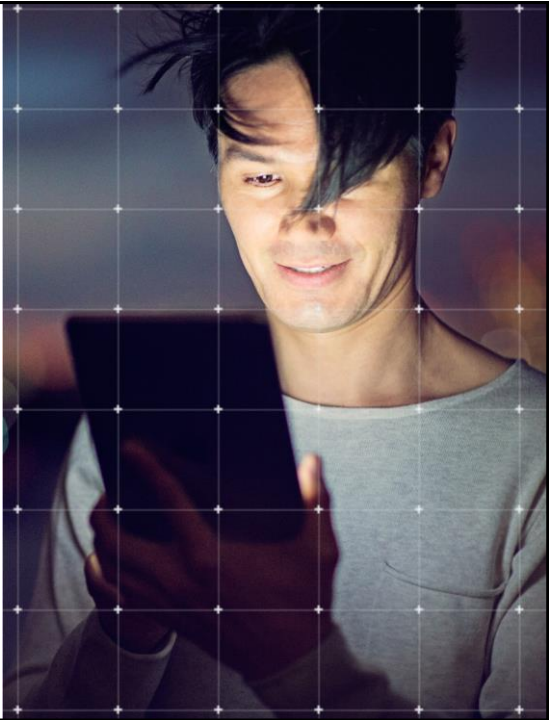
30 xxxx rev 00000

# Module summary

In this module we have covered

Interfaces

DTI protocol and BAS interconnect



31 xxxx rev 00000

arm

Thank You

Danke

Gracias

谢谢

ありがとう

Asante

Merci

감사합니다

धन्यवाद

Kiitos

شكراً

ধন্যবাদ

תודה

Confidential © 2024 Arm



# DTI Protocol Overview

Confidential © 2025 Arm

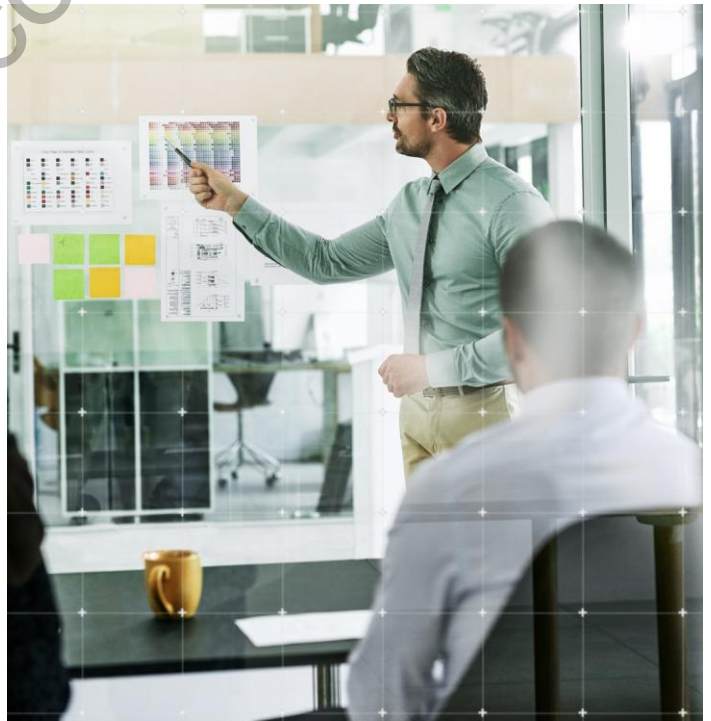
2

## Module agenda

### Introduction

DTI protocol messages

Documentation



3 2006 rev 51737

3

## DTI protocol

### Message-based point-to-point protocol for internal communication and with external translation agents

- DTI messages are carried over the physical BAS protocol in Arm SMMUs
- MMU-600 and MMU-600AE implement DTIv1 internally between TBUs and TCUs
- MMU-700 and MMU-700AE implement DTIv2
- MMU S3 R0 and MMU-720AE implement DTIv3
- MMU S3 R1 implements DTIv4

### Made up of two sub-protocols

- DTI-TBU – for communication between a TBU and TCU
- DTI-ATS – for communication between a PCIe controller and TCU
- DTI-ATSv1, DTI-ATSv2, DTI-ATSv3 and DTI-ATSv4 may be supported externally depending on SMMU implementation (with ATS managers such as PCIe controllers)
- For Arm SMMUs, only certain DTI-TBU versions are supported externally (for devices with built-in TBUs), check product TRM for details

### DTI messages are grouped according to function and identified by a MSG\_TYPE field

- Connection and disconnection, translation request and response, invalidation and synchronization, page request, register access

4 2006 rev 51737

4

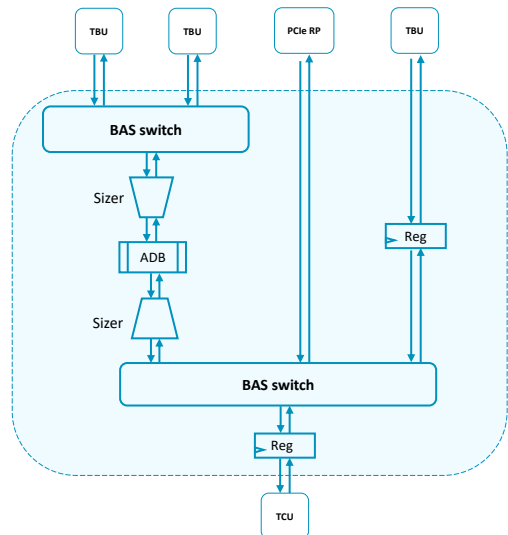
## Arm SMMUs – DTI over BAS interconnect

### DTI implemented over Bidirectional AXI4-Stream

- Alternative transport layers are allowed

### Transport layer needs to:

- Identify different DTI manager components
  - Done via TID / TDEST in AXI4-Stream
- Manage link-level flow control
  - E.g. TVALID/TREADY and TLAST in AXI4-Stream
- Not interleave transfers



5 2006 rev 51737

5

## DTI flow control

### Tokens provide flow control

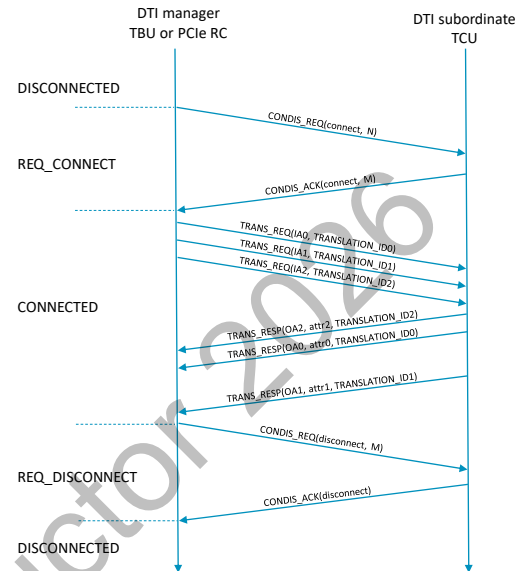
- Translation tokens – to limit outstanding translation requests
- Invalidation tokens – to limit outstanding invalidations

### Tokens are needed to send requests

- Request messages consume tokens
- Response/acknowledgement messages return tokens

### Translation token allocation

- Translation tokens are granted by the TCU
- Normally, TBU tie-off controls number of tokens requested by TBU in connection request message
- Tokens are returned when TBU requests disconnection



6 2006 rev 51737

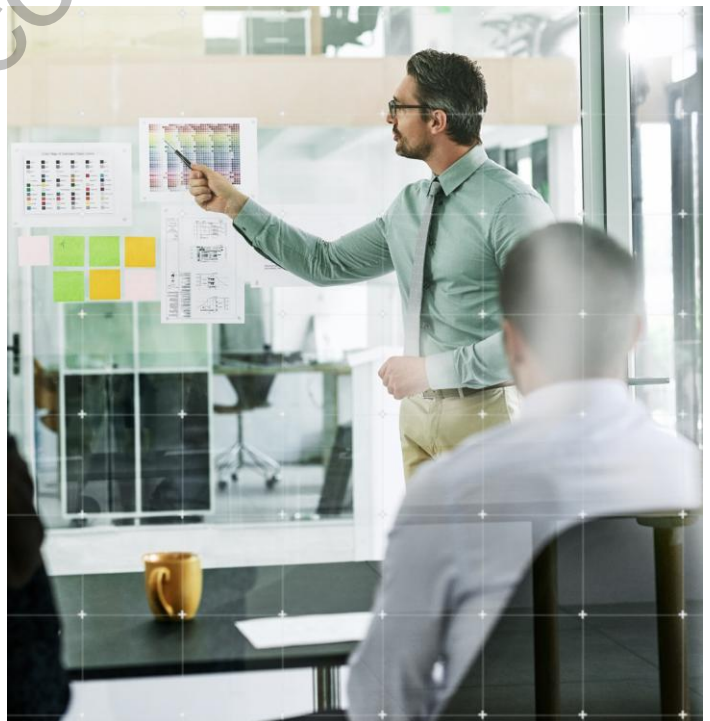
6

## Module agenda

Introduction

**DTI protocol messages**

Documentation



7 2006 rev 51737

7

# Connection and Disconnection

## What:

DTI\_<TBU/ATS>\_CONDIS\_REQ/ DTI\_<TBU/ATS>\_CONDIS\_ACK

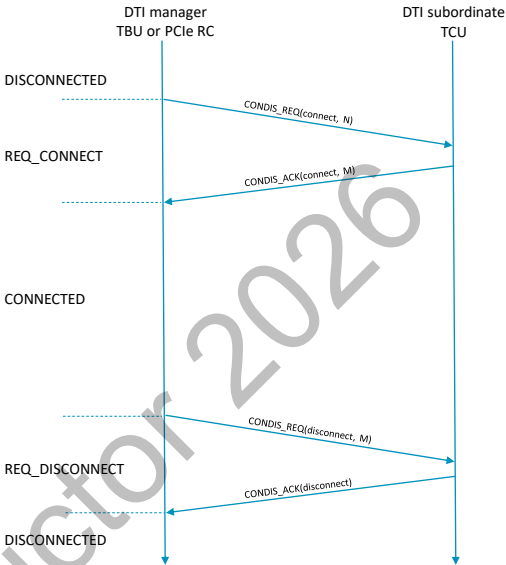
- Request and response to **connect** or **disconnect** to the rest of the SMMU
- At connection, requests translation tokens and grants invalidation tokens
- At disconnection, returns translation tokens to TCU

## When:

- Connect after reset
- Disconnect before power-down

## Notes:

- Initiated by DTI manager (TBU or PCIe RC)
- Responded (or denied) by DTI subordinate(TCU)



8 2006 rev 51737

# DTI-TBU Translation Request/Response/Fault

## What:

DTI\_TBU\_TRANS\_REQ

- Carries request to translate for {IA, SID, SSID, SEC\_SID} as well as relevant transaction attributes for permission checking

DTI\_TBU\_TRANS\_RESP

- Carries result of bypass/translation (OA, ASID/VMID, attributes)

DTI\_TBU\_TRANS\_FAULT

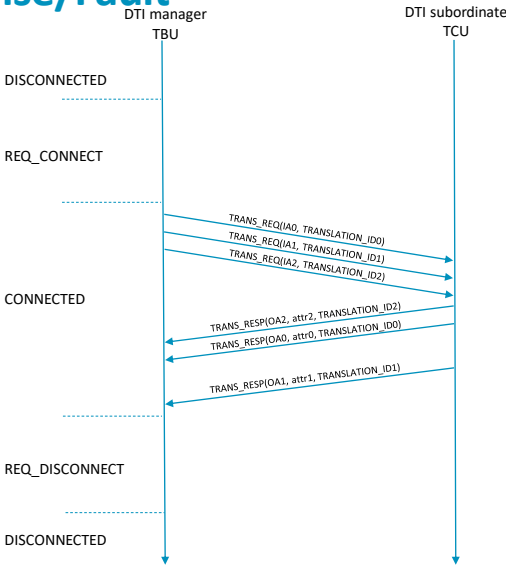
- Carries fault information (type, can/cannot be cached)

## When:

- TBU issues requests upon receiving transactions which miss in local TLBs (but can also be speculative)
- TCU issues responses after translation is complete

## Notes:

- Requests and responses have matching unique-in-flight TIDs/TDESTs



9 2006 rev 51737

## DTI-TBU Translation Response Extended

### What:

DTI\_TBU\_TRANS\_RESPEX (DTI-TBUv3 and DTI-TBUv4)

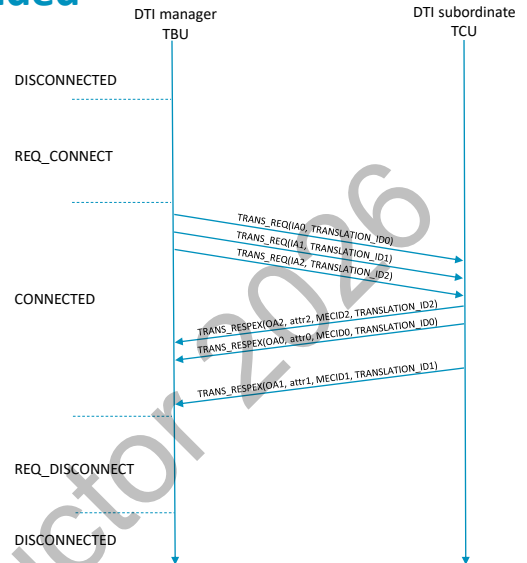
- Carries result of bypass/translation (OA, ASID/VMID, attributes)
- Encapsulates DTI\_TBU\_TRANS\_RESP message and adds new MECID field
- Extends the data width of the DTI\_TBU\_TRANS\_RESPEX response to 192 bits instead of 160, so interconnect must be sized to account for this

### When:

- TBU issues requests with DTI\_TBU\_TRANS\_REQ.REQEX=1
- TCU can issue RESPEX responses after translation is complete, but is not required to (can still be DTI\_TBU\_TRANS\_RESP/FAULT)

### Notes:

- All protocol rules for DTI\_TBU\_TRANS\_RESP apply to DTI\_TBU\_TRANS\_RESPEX unless stated otherwise
- If DTI\_TBU\_TRANS\_RESP is returned, equivalent to a RESPEX with all bits 0 in the MECID field



10 2006 rev 51737

## DTI-ATS Translation Request/Response/Fault

### What:

DTI\_ATS\_TRANS\_REQ

- Carries request to translate for {IA, SID, SSID, T} as well as relevant transaction attributes for permission checking

DTI\_ATS\_TRANS\_RESP

- Carries result of bypass/translation (OA, ASID/VMID, attributes)

DTI\_ATS\_TRANS\_FAULT

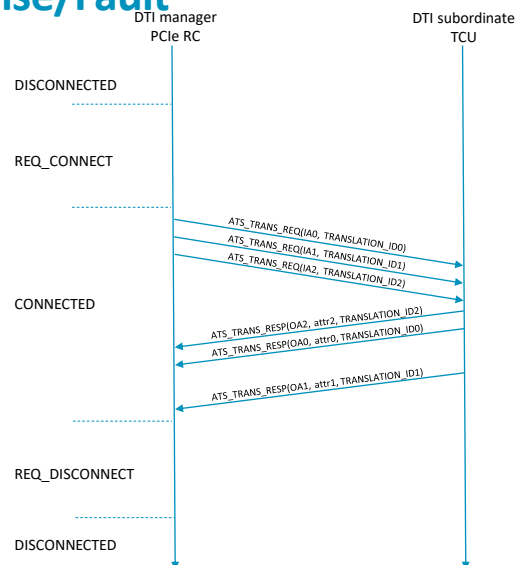
- Carries fault information (type, can/cannot be cached)

### When:

- PCIe RC issues requests upon receiving transactions which miss in local TLBs (but can also be speculative)
- TCU issues responses after translation is complete

### Notes:

- Requests and responses have matching unique-in-flight TIDs



11 2006 rev 51737

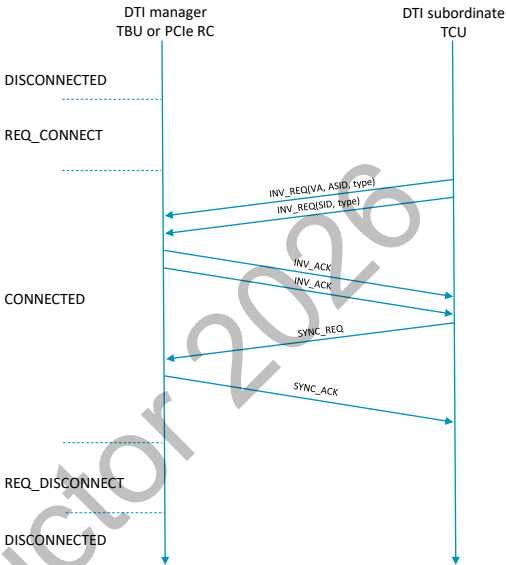
# Invalidation Request / Sync

**What:**

- DTI\_<TBU/ATS>\_INV\_REQ
- Requests invalidation of a TLB or configuration cache
- DTI\_<TBU/ATS>\_INV\_ACK
- Simply an ack for flow control purposes
- DTI\_<TBU/ATS>\_SYNC\_REQ
- Requests confirmation that previous invalidations have completed
- DTI\_<TBU/ATS>\_SYNC\_ACK
- Confirms completion of previous invalidations

**When:**

- TCU has received a DVM TLBI or Sync request (not for CFG cache)
- TCU has read an invalidation or sync command from command queue
- TCU specific register changes ("implicit invalidation")



12 2006 rev 51737

# DTI-TBU Register Access

**What:**

- DTI\_TBU\_REG\_<WRITE/READ>
- Signals a TBU register write or read
- DTI\_TBU\_REG\_<WACK/RDATA>
- Completes the TBU register access

**When:**

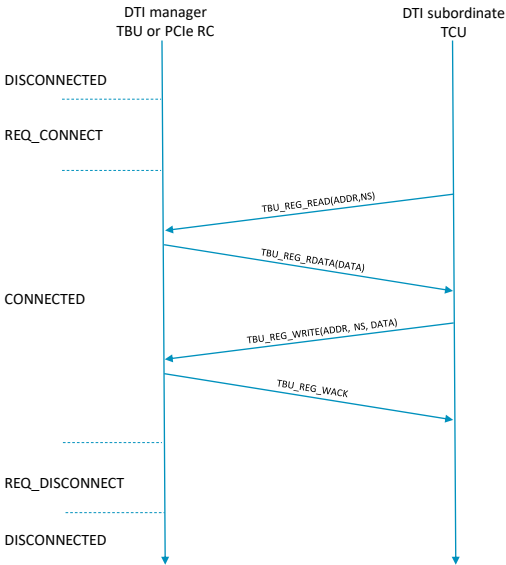
- TCU programming interface has received a register access for a TBU-specific register

**Notes:**

Applies to TBU only

**Register examples:**

- TBU u-Arch registers
- TBU RAS registers
- TBU MPAM registers



13 2006 rev 51737

## DTI-ATS Page Request / Response

### What:

#### DTI\_ATS\_PAGE\_REQ

- Used by the endpoint to issue a PRI page request

#### DTI\_ATS\_PAGE\_ACK

- Simply an ack

#### DTI\_ATS\_PAGE\_RESP

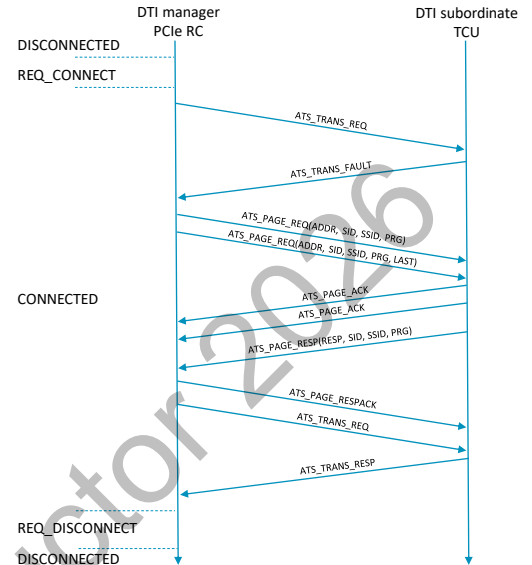
- Issued by TCU when SW has made page available

#### DTI\_ATS\_PAGE\_RESPACK

- Simply an ack

### When:

- Endpoint issues PRI page request when translation request fails
- In SMMUv3, TCU presents this to SW using PRI queue
- SW is responsible for fixing and issuing PRI resp command to the TCU



14 2006 rev 51737

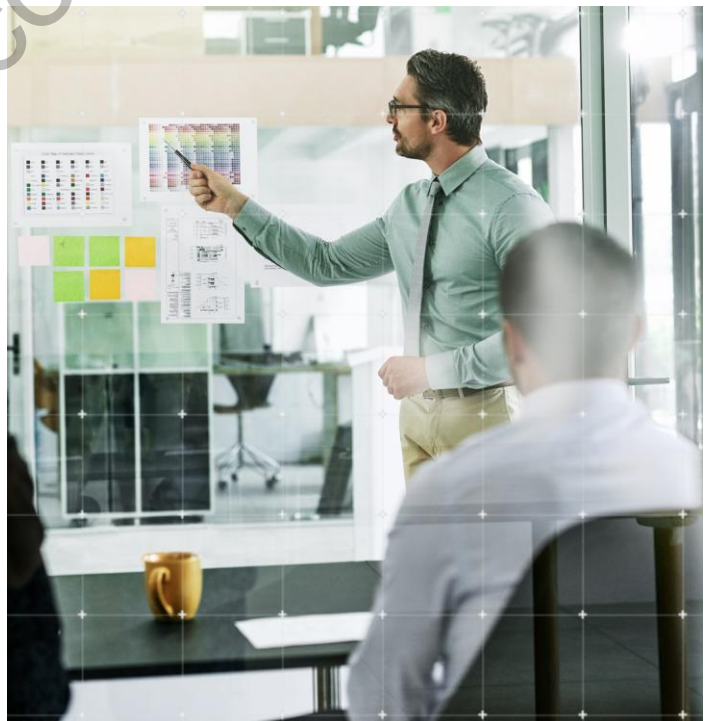
14

## Module agenda

Introduction

DTI protocol messages

Documentation



15 2006 rev 51737

15

## Documentation

DTI is formally part of the AMBA5 family of protocols

Arm® AMBA® Distributed Translation Interface (DTI) Protocol Specification

Consult the specific SMMU TRM to check DTI compliance and any implementation specific details

16 2006 rev 51737

16

## Quiz questions

1. Which physical protocol are the DTI messages carried over in Arm SMMUs?

[BAS \(Bidirectional AXI4-Stream\) protocol](#)

2. Which of these is *not* a DTI message type?

- a) Connection and disconnection
- b) Translation request
- c) TCU power down
- d) Invalidation and synchronization

3. Which SMMU component grants *translation* tokens?

- a) TBU
- b) TCU

4. What causes a DTI-ATS Page Request message?

[An ATS Translation Fault](#)

17 2006 rev 51737

17

## Module summary

In this module we have covered

Introduction

Topology

Interfaces

DTI protocol and BAS interconnect

Translation

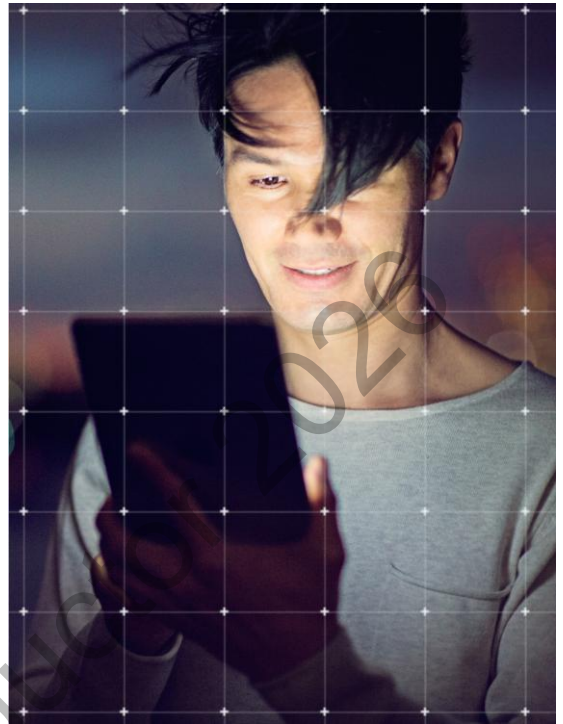
Caching

Additional features

Internal structures

Configuration and integration

18 2006 rev 51737



18

arm

Thank You  
Danke  
Gracias  
谢谢  
ありがとう  
Asante  
Merci  
감사합니다  
धन्यवाद  
Kiitos  
شكراً  
ধন্যবাদ  
תודה

Confidential © 2021 Arm

19

# arm

## MMU S3

Translation

Confidential © 2024 Arm

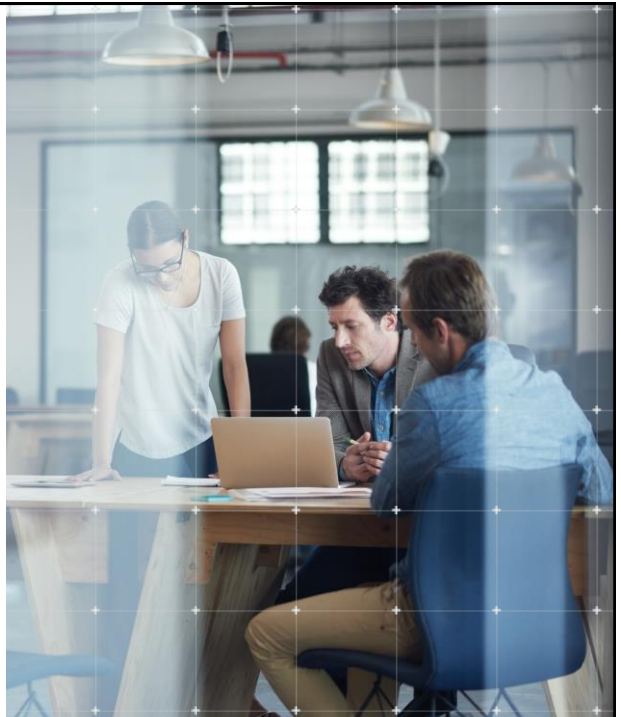
2

## Objectives

At the end of this module you will be able to:

- Recall the Security states supported by MMU S3 translation
- Recall the Security states supported by MMU S3 protection
- Differentiate between Non-secure, Secure and Realm translation controls and resources
- Describe the controls to override the Security state of register ownership
- Demonstrate the end-to-end MMU S3 translation flow
- Explain how many lookups are required for each step of the translation process
- Explain how granule protection checks (GPCs) protect memory accesses
- Describe how granule protection information (GPI) is stored in granule protection tables (GPTs)
- Recall the causes of granule protection faults (GPFs)

3 xxxx rev 00000



3

## Agenda

### Security

- Security states
- Resource banking
- Secure State Determination (SSD)
- Registers
  - Secure registers
  - Root registers

### Translation

- Translation flow
- Lookup process

### Protection

- Granule protection checks (GPCs)
- Granule protection information (GPI)
- Granule protection table (GPT) walks
- Granule protection check (GPC) faults

4 xxxx rev 00000

4

## Security states

### RME extends the TrustZone Security model and provides four Security states

- Non-secure – controlled by **Non-secure** software
- Secure – controlled by **Secure** software
- Realm – controlled by **Realm** software
- Root – controlled by **Root** software

### Translation and protection Security

- MMU S3 supports Secure, Non-secure and Realm **translation**
  - I/O devices are never owned by Root so Root translation is not supported
- MMU S3 supports Secure, Non-secure, Realm and Root **protection**
  - **Granule protection** is controlled by the Root state
- DPT checks are only supported for Realm translations

5 xxxx rev 00000

5

## Resource banking

### SMMU resources are banked by Security

- Separate translation controls and resources for **Non-secure**, **Secure** and **Realm** streams
  - Registers (**SMMU\_\*** / **SMMU\_S\_\*** / **SMMU\_R\_\***)
  - Configuration structures (Stream Table and Context Descriptor Table)
  - Command queue and Event queue
  - PRI queue (Non-secure and Realm only)
  - Interrupts
  - Performance counters
- **Root** registers exist for GPT control and invalidation (**SMMU\_ROOT\_\***)

### Translation security restricts permitted outgoing physical address space (PAS)

| Translation security | Non-secure PAS | Secure PAS | Realm PAS | Root PAS |
|----------------------|----------------|------------|-----------|----------|
| Non-secure           | ✓              | ✗          | ✗         | ✗        |
| Secure               | ✓              | ✓          | ✗         | ✗        |
| Realm                | ✓              | ✗          | ✓         | ✗        |

6 xxxxx rev 00000

6

## Secure State Determination (SSD)

### SEC\_SID signals determine the Security state of the translation

- **AxMMUSECSID** signal of TBS interface for ACE-Lite TBU
- **LASECSID** signal of LTI interface for LTI TBU

### SEC\_SID assignments

- When **SEC\_SID** == 0b00, use the **Non-secure** state
- When **SEC\_SID** == 0b01, use the **Secure** state
- When **SEC\_SID** == 0b10, use the **Realm** state

### Translation Security can be independent from device Security (incoming Security attribute)

- In most cases
  - **SEC\_SID[1]** = **AxNSE** & **AxPROT[1]** or **SEC\_SID[1]** = **LANSE** & **LAPROT[1]**
  - **SEC\_SID[0]** = **~AxNSE** & **~AxPROT[1]** or **SEC\_SID[0]** = **~LANSE** & **~LAPROT[1]**
- In some cases **SEC\_SID** can be controlled **independently** or tied to a **fixed value**

PCIe transactions should always use the Non-secure (**SEC\_SID** == 0b00) or Realm (**SEC\_SID** == 0b10) states

7 xxxxx rev 00000

7

## Secure registers

### Secure registers can usually only be accessed by Secure or Root software

- APB5 `pnse_prog` and `pprot_prog` signals of TCU determines the Security of programming transactions

### Secure Control register (TBU\_SCR / TCU\_SCR)

- Allows Non-secure software to access certain Secure registers
  - `SMMU_S_INIT` for invalidating all caches and TLBs
  - RAS registers for error control and reporting
  - TBU and TCU microarchitectural registers

### TBU\_SCR / TCU\_SCR can only be accessed by Secure or Root software

- Reset values for TBU\_SCR / TCU\_SCR determined by `sec_override` tie-off signal

8 xxxxx rev 00000

8

## Root registers

### Root registers can usually only be accessed by Root software

- APB5 `pnse_prog` and `pprot_prog` signals of TCU determines the Security of programming transactions

### Root Control register (TBU\_RCR / TCU\_RCR)

- Allows Secure or Non-secure software to access Root microarchitectural registers

### TBU\_RCR / TCU\_RCR can only be accessed by Root software

### Summary of controls to override Security state of register ownership

| TxU_SCR.feature | TxU_RCR.feature | Non-secure register | Secure register | Realm register | Root register |
|-----------------|-----------------|---------------------|-----------------|----------------|---------------|
| 0               | 0               | Non-secure          | Secure          | Realm          | Root          |
| 0               | 1               | Non-secure          | Secure          | Realm          | Secure        |
| 1               | 0               | Non-secure          | Non-secure      | Non-secure     | Root          |
| 1               | 1               | Non-secure          | Non-secure      | Non-secure     | Non-secure    |

9 xxxxx rev 00000

9

## Agenda

### Security

- Security states
- Resource banking
- Secure State Determination (SSD)
- Registers
  - Secure registers
  - Root registers

### Translation

- Translation flow
- Lookup process

### Protection

- Granule protection checks (GPCs)
- Granule protection information (GPI)
- Granule protection table (GPT) walks
- Granule protection check (GPC) faults

10 xxxx rev 00000

10

## Translation flow (1/2)

### Configuration lookup

- Use **SEC\_SID** to determine the Security state
- Obtain **Stage 2** translation configuration
  - **SMMU\_(\*\_)STRTAB\_BASE** provides Stream table base address
  - **SMMU\_(\*\_)STRTAB\_BASECFG** provides Stream table format and size
  - Use StreamID to index Stream table
  - Stream Table Entry (STE) contains Stage 2 translation information
- Obtain **Stage 1** translation configuration (if applicable)
  - **STE.S1ContextPtr** provides CD table base address
  - **STE.S1Fmt** and **STE.S1CDMax** provide CD table format and size
  - Use SubstreamID to index CD table
  - Context Descriptor (CD) contains Stage 1 translation information

11 xxxx rev 00000

11

## Translation flow (2/2)

### Translation lookup

- STE and CD provide translation table base address
- Perform translation table walks
  - Read translation table descriptors
  - Descriptor provides translation or base address of next-level table
- Use translation information to provide translation response

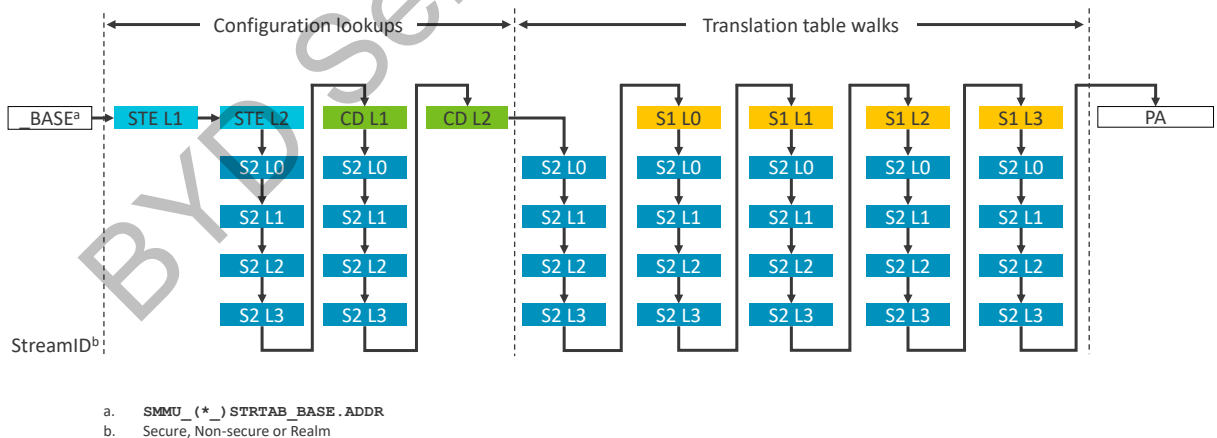
### Faults

- Faults can occur in both lookup processes
- Transactions that cause a translation-related fault can be **terminated** or **stalled**
- Stall fault model enables memory to be **unpinned** and **dynamically paged**
- Stalled transactions can be retried once a page has been made available

12 xxxx rev 00000

12

## Lookup process



13 xxxx rev 00000

13

## Agenda

### Security

- Security states
- Resource banking
- Secure State Determination (SSD)
- Registers
  - Secure registers
  - Root registers

### Translation

- Translation flow
- Lookup process

### Protection

- Granule protection checks (GPCs)
- Granule protection information (GPI)
- Granule protection table (GPT) walks
- Granule protection check (GPC) faults

14 xxxx rev 00000

14

## Granule protection checks (GPCs)

### Accesses to physical memory are subject to granule protection checks (GPCs)

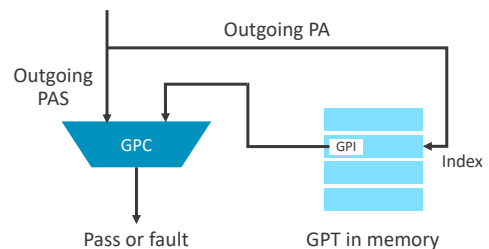
- Enables **dynamic assignment** of pages to **physical address spaces (PASs)**
- Avoids completer-side filtering based on fixed memory carveouts
- GPC ensures the Security attribute of the translated transaction is permitted to access the target address
- MMU S3 does not support elision

### SMMU-originated accesses

- Configuration and translation/page table walks (CTWs and PTWs)
- Hardware translation table updates (HTTU)
- Device permission table (DPT) walks (r1 only)
- Queue accesses (Event/Command/PRI)
- Message-signalled interrupts (MSIs)

### Client-originated accesses

- GPC on the final translated address



15 xxxx rev 00000

15

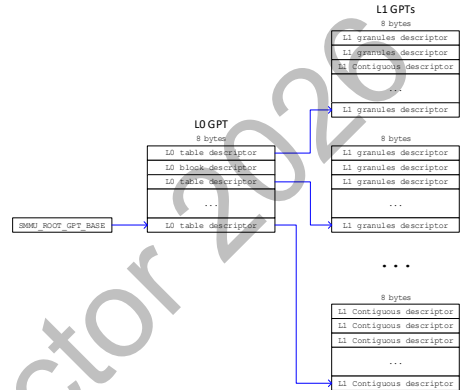
## Granule protection information (GPI)

### Granule protection table (GPT)

- A **granule protection table (GPT)** is a lookup table in memory that specifies the permitted PAS for each physical memory region (granule)
- Up to two levels of GPT walk may be required
- GPT entries can be **cached**
- GPT fetches are made to the **Root PAS** and are not subject to GPCs
- GPT fetches for translation table reads can be **speculated**

### Granule protection information (GPI)

- GPT Table (L0) descriptors point to the base address of a L1 table
- GPT Block (L0), Granules (L1) and Contiguous (L1) descriptors contain **GPI fields**
- Each GPI field encodes a **4-bit access value**
  - 0b0000: No access permitted
  - 0b1000: Access to **Secure PAS** only
  - 0b1001: Access to **Non-secure PAS** only
  - 0b1010: Access to **Root PAS** only (Only for NoStreamID transactions)
  - 0b1011: Access to **Realm PAS** only
  - 0b1111: Access to all PASs permitted (Use completer-side filtering)



16 xxxx rev 00000

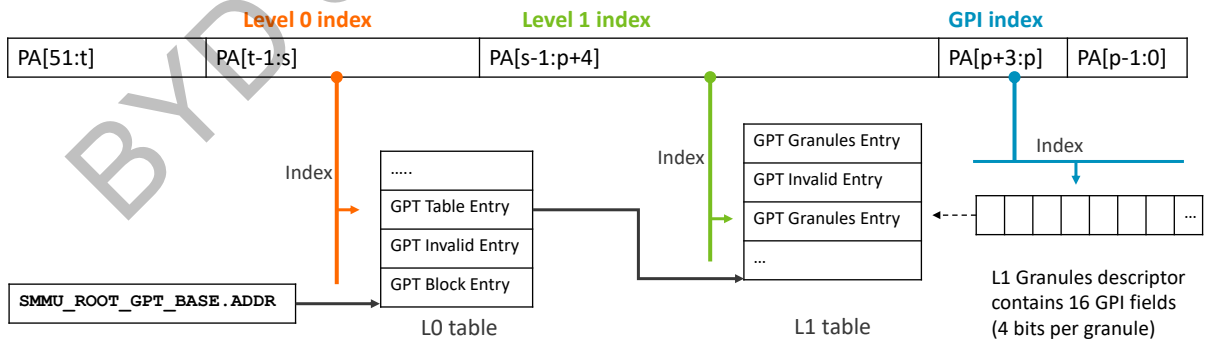
16

## Granule protection table (GPT) walks

$t$  is the protected physical address size (PPS)  
`SMMU_ROOT_GPT_BASE_CFG.PPS`

$s$  is the Level 0 GPT entry size  
`SMMU_ROOT_GPT_BASE_CFG.LOGPTSZ`

$p$  is the physical granule size (PGS)  
`SMMU_ROOT_GPT_BASE_CFG.PGS`



17 xxxx rev 00000

17

## Granule protection check (GPC) faults

### Granule protection fault (GPF)

- Causes
  - Access to location is **not permitted by GPT configuration** (PAS mismatch)
  - Access to Secure, Realm or Root PAS **exceeds protected physical address size (PPS)**
- Action
  - Report to software by writing to `SMMU_ROOT_GPF_FAR` and triggering `gpf_far` interrupt

### GPT lookup error

- Causes
  - Illegal `SMMU_ROOT_GPT_BASE_CFG` configuration
  - GPT L0 or L1 base address exceeds PPS
  - GPT descriptor is invalid
  - External abort (SLVERR) occurred during GPT fetch
  - RAS error occurred during GPT fetch
- Action
  - Report to software by writing to `SMMU_ROOT_GPT_CFG_FAR` and triggering `gpt_cfg_far` interrupt

18 xxxx rev 00000

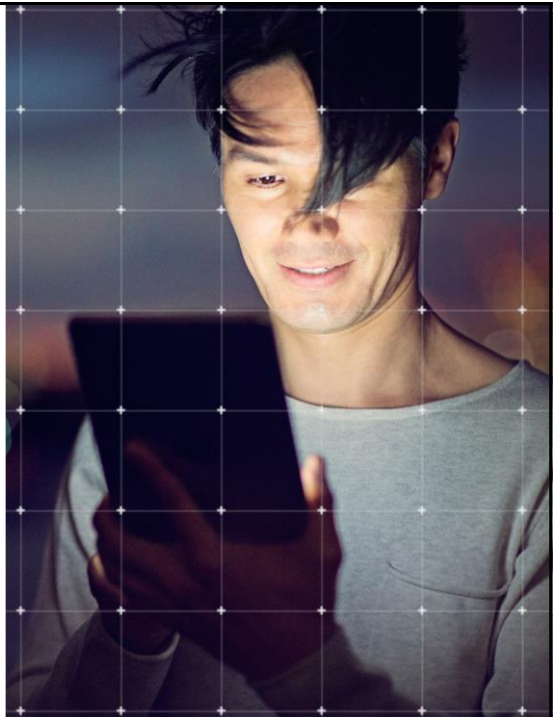
18

## Module summary

In this module we have covered

Security  
Translation  
Protection

19 xxxx rev 00000



19

arm

Thank You

Danke

Gracias

谢谢

ありがとう

Asante

Merci

감사합니다

धन्यवाद

Kiitos

شكراً

ধন্যবাদ

תודה

Confidential © 2024 Arm

# arm

## MMU S3

### Caching

Confidential © 2026 Arm

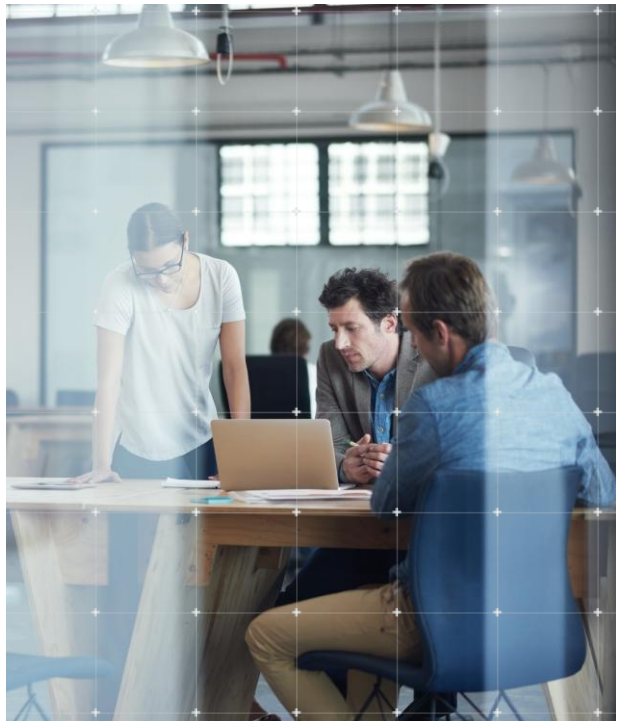
2

## Objectives

At the end of this module you will be able to:

- Explain the motivation for caching in MMU S3
- Recall the TBU and TCU caches
- Demonstrate which stages of lookup are covered by each cache
- Describe the differences between the TBU  $\mu$ TLB and MTLB
- Explain how the TBU hazards transactions that are expected to target the same page
- Understand how MTLB location management can aid predictability for real-time devices
- Describe the TCU translation caches
- Describe the TCU protection caches
- Explain how the TCU hazards transactions that target the same read address
- Explain how TCU prefetch can be used for linear access patterns
- Explain how StashTranslation transactions can be used by an intelligent devices that is aware of expected access patterns

3 xxxx rev 00000



3

# Agenda

## Caching overview

### TBU caches

- $\mu$ TLB and MTLB
- $\mu$ TLB way allocation
- TBU hazarding

### TCU caches

- TCU translation caches
- TCU protection caches
- TCU hazarding

### Cache prefetching

### Cache summary

4 xxxxx rev 00000

4

## Caching overview (1/2)

Caches can help improve translation and protection performance

### TBU caches

- Micro TLB ( $\mu$ TLB)
- Main TLB (MTLB)

### TCU translation caches

- Configuration cache
- Walk cache

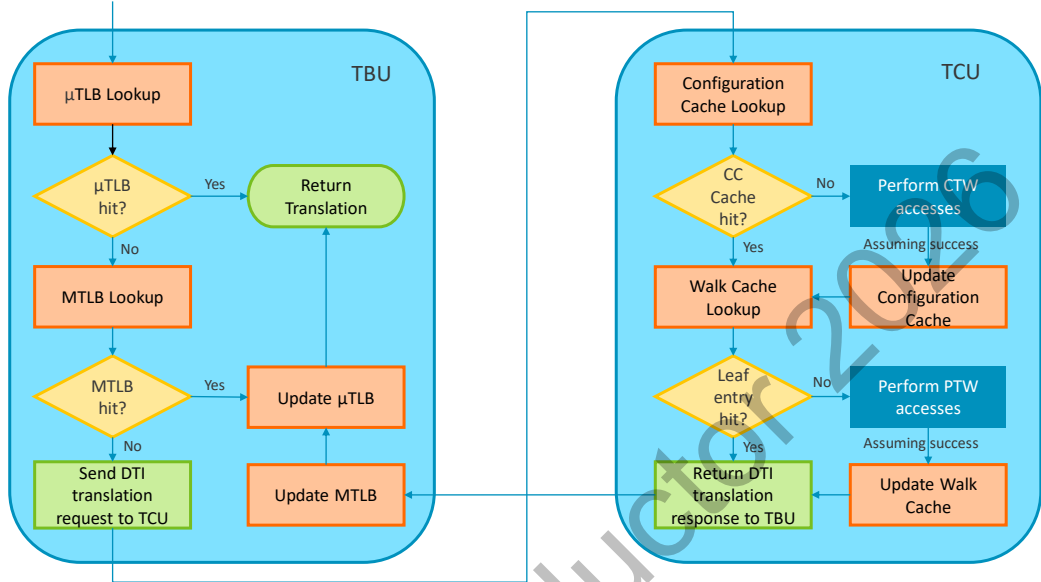
### TCU protection caches

- TMU DTI GPC wrapper GPT cache block (TMU DGW GCB)
- TMU AXI GPC wrapper GPT cache block (TMU AGW GCB)
- PIU AXI GPC wrapper GPT cache block (PIU AGW GCB)
- DPT cache (R1 only)

5 xxxxx rev 00000

5

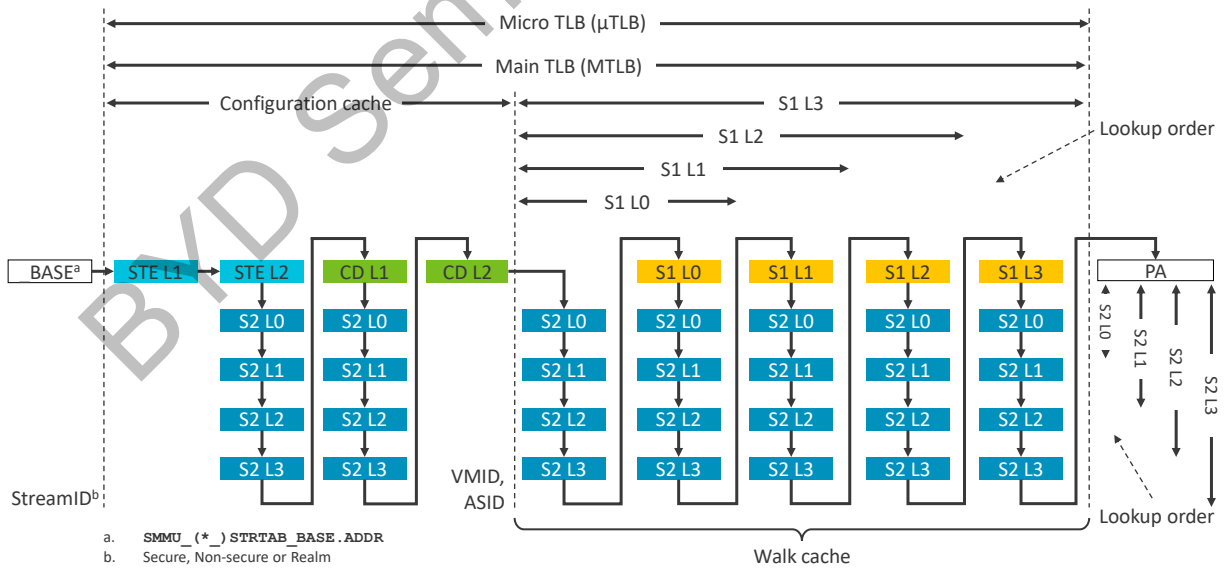
## Cache Lookups (minus GPT/DPT caches)



6 xxxx rev 00000

6

## Caching overview (2/2)



7 xxxx rev 00000

7

# Agenda

## Caching overview

### TBU caches

- $\mu$ TLB and MTLB
- $\mu$ TLB way allocation
- TBU hazarding

### TCU caches

- TCU translation caches
- TCU protection caches
- TCU hazarding

## Cache prefetching

## Cache summary

8 xxxx rev 00000

8

## TBU caches

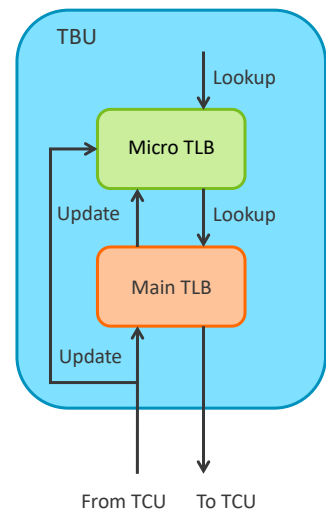
End-to-end translation and protection can be cached in the  $\mu$ TLB and MTLB

### $\mu$ TLB

- Implemented in flops
- Fully associative
- All entries can be looked up **simultaneously**
- Configurable Pseudo Least Recently Used (PLRU) or round-robin replacement

### MTLB

- Optional
- Implemented in SRAM
- Multi-bank structure with configurable associativity (4, 8 or 16 ways)
- A **separate lookup** is required for each page size
- **Hitmap** and **last-hit pointer** for different page sizes reduces lookups required
- DTI translation request to TCU is issued if there is an MTLB miss
- Support for MPAM cache maximum-capacity partitioning
- Static Re-Reference Interval Prediction (SRRIP) replacement



9 xxxx rev 00000

9

## μTLB way allocation

### Four special ways to avoid evicting special translations

- Non-secure GlobalDisabled/GlobalBypass/GlobalBypass ATS
- Secure GlobalDisabled/GlobalBypass
- Realm GlobalDisabled
- GlobalBypass for protection only (NoStreamID transactions)

### Minimum μTLB depth of 4 (4 special ways and 0 regular ways)

- Increasing μTLB depth adds **regular ways** for caching normal translation results
- Special ways can cache regular entries when special entries are not present
- SMMUV3 does not support Realm GlobalBypass

10 xxxx rev 00000

10

## TBU hazarding

### Hazarding combines multiple lookups/requests into a single operation

- Possible when multiple transactions are expected to require the **same page** and belong to the **same ordering group**
- Reduces MTLB, TCU and page table walk bandwidth requirements

### Hazards are formed after a μTLB miss

- Check incoming transaction against all active **hazard slots**
- If hazard slot matches
  - Append transaction to existing **hazard list**
  - Wait for existing translation request to complete
  - Existing transaction will unroll hazard list and provide a translation response
  - Use translation response to translate transaction
- If all hazard slots do not match
  - Allocate a new hazard slot and append transaction to a new hazard list
    - Slot replacement is Pseudo Least Recently Used (PLRU)
  - Proceed with translation request by looking up MTLB
  - If MTLB misses, send translation request to TCU
  - Wait for translation response
  - Unroll hazard list and use translation response for all transactions in list

11 xxxx rev 00000

11

## MTLB location management

### TBU can support external management of MTLB entries

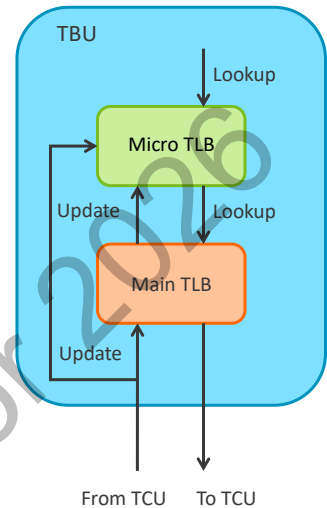
- More predictable performance for **real-time** devices
- Controlled by TLB Location (TLBLOC) attributes
  - AxUSER bits on TBS interface for ACE-Lite TBUs
  - LATLBLOC signal on LTI interface for LTI TBUs

### MTLB direct indexing

- Uses **mtlbidx** and **mtlbway** attributes from TLBLOC
- Update – specify a particular index and way
- Lookup – specify an index and all ways are looked up in parallel

### MTLB partitioning

- MTLB split into equal partitions
- **mtlbpart** attribute from TLBLOC specifies the partition to use for lookups/updates
- Generally superseded by MPAM which allows for easier control of partitions in SW



12 xxxx rev 00000

12

## Agenda

### Caching overview

#### TBU caches

- $\mu$ TLB and MTLB
- $\mu$ TLB way allocation
- TBU hazarding

#### TCU caches

- TCU translation caches
- TCU protection caches
- TCU hazarding

### Cache prefetching

### Cache summary

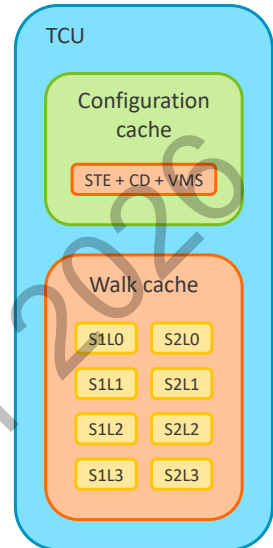
13 xxxx rev 00000

13

## TCU translation caches (1/2)

### Configuration cache

- Implemented in SRAM
- Single bank with configurable associativity (4, 8 or 16-ways)
- Caches **complete configuration information** (STE + CD + VMS)
- Each entry corresponds to a particular combination of SID, SSID and SEC\_SID
- Entries are grouped into three contiguous groups depending on the STE.CONT value
- Separate lookups and hitmap for each contiguous group
- Configuration table walk is performed on a configuration cache miss
- Support for MPAM cache maximum-capacity partitioning
- Static Re-Reference Interval Prediction (SRRIP) replacement



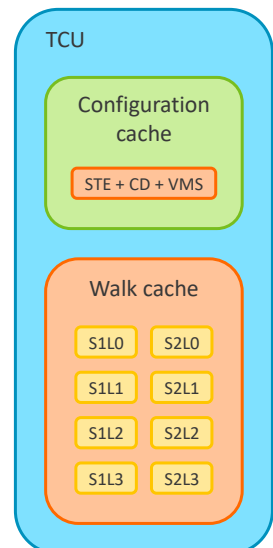
14 xxxx rev 00000

14

## TCU translation caches (2/2)

### Walk cache

- Implemented in SRAM
- Multi-bank structure with configurable associativity (4, 8 or 16 ways)
- Caches **translation table descriptors**
- Each entry provides the result of a single page table walk lookup
- Separate entries for each Page Table Walk (PTW) lookup level and stage of translation
- Hitmap for each combination of level, stage and Contiguous bit value reduces lookups required
- Maximum capacity per stage and level is run-time programmable with `TCU_WC_SxLy_CMAX` registers
- Support for MPAM cache maximum-capacity partitioning
- Lookups take place in **reverse order** from L3 to L0
- Stage 1 entries can cache IPA or PA
- Translation/page table walk is performed for every level where there is a walk cache miss
- Static Re-Reference Interval Prediction (SRRIP) replacement



15 xxxx rev 00000

15

## TCU protection caches (1/2)

### GPT Cache Blocks (GCBs) reduce latency of GPCs and minimize GPT walks

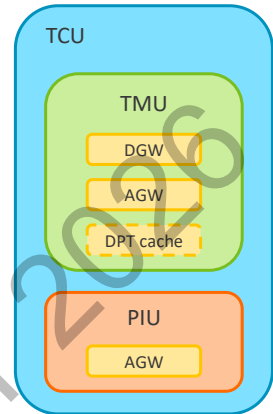
- Each GCB is tightly paired with a **GPT Management Block (GMB)**
- GMB performs GPCs and initiates GPT walks upon a GCB miss
- The GMB and GCB are instantiated three times within the MMU S3 TCU
  - TMU DGW for GPC on the final translated address
  - TMU AGW for GPC on table walks
  - PIU AGW for GPC on queue accesses and MSIs
- TMU DGW and AGW are configurable, PIU AGW has a fixed configuration

### GCB structure

- Multi-bank structure with configurable associativity (4, 8 or 16 ways) implemented in SRAM
- Caches GPT descriptors to provide GPI values
- Indexed by hashed Physical Address (PA) and Physical Address Space (PAS)
- Separate entries for each descriptor type
- Hitmap** and **last-hit pointer** for different descriptor types reduces lookups required
- Maximum capacity for each level is run-time programmable with **TCU\_\*GW\_GC\_Lx\_CMAX** registers
- Support for MPAM cache maximum-capacity partitioning (TMU AGW and DGW only)
- GPT walk is performed on a GCB miss
- Static Re-Reference Interval Prediction (SRRIP) replacement

17 xxxx rev 00000

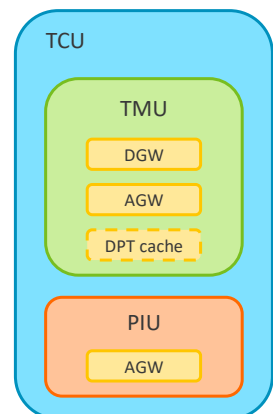
17



## TCU protection caches (2/2)

### Device Permission Table (DPT) cache (R1 only)

- Implemented in SRAM
- Multi-bank structure with configurable associativity (4, 8 or 16 ways)
- Caches DPT entries to provide device access permissions for ATS-translated transactions
- Separate entries for each descriptor type
- Hitmap for different DPT sizes
- Support for MPAM cache maximum-capacity partitioning
- DPT walk is performed on a DPT cache miss
- Static Re-Reference Interval Prediction (SRRIP) replacement



18 xxxx rev 00000

18

## TCU hazarding (1/2)

### Address-based hazarding on outstanding memory reads

- Hazarding possible when multiple translations require the same configuration structure, translation table descriptor or GPT descriptor
- Separate hazard units for QTW/DVM and PTW interfaces
- Avoids duplication of accesses and reduces memory bandwidth

### Hazard flow

- Check physical address against all active **hazard slots**
- If a hazard slot matches
  - Append transaction to existing **hazard list**
  - Wait for read response for existing transaction
- If all hazard slots do not match
  - Allocate a new hazard slot and append transaction to a new hazard list
  - Wait for read response
- Unroll hazard list and use translation response for all transactions in list
- Deallocate hazard list

19 xxxx rev 00000

19

## TCU hazarding (2/2)

### Transactions may not use the full data bus width

- L1 STE/CD fetch: 64 bits
- L2 STE/CD fetch: 512 bits
- Translation table descriptor fetch: 64 bits
- GPT descriptor fetch: 64 bits

### Hazard expansion

- TCU can be configured to perform reads at a wider data width, up to the **full data bus width**
- Wider reads can often be made without penalty
- Read address will be aligned to the expanded data width
- Hazard scope will be expanded to cover this wider data width
- Expanded hazard scope should increase the **hazard hit rate**
- Configured with hazard\_width tie-off signal
- Configuration should consider the **natural data width** of downstream interconnect and memory system

20 xxxx rev 00000

20

# Agenda

## Caching overview

### TBU caches

- $\mu$ TLB and MTLB
- $\mu$ TLB way allocation
- TBU hazarding

### TCU caches

- TCU translation caches
- TCU protection caches
- TCU hazarding

## Cache prefetching

### Cache summary

21 xxxx rev 00000

21

## Cache prefetching (1/2)

### TCU prefetch

- Can improve translation performance for real-time devices that perform **linear accesses**
- TCU proactively issues a prefetch request for the next page once the original translation request is complete
- Prefetch requests are **speculative** and allocate into the **TCU walk cache** only
- Improves the walk cache hit rate for linear accesses
- Can be enabled by software on a **per-stream** basis
- Controlled by **STE[121:120]**
  - **0b00** Prefetch disabled
  - **0b10** Prefetch forwards (incrementing address)
  - **0b11** Prefetch backwards (decrementing address)

22 xxxx rev 00000

22

## Cache prefetching (2/2)

### StashTranslation transactions

- Transaction type defined by ACE5-Lite protocol
- **Dataless** transaction involving AW and B channels only
  - SMMU performs speculative translation for incoming address
  - Translation process allocates into TCU and TBU caches
  - Incoming transaction terminated within BIU
- StashTranslation transaction must be **explicitly initiated** by upstream device
- Device must be aware of **expected access pattern** to send StashTranslation transactions before real transactions
- Also supported with LTI devices by using LATRANS signal

23 xxxx rev 00000

23

## Agenda

### Caching overview

#### TBU caches

- $\mu$ TLB and MTLB
- $\mu$ TLB way allocation
- TBU hazarding

#### TCU caches

- TCU translation caches
- TCU protection caches
- TCU hazarding

### Cache prefetching

#### Cache summary

24 xxxx rev 00000

24

# Cache summary

| Cache name          | Location | Implementation | Ways     | Banks   | Depth                | Lookup slots               | Hitmap | MPAM | Replacement         |
|---------------------|----------|----------------|----------|---------|----------------------|----------------------------|--------|------|---------------------|
| μTLB                | TBU      | Flops          | =Depth   | 1       | 4, 8, 12, 16, 32, 64 | 2-28 (R0)<br>2-32 (R1)     | ✗      | ✗    | PLRU or Round-robin |
| MTLB                | TBU      | SRAM           | 4, 8, 16 | 1, 2, 4 | 0, $2^5$ - $2^{16}$  | 2-28 (R0)<br>2-32 (R1)     | ✓      | ✓    | SRRIP               |
| Config cache        | TCU      | SRAM           | 4, 8, 16 | 1       | $2^2$ - $2^{12}$     | =Banks+5 (R0)<br>2-16 (R1) | ✓      | ✓    | SRRIP               |
| Walk cache          | TCU      | SRAM           | 4, 8, 16 | 1, 2, 4 | 8, $2^6$ - $2^{16}$  | 2-28 (R0)<br>2-32 (R1)     | ✓      | ✓    | SRRIP               |
| TMU DGW GCB         | TCU      | SRAM           | 4, 8, 16 | 1, 2, 4 | 8, $2^6$ - $2^{16}$  | 2-28 (R0)<br>2-32 (R1)     | ✓      | ✓    | SRRIP               |
| TMU AGW GCB         | TCU      | SRAM           | 4, 8, 16 | 1, 2, 4 | 8, $2^6$ - $2^{16}$  | 2-28 (R0)<br>2-32 (R1)     | ✓      | ✓    | SRRIP               |
| PIU AGW GCB         | TCU      | SRAM           | 4        | 1       | 8                    | 2                          | ✓      | ✗    | SRRIP               |
| DPT cache (R1 only) | TCU      | SRAM           | 4, 8, 16 | 1, 2, 4 | 8, $2^6$ - $2^{16}$  | 2-32                       | ✓      | ✓    | SRRIP               |

26 xxxx rev 00000

26

# Module summary

In this module we have covered

- Caching overview
- TBU caches
- TCU caches
- Cache prefetching
- Cache summary



27 xxxx rev 00000

27

arm

Thank You

Danke

Gracias

谢谢

ありがとう

Asante

Merci

감사합니다

धन्यवाद

Kiitos

شكراً

ধন্যবাদ

תודה

Confidential © 2026 Arm

# arm

## MMU S3

Additional features

Confidential © 2024 Arm

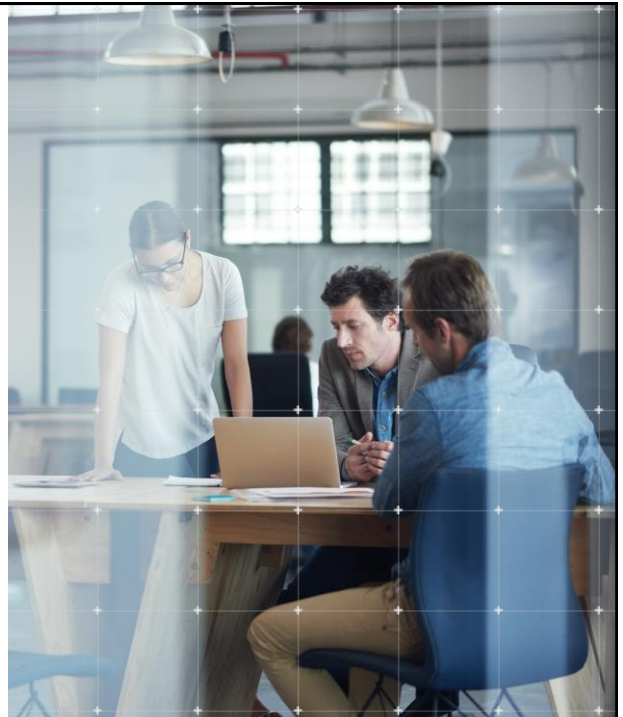
2

## Objectives

At the end of this module you will be able to:

- Describe the Memory System Resource Partitioning and Monitoring (MPAM) features MMU S3 implements
- Explain how cache maximum-capacity partitioning is controlled
- Explain how MPAM affects the cache replacement policy
- Describe the MMU S3 Quality of Service (QoS) features
- Explain how nodes or individual LTI ports are assigned to priority groups
- Understand the error detection mechanisms MMU S3 implements
- Describe how Reliability Availability and Serviceability (RAS) errors are reported
- Explain the flop parity feature
- Describe how MMU S3 supports poison
- List example Performance Monitoring Unit (PMU) events that can be generated
- Describe how to setup PMU event counters
- Describe how to filter PMU events by Security state and StreamID
- Explain how MMU S3 and ELA-600 can be integrated to capture traffic on external SMMU interfaces

3 xxxxx rev 00000



3

## Agenda

### Memory System Resource Partitioning and Monitoring (MPAM)

- MPAM implementation
- MPAM replacement

### Quality of Service (QoS)

- QoS features
- Priority group assignment

### Reliability, Availability and Serviceability (RAS)

- RAS implementation
- Flop parity
- Poison

### Debug

- Performance Monitoring Unit (PMU)
- Event selection and filtering
- Embedded Logic Analyzer (ELA) integration

4 xxxxx rev 00000

4

## Memory System Resource Partitioning & Monitoring (MPAM)

### MMU S3 implements the MPAM Extension of the Armv9-A architecture

- Increasingly common for systems to run multiple applications or multiple virtual machines concurrently
- MPAM aims to share **memory system performance** fairly between different software entities running on the same system
- System-level feature requiring support by multiple components and control by compatible software

### Cache maximum-capacity partitioning

- Supported by MTLB, configuration cache, walk cache, TMU DGW GCB, TMU AGW GCB, DPT cache (R1 only)
- Each translation context has a **PARTID**
- **PARTID** determines which cache partition is used
- Occupancy limits set for each partition by programming **MPAMCFG\_CMAX** registers
- Partitions are permitted to overlap if sum of occupancy limits exceeds total capacity
- Other architectural MPAM registers provide further control, resource monitoring and status information

### MPAM information also propagated downstream for use by the rest of the system

- AxiMPAM signals on TBM interface of ACE-Lite TBU
- LRMPAM signal on LTI interfaces of LTI TBU
- AxiMPAM signals on QTW/DVM and PTW interfaces of TCU

5 xxxxx rev 00000

5

## MPAM replacement

### MMU S3 supports a **PARTID** width of 1, 6, 9 or 10 bits (R1 only)

- Each **PARTID** has an associated occupancy counter and **CMAx** capacity limit
- Out of reset the CMAx limit for all partitions is 100% of the total cache capacity
- An update is **dropped** if it would cause its **PARTID** to exceed the CMAx limit

### Replacement hierarchy

1. Ways that have the same tag as the new entry
2. Ways that are invalid
3. Ways that have the same **PARTID** as the new entry and the **PARTID** occupancy counter is at or above its **soft limit**
4. All remaining ways

### Replacement process

- The search for a replacement location follows the replacement hierarchy and stops when any group has one or more **eligible ways**
- A selection between eligible ways is made according to the Static Re-Reference Interval Prediction (SRRIP) replacement policy

### Considerations

- Soft limit is an automatically calculated fraction of the CMAx hard limit and aims to promote self-replacement instead of thrashing
- Greater associativity (more ways) reduces the risk of dropped updates but is more challenging for physical implementation (timing)

6 xxxxx rev 00000

6

## Agenda

### Memory System Resource Partitioning and Monitoring (MPAM)

- MPAM implementation
- MPAM replacement

### Quality of Service (QoS)

- QoS features
- Priority group assignment

### Reliability, Availability and Serviceability (RAS)

- RAS implementation
- Flop parity
- Poison

### Debug

- Performance monitoring unit (PMU)
- Event selection and filtering
- Embedded Logic Analyzer (ELA) integration

7 xxxxx rev 00000

7

## Quality of Service (QoS)

### TBU

- For ACE-Lite TBU, **AxQoS** value from upstream device is propagated from TBS interface to TBM interface without modification
- Short translation latency for  $\mu$ TLB or MTLB hit so no internal prioritization required
- TCU prioritizes between LTI ports when MTLB misses and DTI translation requests are issued

### TCU

- Separate **priority groups** exist for different transactions on the QTW/DVM and PTW interfaces
- DVM Complete messages, MSIs and queue accesses each have their own priority group
- Four assignable priority groups for configuration table walks, translation table walks, and GPT walks (**QoS\_PTW3 – QoS\_PTW0**)
- Each priority group assigned a 4-bit QoS value in **TCU\_QOS** register
- AxQoS signals on QTW/DVM and PTW interfaces use the QoS value corresponding to the priority group
- AxQoS value can be used by downstream interconnect and memory system to prioritize different accesses
- QoS\_PTW3 – QoS\_PTW0** values also determine prioritization within TCU during translation process

#### TCU\_QOS

|          |    |    |    |    |    |    |    |             |    |    |    |         |    |    |    |           |    |    |    |          |    |   |   |          |   |   |   |          |   |   |   |          |  |  |  |
|----------|----|----|----|----|----|----|----|-------------|----|----|----|---------|----|----|----|-----------|----|----|----|----------|----|---|---|----------|---|---|---|----------|---|---|---|----------|--|--|--|
| 31       | 30 | 29 | 28 | 27 | 26 | 25 | 24 | 23          | 22 | 21 | 20 | 19      | 18 | 17 | 16 | 15        | 14 | 13 | 12 | 11       | 10 | 9 | 8 | 7        | 6 | 5 | 4 | 3        | 2 | 1 | 0 |          |  |  |  |
| Reserved |    |    |    |    |    |    |    | QoS_DVMSYNC |    |    |    | QoS_MSI |    |    |    | QoS_QUEUE |    |    |    | QoS_PTW3 |    |   |   | QoS_PTW2 |   |   |   | QoS_PTW1 |   |   |   | QoS_PTW0 |  |  |  |

8 xxxx rev 00000

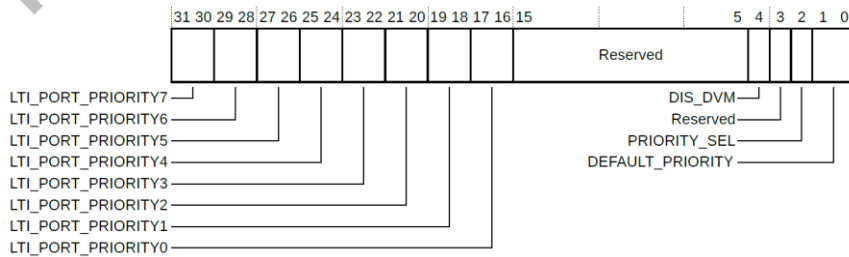
8

## Priority group assignment (1/2)

### Per node priority

- All translations requested by this node have the same priority
- Set **TCU\_NODE\_CTRLn.PRIORITY\_SEL = 1**
- TCU\_NODE\_CTRLn.DEFAULT\_PRIORITY** specifies the index into the **TCU\_QOS** register (**QoS\_PTW3 – QoS\_PTW0**)
- QoS value for the whole node is given by
  - $\text{TCU\_QOS}[(\text{TCU\_NODE\_CTRLn.DEFAULT\_PRIORITY}+1)*4-1 : (\text{TCU\_NODE\_CTRLn.DEFAULT\_PRIORITY}+1)*4-4]$

#### TCU\_NODE\_CTRLn



9 xxxx rev 00000

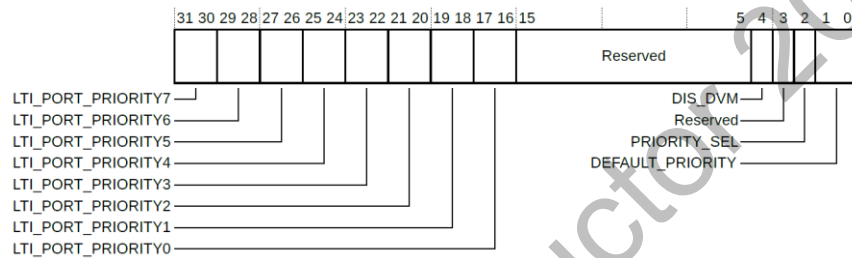
9

## Priority group assignment (2/2)

### Per LTI port priority

- Each LTI port of the LTI TBU can have its own priority
- Set `TCU_NODE_CTRLn.PRIORITY_SEL = 0`
- `TCU_NODE_CTRLn.LTI_PORT_PRIORITYx` specifies the index into the `TCU_QOS` register (`QoS_PTW3 – QoS_PTW0`) for LTI port `x`
- QoS value for LTI port `x` is given by
  - `TCU_QOS[(TCU_NODE_CTRLn.LTI_PORT_PRIORITYx+1)*4-1 : (TCU_NODE_CTRLn.LTI_PORT_PRIORITYx+1)*4-4]`

#### TCU\_NODE\_CTRLn



10 xxxx rev 00000

10

## Agenda

### Memory System Resource Partitioning and Monitoring (MPAM)

- MPAM implementation
- MPAM replacement

### Quality of Service (QoS)

- QoS features
- Priority group assignment

### Reliability, Availability and Serviceability (RAS)

- RAS implementation
- Flop parity
- Poison

### Debug

- Performance monitoring unit (PMU)
- Event selection and filtering
- Embedded Logic Analyzer (ELA) integration

11 xxxx rev 00000

11

## Reliability, Availability, and Serviceability (RAS)

### MMU S3 implements the RAS Extension of the Armv9-A architecture

- Supported RAS features given by **TBU\_ERRFR** and **TCU\_ERRFR** registers
- Other RAS registers implemented for error control, error status, and error generation (for testing purposes)

### Error detection mechanisms

- Single-Error-Correct-Double-Error-Detect (SECCDED)
  - Used for RAMs where the SMMU **cannot proceed** after a double error
  - Single error is correctable and triggers FHI
  - Double error in ACE-Lite TBU write data buffer is deferrable and triggers FHI if **poison** is supported
  - Otherwise double error is uncorrectable and triggers FHI, ERI and CRI
- Double-Error-Detect (DED)
  - Used for RAMs where data can be recovered by **invalidating** corrupted cache lines and **refetching** from DRAM
  - Single or double error detected during a cache lookup triggers FHI and is treated as a cache miss

### Error notification

- Separate interrupts depending on the severity of the error and the action required
- Type and source of error indicated by **TBU\_ERRSTATUS** and **TCU\_ERRSTATUS** syndrome registers

12 xxxx rev 00000

12

## Flop parity

### Flop parity can be implemented on large or critical groups of flops

- Key configuration registers are always parity protected
- Flop parity is configurable for other programmable registers and large flop structures
- Configured with **TBUCFG\_PARITY\_ON** and **TCUCFG\_PARTITY\_ON** configuration parameters
- Implemented with common **ff\_parity** block which is instantiated throughout the design

### Behavioral description

- On write:
  - Data input is stored in internal flops
  - Parity of data input is computed and stored in parity flops
- On read:
  - Parity of data output is computed and compared against parity stored in parity flops
  - Parity check failure is uncorrectable and triggers FHI, ERI and CRI

### Flop parity has a high area impact if enabled

13 xxxx rev 00000

13

## Poison

### ACE-Lite TBU

- Behavior controlled by **poison\_support** tie-off signal
- If **poison\_support == 0**
  - xPOISON signals are unmodified by TBU and propagate between TBS and TBM interfaces
  - Double error in write data buffer is **uncorrectable** and triggers Fault Handling Interrupt (FHI), Error Recovery Interrupt (ERI) and Critical Error Interrupt (CRI)
- If **poison\_support == 1**
  - Double error in write data buffer is **deferred** by issuing a poisoned write downstream
  - Outgoing WPOISON value is the OR of the incoming WPOISON value with the poison indication from the write data buffer
  - RPOISON signal is unmodified by TBU and propagates between TBM and TBS interfaces

### TCU

- Poisoned read data can be consumed by QTW/DVM and PTW interfaces
- Poison detected in TCU is uncorrectable and triggers FHI and ERI
- **TCU\_ERRSTATUS** register indicates the type of read transaction that returned poisoned data
- TCU does not generate poisoned writes

14 xxxx rev 00000

## Agenda

### Memory System Resource Partitioning and Monitoring (MPAM)

- MPAM implementation
- MPAM replacement

### Quality of Service (QoS)

- QoS features
- Priority group assignment

### Reliability, Availability and Serviceability (RAS)

- RAS implementation
- Flop parity
- Poison

### Debug

- Performance monitoring unit (PMU)
- Event selection and filtering
- Embedded Logic Analyzer (ELA) integration

15 xxxx rev 00000

## Performance monitoring unit (PMU)

### Separate Performance Monitor Counter Group (PMCG) for each TBU and TCU

- Controlled by `SMMU_PMC*_*` registers or PMU snapshot interface
- Counts **architectural** and **microarchitectural** performance events

### Example events

- Incoming transaction
- Cache lookup (separate event for each cache)
- Cache miss (separate event for each cache)
- Cache lookup/update slots full (separate event for each cache)
- Translation slots full
- Out of translation tokens
- Write data buffer full
- Hazard list hit/miss/full
- Translation request to TCU
- Speculative request
- PCIe Address Translation Services (ATS) request
- NoStreamID request
- Hardware Translation Table Update (HTTU)

16 xxxx rev 00000

16

## Event selection and filtering

### MMU S3 supports 4, 16 or 32 PMU counters per PMCG

- Configured with `TBUCFG_PMU_COUNTERS` and `TCUCFG_PMU_COUNTERS` configuration parameters
- `SMMU_PMC*_EVCNTR<n>` records **counter value**
- `SMMU_PMC*_EVTYPEPER<n>.EVENT` selects **which event** is counted
- `SMMU_PMC*_INTENSET0.INTEN[<n>]` enables `pmu_irtpt` interrupt upon **counter overflow**

### Some events can be filtered by Security state and StreamID

- Filter configuration applies to **all counters** in PMCG
- Filtering by MPAM PARTID and PMG is not supported
- Let `Re1 = SMMU_PMC*_EVTYPEPERn.FILTER_REALM_SID & SMMU_PMC*_ROOTCR.RLO`
- Let `Sec = SMMU_PMC*_EVTYPEPERn.FILTER_SEC_SID & SMMU_PMC*_SCR.SO`
  - If `Re1 == 0 && Sec == 0`, count only **Non-secure** events
  - If `Re1 == 0 && Sec == 1`, count only **Secure** events
  - If `Re1 == 1 && Sec == 0`, count only **Realm** events
- `SMMU_PMC*_SMR<n>` provides the StreamID to match when filtering
- `SMMU_PMC*_EVTYPEPER<n>.FILTER_SID_SPAN` controls whether the StreamID must match exactly or match a configurable number of MSBs

17 xxxx rev 00000

17

## Embedded Logic Analyzer (ELA) integration

### MMU S3 supports integration with an external ELA-600 Embedded Logic Analyzer

- Can be used to **capture and inspect** traffic on external TBU and TCU interfaces
- ELA-600 is **delivered separately** and not part of MMU S3
- TBU and TCU **observation interfaces** can be connected to ELA observation interface
  - TBU has one observation interface for all external AMBA signals (ACE-Lite or LTI)
  - TCU has one observation interface for DTI and QTW/DVM interface signals and one observation interface for PTW interface signals (if present)
- All **signal groups** are 128 bits wide
- Number of signal groups depends on **TCUCFG\_QTW\_DATA\_WIDTH** parameter for TCU and **TBUCFG\_LTI\_LOOP\_WIDTH** parameter for LTI TBU

### ELA usage

- **TBUCFG\_USE\_ELA\_DEBUG** and **TCUCFG\_USE\_ELA\_DEBUG** configuration parameters control presence of ELA output logic
- **ela\_enable** signal gates observation interfaces when ELA is inactive – should be connected to ELAQAACTIVE output of ELA-600
- ELA trace can be logged to embedded SRAM or exported over ATB interface
- MMU S3 Technical Reference Manual (TRM) provides mapping between traceable interface signals and observation interface signal groups

18 xxxx rev 00000

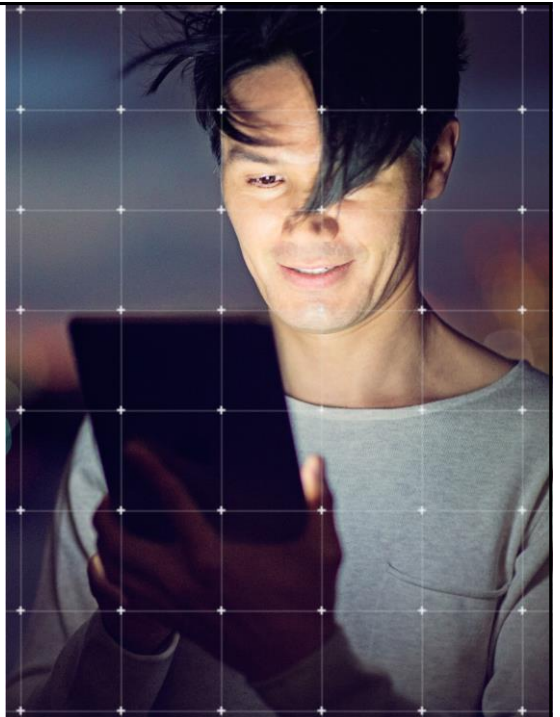
18

## Module summary

In this module we have covered

MPAM  
QoS  
RAS  
Debug

19 xxxx rev 00000



19

arm

Thank You

Danke

Gracias

谢谢

ありがとう

Asante

Merci

감사합니다

धन्यवाद

Kiitos

شكراً

ধন্যবাদ

תודה

Confidential © 2024 Arm

# arm

## MMU S3

Configuration and integration

Confidential © 2024 Arm

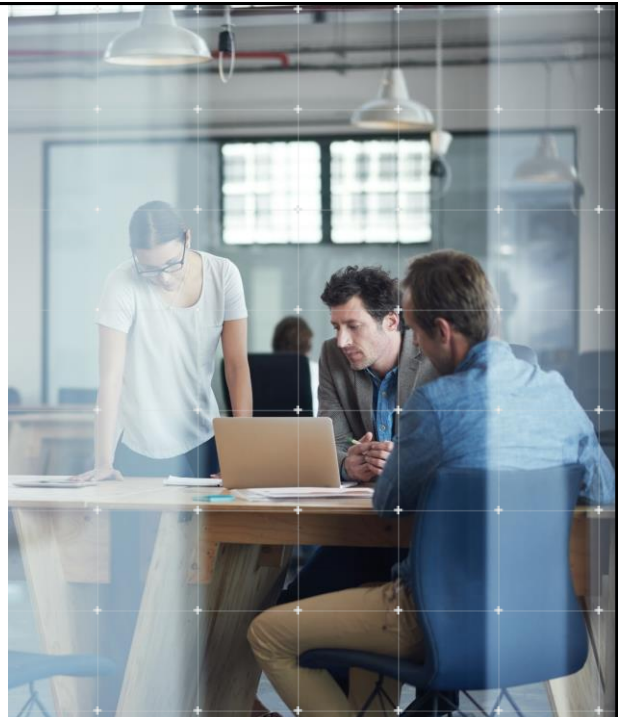
2

### Objectives

At the end of this module you will be able to:

- Recall the individual MMU S3 components
- Understand the MMU S3 internal structures
- Describe the parameters for configuring MMU S3
- Explain how tie-off signals affect the MMU S3 functionality
- Describe which generic cells must be replaced with library cells
- Explain the supported RAM configurations and implementations
- Recall how to determine the required RAM type and dimensions
- Explain how the execution testbench can be used for limited RTL validation
- Describe where to find further reading materials and additional MMU S3 resources

3 xxxxx rev 00000



3

## Agenda

### Internal structures

- MMU S3 components revisited
- TBU internal structure
  - BIU internal structure
  - TLBU internal structure
- TCU internal structure
  - TMU internal structure
  - PIU internal structure

### Configuration parameters

- Common TBU configuration parameters
- ACE-Lite TBU configuration parameters
- LTI TBU configuration parameters
- TCU configuration parameters
- BAS interconnect configuration parameters

### Tie-off signals

- TBU tie-off signals
- TCU tie-off signals

### Integration and testing

- Cell replacement
- RAM integration
- Execution testbench

### Further reading

- Architecture specifications
- MMU S3 manuals
- Related product information
- Protocol specifications

4 xxxxx rev 00000

4

## MMU S3 components revisited

### ACE-Lite TBU

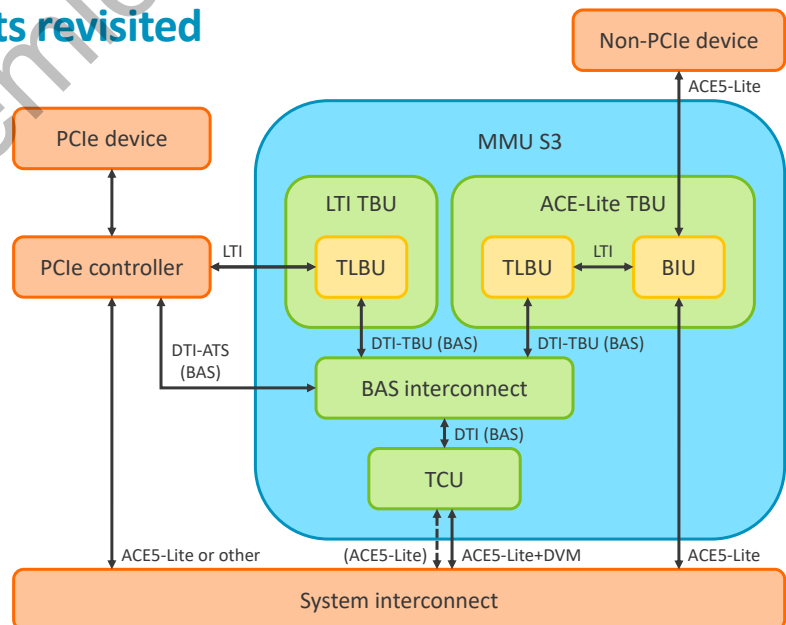
- Wrapper around Bus Interface Unit (BIU) and Translation Lookaside Buffer Unit (TLBU)
- BIU and TLBU connected with internal LTI interface

### LTI TBU

- Wrapper around TLBU only

### TCU

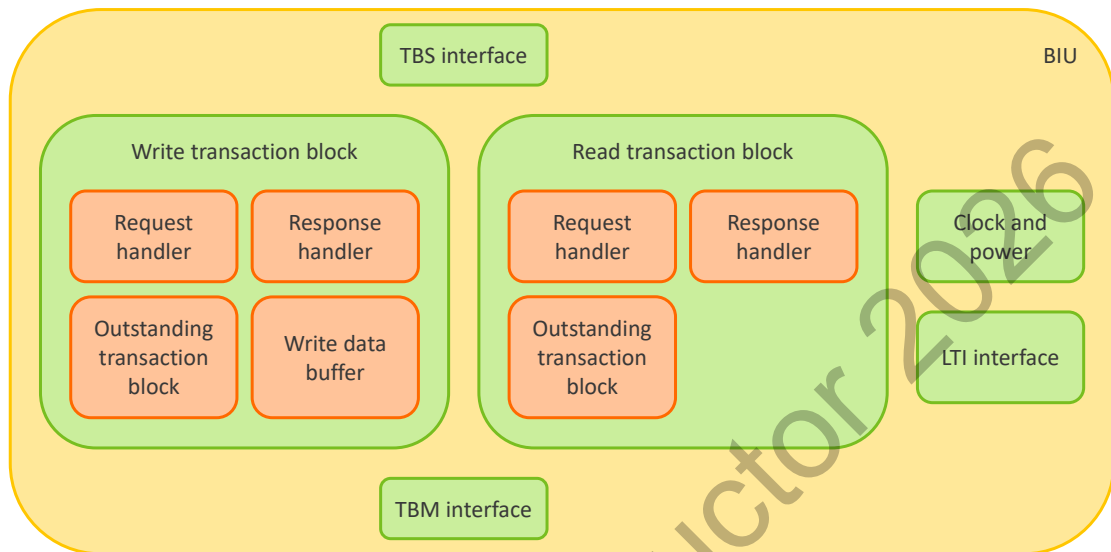
- Wrapper around Translation Management Unit (TMU) and Programming Interface Unit (PIU)
- TMU and PIU connected with internal custom interface (PTI)



5 xxxxx rev 00000

5

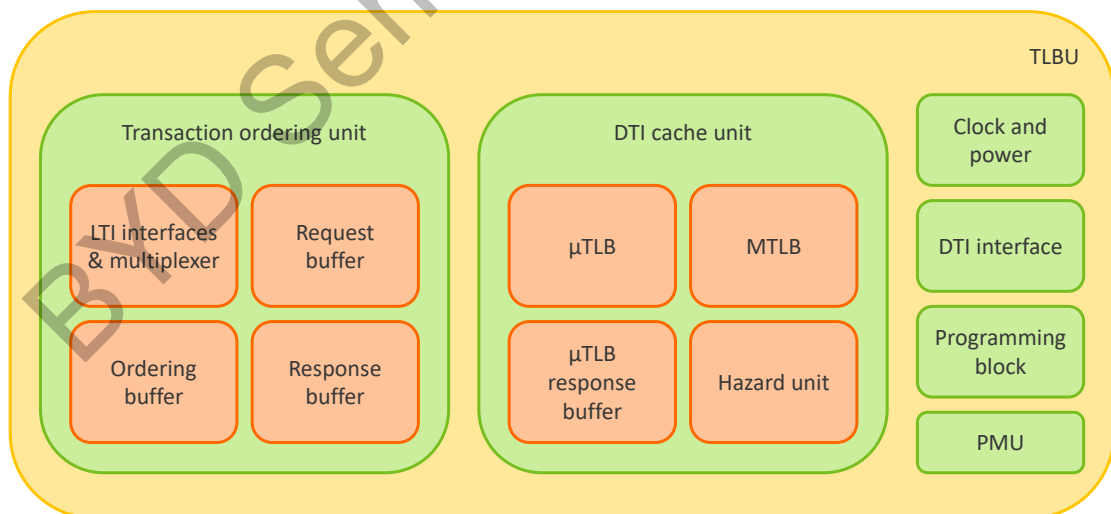
## BIU internal structure



6 xxxx rev 00000

6

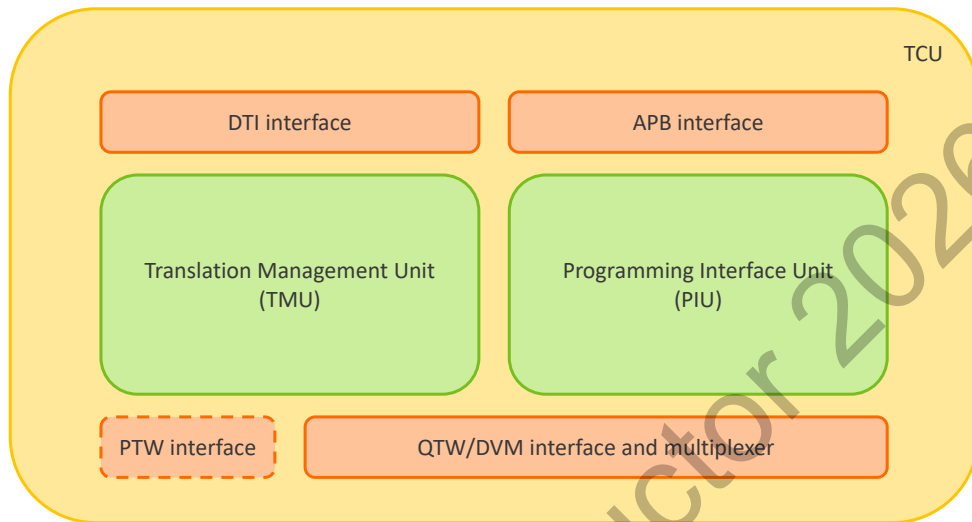
## TLBU internal structure



7 xxxx rev 00000

7

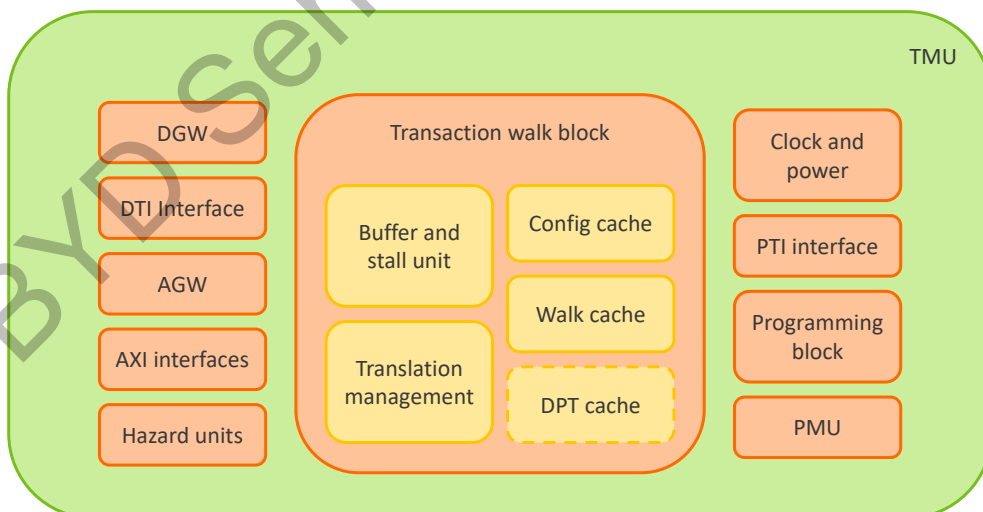
## TCU internal structure



8 xxxx rev 00000

8

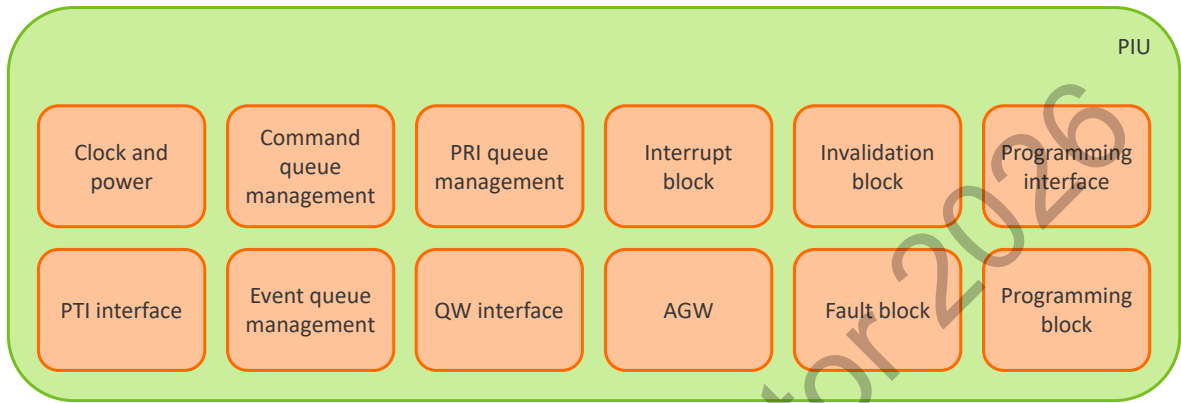
## TCU TMU internal structure



9 xxxx rev 00000

9

## TCU PIU internal structure



10 xxxx rev 00000

10

## Agenda

### Internal structures

- MMU S3 components revisited
- TBU internal structure
  - BIU internal structure
  - TLBU internal structure
- TCU internal structure
  - TMU internal structure
  - PIU internal structure

### Configuration parameters

- Common TBU configuration parameters
- ACE-Lite TBU configuration parameters
- LTI TBU configuration parameters
- TCU configuration parameters
- BAS interconnect configuration parameters

### Tie-off signals

- TBU tie-off signals
- TCU tie-off signals

### Integration and testing

- Cell replacement
- RAM integration
- Execution testbench

### Further reading

- Architecture specifications
- MMU S3 manuals
- Related product information
- Protocol specifications

11 xxxx rev 00000

11

## Common TBU configuration parameters (1/2)

### Number of translation slots

- Controls how many incoming transactions/translation requests can be processed in parallel

### Number of ordering groups

- Controls the possible level of reordering between reorderable transactions

### μTLB settings

- Depth

### MTLB settings

- Depth, ways, banks, lookup slots, update slots, MTLB direct indexing, MTLB partitioning, PARTID width

### Number of hazard entries

- Controls the number of open hazard lists that can be appended to by new requests

12 xxxx rev 00000

12

## Common TBU configuration parameters (2/2)

### Implemented security features

- RME or TrustZone Security, MECID width

### Number of PMU counters

- For counting performance events

### Support for ELA integration

- Enabled or disabled

### Flop parity

- Enabled or disabled

### RAM configuration for each RAM type

- One port, two ports or banked (2x one port)

13 xxxx rev 00000

13

## ACE-Lite TBU configuration parameters

### TBS and TBM interface properties

- ID, data, user signal, loopback, StreamID, SubstreamID widths

### Write data buffer depth

- Controls how many beats of write data can be stored during address translation

### Transaction tracker type

- Table tracking or loopback tracking

### Maximum outstanding transactions on TBM interface

- Separate for reads and writes

### Register slices

- Can help with timing at TBS and TBM interfaces as well as internal LTI connection between BIU and TLBU

14 xxxx rev 00000

## LTI TBU configuration parameters

### LTI interface settings

- ID, loopback, StreamID, SubstreamID widths

### Register slices

- Can help with timing at LTI interfaces

### Number of additional LA FIFO entries (R1 only)

- Additional LA channel credits can be provided to cover higher credit loop latency caused by external bridges or register slices

15 xxxx rev 00000

## TCU configuration parameters (1/4)

### QTW/DVM and PTW interface data width

- Parameter value applies to both interfaces

### Number of translation slots

- Controls how many DTI translation requests can be processed in parallel

### Number of CTW slots

- Controls how many configuration table walk sequences can be performed in parallel

### Number of PTW slots

- Controls how many page table walk sequences (including GPCs and GPT walks on CTWs and PTWs) can be performed in parallel

### Number of DGW slots

- Controls how many GPCs and GPT walks on DTI translation responses can be performed in parallel

### Number of DTI managers

- Controls address map size and address width of APB5 programming interface

16 xxxx rev 00000

16

## TCU configuration parameters (2/4)

### Common cache settings

- PARTID width

### Configuration cache settings

- Depth, ways, lookup slots (R1 only), indexing mode (simple or polynomial)

### Walk cache settings

- Depth, ways, banks, lookup slots

### TMU AGW GCB settings

- Depth, ways, banks, lookup slots

### TMU DGW GCB settings

- Depth, ways, banks, lookup slots

### DPT checks and DPT cache settings (R1 only)

- DPT support enabled, depth, ways, banks, lookup slots

17 xxxx rev 00000

17

## TCU configuration parameters (3/4)

### DTI-ATS settings

- Maximum number of ATS managers, maximum number of outstanding invalidations per manager

### Number of hazard entries

- Controls the number of open hazard lists that can be appended to by new requests
- Applies to both hazard units (QTW/DVM and PTW)

### Support for TCU prefetch

- Enabled or disabled

### System virtual address size

- Used to correctly interpret TLB invalidate DVM messages

### Implemented security features

- RME or TrustZone Security, MECID width

18 xxxx rev 00000

## TCU configuration parameters (4/4)

### Number of PMU counters

- For counting performance events

### Support for ELA integration

- Enabled or disabled

### Flop parity

- Enabled or disabled

### RAM configuration for each RAM type

- One port, two ports or banked (2x one port)

19 xxxx rev 00000

## BAS interconnect configuration parameters

### Common BAS interconnect settings

- Upstream data width, downstream data width, ID width

### BAS switch settings

- Minimum/maximum ID decode values for each subordinate interface

### BAS sizer settings

- Upstream data width, downstream data width for the other interface

### BAS register slice settings

- Register slice mode – forward, reverse, full or bypass

20 xxxx rev 00000

## Agenda

### Internal structures

- MMU S3 components revisited
- TBU internal structure
  - BIU internal structure
  - TLBU internal structure
- TCU internal structure
  - TMU internal structure
  - PIU internal structure

### Configuration parameters

- Common TBU configuration parameters
- ACE-Lite TBU configuration parameters
- LTI TBU configuration parameters
- TCU configuration parameters
- BAS interconnect configuration parameters

### Tie-off signals

- TBU tie-off signals
- TCU tie-off signals

### Integration and testing

- Cell replacement
- RAM integration
- Execution testbench

### Further reading

- Architecture specifications
- MMU S3 manuals
- Related product information
- Protocol specifications

21 xxxx rev 00000

## TBU tie-off signals

| Tie-off signal    | Description                                                                              |
|-------------------|------------------------------------------------------------------------------------------|
| ns_sid_high       | High order StreamID value for Non-secure state                                           |
| s_sid_high        | High order StreamID value for Secure state                                               |
| r_sid_high        | High order StreamID value for Realm state                                                |
| max_tok_trans     | Number of translation tokens requested from TCU upon DTI connection                      |
| sec_override      | Permits access to TBU_SCR register by Non-secure software out of reset                   |
| ecorevnum         | Must be tied to 0, unless instructed by Arm                                              |
| utlb_roundrobin   | Pseudo Least Recently Used (PLRU) or round-robin replacement policy for $\mu$ TLB        |
| pcie_mode         | Tie HIGH if TBU is connected to a PCIe Root Port                                         |
| same_power_domain | Tie HIGH if TBU is in the same power domain as TCU                                       |
| legacy_tz_en      | RME or TrustZone Security mode                                                           |
| poison_support    | Enables poison generation to defer double error in write data buffer (ACE-Lite TBU only) |

22 xxxx rev 00000

22

## TCU tie-off signals (1/2)

| Tie-off signal | Description                                                                                |
|----------------|--------------------------------------------------------------------------------------------|
| sup_cohacc     | Indicates whether the QTW/DVM and PTW interfaces can issue shareable transactions          |
| sup_btm        | Indicates whether the system can issue DVM messages to TCU for TLB maintenance             |
| sup_sev        | Indicates whether the send event mechanism is supported                                    |
| sup_httu       | Indicates whether atomic hardware updates to the Access flag and Dirty state are supported |
| sup_oas        | Output address size supported by the system                                                |
| sec_override   | Permits access to TCU_SCR register by Non-secure software out of reset                     |
| ecorevnum      | Must be tied to 0, unless instructed by Arm                                                |
| msi_addr       | Address for MSIs that should be sent from the AXI5-Stream MSI interface                    |
| tcu_sid        | StreamID value used as DeviceID for AXI5-Stream MSIs originating from MMU S3               |

23 xxxx rev 00000

23

## TCU tie-off signals (2/2)

| Tie-off signal     | Description                                                         |
|--------------------|---------------------------------------------------------------------|
| legacy_tz_en       | RME or TrustZone Security mode                                      |
| l0_gpt_sz          | Level 0 GPT entry size (LOGPTSZ)                                    |
| hazard_width       | Width of memory accesses performed to provide expanded hazard width |
| ns_gbpa_abort_init | Bypass or abort Non-secure translation out of reset                 |
| s_gbpa_abort_init  | Bypass or abort Secure translation out of reset                     |

24 xxxx rev 00000

24

## Agenda

### Internal structures

- MMU S3 components revisited
- TBU internal structure
  - BIU internal structure
  - TLBU internal structure
- TCU internal structure
  - TMU internal structure
  - PIU internal structure

### Configuration parameters

- Common TBU configuration parameters
- ACE-Lite TBU configuration parameters
- LTI TBU configuration parameters
- TCU configuration parameters
- BAS interconnect configuration parameters

### Tie-off signals

- TBU tie-off signals
- TCU tie-off signals

### Integration and testing

- Cell replacement
- RAM integration
- Execution testbench

### Further reading

- Architecture specifications
- MMU S3 manuals
- Related product information
- Protocol specifications

25 xxxx rev 00000

25

## Cell replacement

**Behavioral cell models must be replaced with technology-specific library cells**

- Synchronizers
- Clock gates
- Glitch-free OR and XOR gates

**Library cells must pass Logical Equivalence Checking (LEC)**

26 xxxx rev 00000

26

## RAM integration

**RAMs are used for internal queues, buffers and caches**

**Supported RAM configurations**

- Two-port RAMs (one port for reads, one port for writes)
- Single-port RAMs (one port for both reads and writes)
- Two single-port RAMs (banked configuration)

**Supported RAM implementations**

- Compiled or custom SRAM
- Compiled or custom Register File (RF) memory
- Generic memory model (implemented as flops during synthesis)

**MBIST information file generator provides individual RAM requirements for chosen configuration**

- RAM type (one port, two port or banked 2x one port)
- Dimensions (depth and width)

**Use the RAM integration testbench to confirm successful RAM integration**

27 xxxx rev 00000

27

## Execution testbench

### Set of tests for limited RTL validation

- **initial\_checktest** – tests basic connectivity and functionality
- **config\_check** – compares parameter values observed from design against parameter values read from System Discovery registers
- **mmu\_s3\_power\_indicative\_min** – minimum power test
- **mmu\_s3\_power\_indicative\_max** – maximum power test
- **mmu\_s3\_power\_indicative\_typ** – typical power test

### Four configuration examples provided

- Four TBUs and a TCUX1 with a simple BAS switch and RME support
- Four TBUs and a TCUX2 with a simple BAS switch and RME support
- Four TBUs and a TCUX2 with a simple BAS switch and TrustZone support only
- 29 TBUs and a TCUX2 with all BAS interconnect components integrated and RME support

### Execution testbench can be customized for additional scenarios

- Use the stitching flow to create your own top-level configuration and execution testbench files
- Modify **initial\_checktest** to create your own UVM-based test

28 xxxx rev 00000

## Agenda

### Internal structures

- MMU S3 components revisited
- TBU internal structure
  - BIU internal structure
  - TLBU internal structure
- TCU internal structure
  - TMU internal structure
  - PIU internal structure

### Configuration parameters

- Common TBU configuration parameters
- ACE-Lite TBU configuration parameters
- LTI TBU configuration parameters
- TCU configuration parameters
- BAS interconnect configuration parameters

### Tie-off signals

- TBU tie-off signals
- TCU tie-off signals

### Integration and testing

- Cell replacement
- RAM integration
- Execution testbench

### Further reading

- Architecture specifications
- MMU S3 manuals
- Related product information
- Protocol specifications

29 xxxx rev 00000

## Further reading (1/2)

### Architecture specifications

- Armv9-A Architecture Reference Manual (ARM) (DDI 0487)
  - Section D8: The Virtual Memory System Architecture (VMSA)
  - Section D9: The Granule Protection Check (GPC) Mechanism
  - Section D7.4: Memory Encryption Contexts (MEC)
  - Section I5: RAS System Architecture
- SMMUv3 Architecture Specification (IHI 0070)
- MPAM System Component Specification (IHI 0099)

### MMU S3 manuals

- MMU S3 Technical Reference Manual (TRM) (102754)
- MMU S3 Configuration and Integration Manual (CIM) (102755)
- MMU S3 Release Note (RN) (107743)
- MMU S3 Software Developers Errata Notice (SDEN) (SDEN-3095623)
- MMU S3 Product Errata Notice (PEN) (PEN-3095623)

30 xxxxx rev 00000

## Further reading (2/2)

### Related product information

- ADB-400 User Guide (DUI 0615)
- LDB-700 Design Manual (102362)
- LPD-500 Technical Reference Manual (TRM) (100361)
- LPD-500 Integration and Implementation Manual (IIM) (100362)

### Protocol specifications

- AMBA AXI and ACE Protocol Specification (IHI 0022)
- AMBA AXI-Stream Protocol Specification (IHI 0051)
- AMBA DTI Protocol Specification (IHI 0088)
- AMBA LTI Protocol Specification (IHI 0089)
- AMBA APB Protocol Specification (IHI 0024)
- AMBA Low Power Interface (LPI) Specification (IHI 0068)

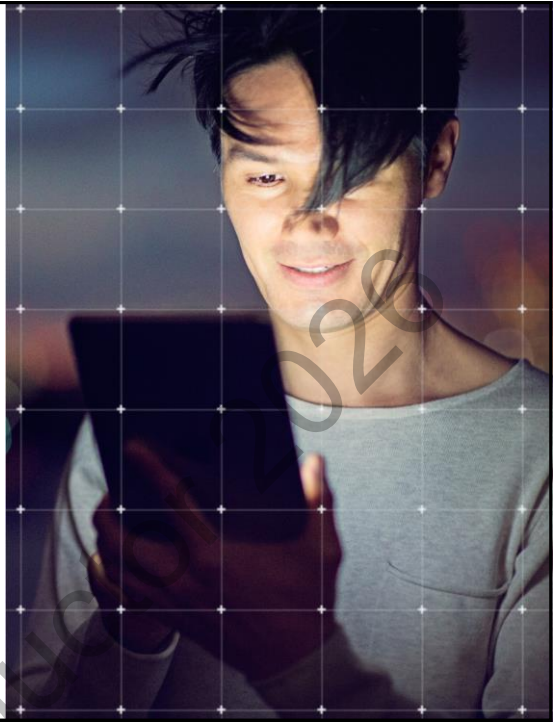
31 xxxxx rev 00000

## Module summary

In this module we have covered

Internal structures  
Configuration parameters  
Tie-off signals  
Integration and testing  
Further reading

32 xxxx rev 00000



32

## Course summary

You should now be familiar with the following MMU S3 topics:

- Introduction and topology
- Interfaces
- Translation
- Caching
- Additional features
- Configuration and integration



33

## Expectations and next steps

Has the course met your expectations?

What parts of the course have benefited you?

Do you have any concerns?

What are the next steps?

34

arm

Thank You

Danke

Gracias

谢谢

ありがとう

Asante

Merci

감사합니다

धन्यवाद

Kiitos

شكراً

ধন্যবাদ

תודה

35



# MMU-720AE Protection Mechanisms

Confidential © 2025 Arm Limited

2

## Module agenda

- [Introduction](#)
- FuSa protection mechanisms in MMU-720AE
  - I/O Interface Protection
  - Logic Duplication
  - RAM Protection
- Fault Management Unit
- FuSa Configuration Parameters

3 2662 rev 51588



3

## Introduction

MMU-720AE is a Functional Safety (FuSa) variant of MMU S3

- Includes a set of Protection Mechanisms for added Fault Detection and Control (FDC)
- FuSa features are “bolted on” to MMU S3 and do not alter the original MMU S3 functionality

The MMU-720AE is considered a Safety Element out of Context (SEooC)

- Development based on documented Assumptions of Use
- System integrator is responsible for ensuring integration of the IP into the design fulfils the relevant functional safety requirements for their qualification process

This module will describe the Protection Mechanisms and the specific behaviors relevant to Functional Safety

## MMU-720AE FuSa Features

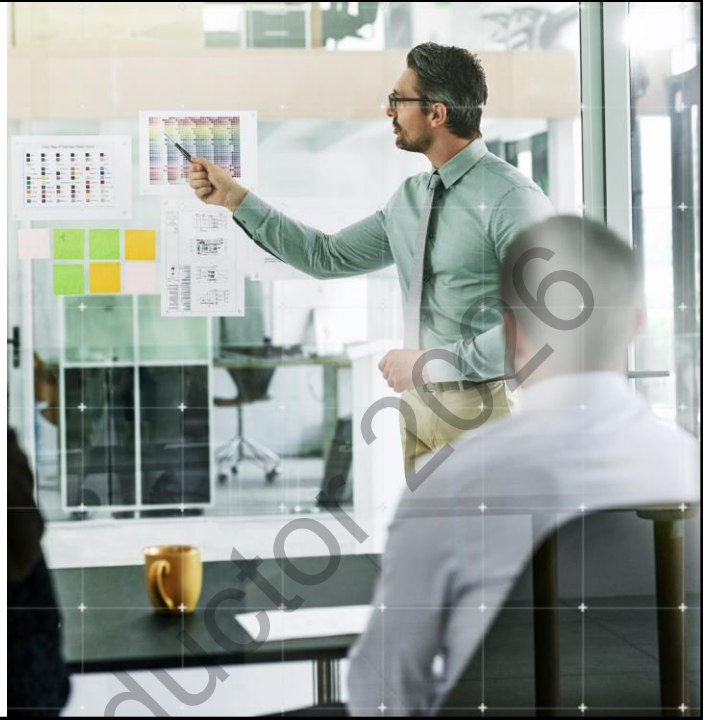
The MMU-720AE provides several Functional Safety (FuSa) features to detect random hardware faults, systematic faults, and latent faults

There are several protection mechanisms for Fault Detection and Control (FDC)

- Interface Protection
- Logic Duplication
- Error Correcting Codes (ECC) to protect RAM memories
- A Fault Management Unit (FMU)
  - Present in each TBU and in the TCU

## Module agenda

- Introduction
- FuSa protection mechanisms in MMU-720AE
  - I/O Interface Protection
  - Logic Duplication
  - RAM Protection
- Fault Management Unit
- FuSa Configuration Parameters



6 2662 rev 51588

6

## I/O Interface Protection

We apply protection to several I/O interfaces of MMU-720AE

- I/O protection is implemented via mechanisms like parity and duplication

The main methods of protecting interfaces are:

- Parity Checks
- Full Duplication
- Asynchronous Filtering
- Clock and Reset Wakeup Protection

These methods are not configurable and are always enabled

- An interface will use only one of these methods to protect that interface/signal
- The following slides describe these methods in more detail

7 2662 rev 51588

7

## Parity Checks

For synchronous interfaces, parity checks are the most area and power efficient checking mechanism

- Interfaces have a corresponding CHK signals associated with the primary signals, i.e. AWADDR and AWADDRCHK

The parity mechanism used is defined by the signal's associated protocol specification

- ACE-Lite parity is defined in the AXI protocol specification, APB parity in the APB protocol specification, etc.
- Non-AMBA signals are protected by parity CHK signals that use single bit odd-parity protection
- DTI and LTI interfaces are not protected by parity (more details on next slide)

The parity check module is duplicated internally to provide latent fault diagnostics

For incoming interface signals, the MMU-720AE flags an error if either the main parity check module or the duplicated module detects a mismatch

8 2662 rev 51588

8

## Full Duplication

Synchronous interfaces are fully duplicated when parity checks add too much complexity

The Distributed Translation Interfaces (DTI) and Local Translation Interfaces (LTI) are fully duplicated

- Each signal in the interface has a duplicated CHK signal

The duplicated CHK interface carries the same information as the primary interface

- All CHK signals are non-inverted
- All CHK signals are the same width as the primary signals

9 2662 rev 51588

9

## Asynchronous Filtering

Asynchronous interfaces are duplicated with CHK signals and require special consideration

- Ambiguity in timing causes the previous protection techniques to incorrectly flag an error
- Examples of asynchronous interfaces/signals are PMUSNAPSHOT and QREQn
  - Refer to “Signal Descriptions” section in TRM for full list of asynchronous signals that are protected

Asynchronous signals are protected by a filtering mechanism which does not propagate changes until the primary and CHK signals are in agreement

- If primary and CHK signals disagree for 256 cycles or more, a timeout signal triggers raising a fault
- No fault raised if primary and CHK signals disagree for fewer than 256 cycles

If you use the FMU\_SMEN register to disable the asynchronous safety mechanisms, you disable this filter and propagate only the primary copy of the signal to both the primary and duplicated logic

10 2662 rev 51588

## Clock and Reset Wakeup Protection

MMU-720AE components include **qactive** and **qactivechk** signals for clock wakeup protection

- qactivechk is an inverted copy of qactive
- These signals are asynchronous

After a reset exit, the qactive signal requires 2 clock cycles to stabilize

- Parity checks on qactive must be disabled (on the receiving IP) for 2 clock cycles out of reset
- FuSa LPD-500 provided with MMU-720AE will enable qactive parity check 2 cycles after reset, so no special consideration is required

For reset protection, the **resetn** and **resetnchk** (duplicated) signals are OR'd together internally

- OR'ing causes transient faults and stuck-at-0 faults on either resetn or resetnchk to have no effect
- Note: Stuck-at-1 faults on resetn might prevent an IP reset and are not detected by MMU-720AE

11 2662 rev 51588

## I/O Interface Protection with Legacy IP

I/O protection is always present however MMU-720AE still supports connections to interfaces without I/O protection

- Legacy IP might not support AMBA parity protection

User will need to leave the CHK signals disconnected/tied-off and disable the associated safety mechanism

Safety mechanisms are disabled through the TCU\_FMU\_SMEN and TBU\_FMU\_SMEN registers

- List of safety mechanisms shown in later slides

12 2662 rev 51588

12

## Module agenda

- Introduction
- FuSa protection mechanisms in MMU-720AE
  - I/O Interface Protection
  - Logic Duplication
  - RAM Protection
- Fault Management Unit
- FuSa Configuration Parameters

13 2662 rev 51588



13

## Logic Duplication (1)

MMU-720AE uses logic duplication as its primary method of detecting faults within its digital logic

- Redundant logic operates in lockstep with primary logic
- Clocks and resets are duplicated
- RAMs are shared

Redundant logic can operate with either a 2-cycle delay or with no delay

- Set through the `FUSA_TEMPORAL_DELAY` parameter
  - Present on all components except the BAS Switch
- Having a 2-cycle delay makes it easier to implement and physically separate the primary and redundant logic
  - Power and total flop count increases slightly when using 2-cycle delay

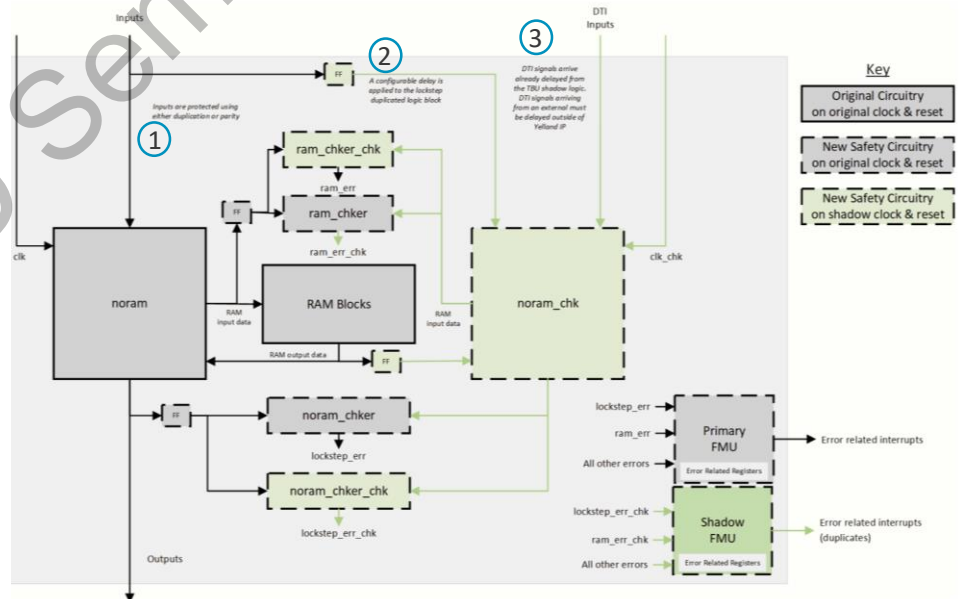
RAMs are not duplicated

- Only primary copy performs accesses
- Read data is sent to both copies

14 2662 rev 51588

## Logic Duplication (2)

1. Inputs are protected using either duplication or parity
2. A configurable delay is applied to the lockstep duplicated logic block
3. DTI Signals arrive already delayed from the TBU shadow logic. DTI signals arriving from an external device must be delayed outside of the MMU



15 2662 rev 51588

## DTI Interconnect Protection (1)

MMU-720AE duplicates DTI components such as register slices, switches, and sizers

- These components are provided in the bundle and the duplication is done internally
- The DTI switches, sizers and register slices are fully duplicated and form separate networks
- These networks must remain in lockstep
- The module names of these components are:
  - mmu\_720ae\_bas\_reg\_slice\_ae
  - mmu\_720ae\_bas\_sizer\_ae
  - mmu\_720ae\_bas\_switch\_<N>x1\_ae where N = 2, 3, 4, 8, or 16

FuSa AMBA Domain Bridge (ADB) provide in MMU-720AE bundle as well

- This bridge passes both the primary and CHK DTI messages across the domain and re-synchronizes the messages while checking for differences
- Faults detected on the ADB are reported to the TCU (more on next slide)

FuSa Low Power Distributor (LPD) also provided in bundle and faults are reported to TCU

16 2662 rev 51588

16

## DTI Interconnect Protection (3)

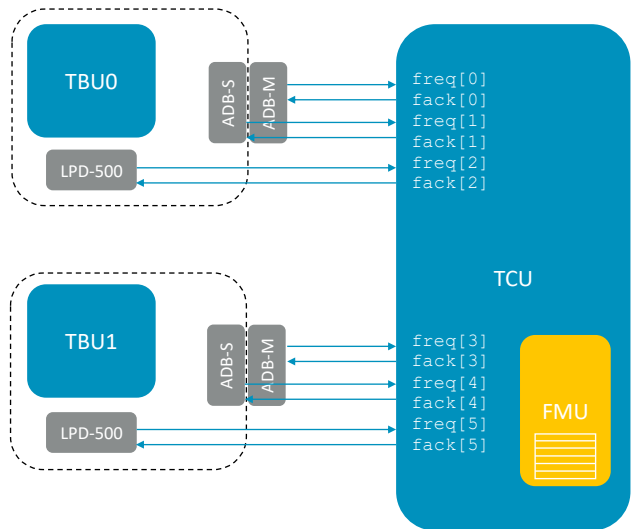
ADBs and LPDs must be connected to send faults to the FMU in the TCU via the **F-channel**:

- ADB and LPD faults signalled on **freq/fack** (and **freq\_chk/fack\_chk** for redundancy)
- Specific TCU FuSa configuration parameters need to be set to match number of channels (more on later slides)

For the ADB, subordinate and manager bridge sides use separate F-channels

LPD uses a single F-channel

*\*\*the diagram doesn't show freq\_chk/fack\_chk*

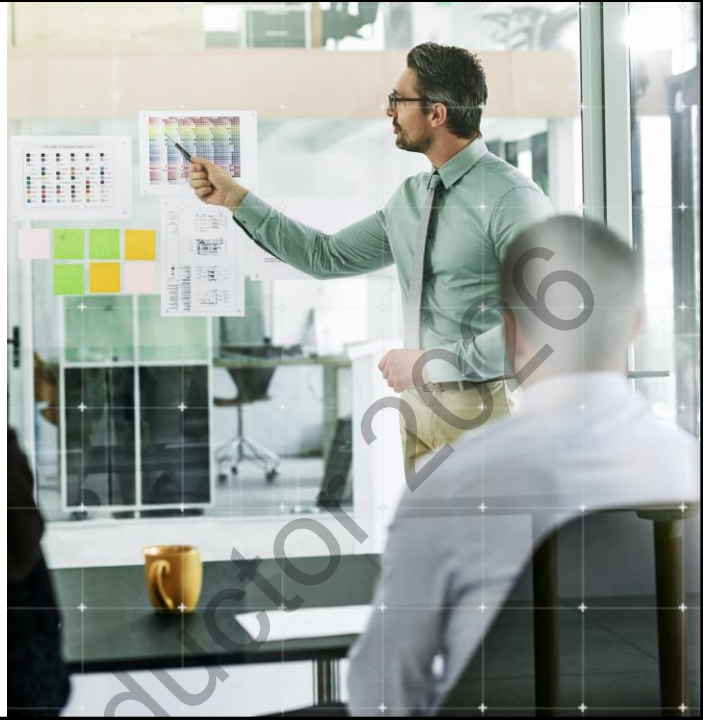


17 2662 rev 51588

17

## Module agenda

- Introduction
- FuSa protection mechanisms in MMU-720AE
  - I/O Interface Protection
  - Logic Duplication
  - **RAM Protection**
- Fault Management Unit
- FuSa Configuration Parameters



18 2023 rev 33324

18

## RAM Protection

RAMs in the MMU-720AE are protected by Error Correcting Codes (ECC)

- Single-Error-Correct-Double-Error-Detect (SECCDED)
  - Used for RAMs where the SMMU cannot proceed after a double error
  - Single error is correctable and triggers FHI
  - Double error in ACE-Lite TBU write data buffer is deferrable and triggers FHI if poison is supported
  - Otherwise double error is uncorrectable and triggers FHI, ERI and CRI
- Double-Error-Detect (DED)
  - Used for RAMs where data can be recovered by invalidating corrupted cache lines and refetching from DRAM
  - Single or double error detected during a cache lookup triggers FHI and is treated as a cache miss

Faults detected in the RAMs will be logged in the normal RAS registers of the TCU and TBU

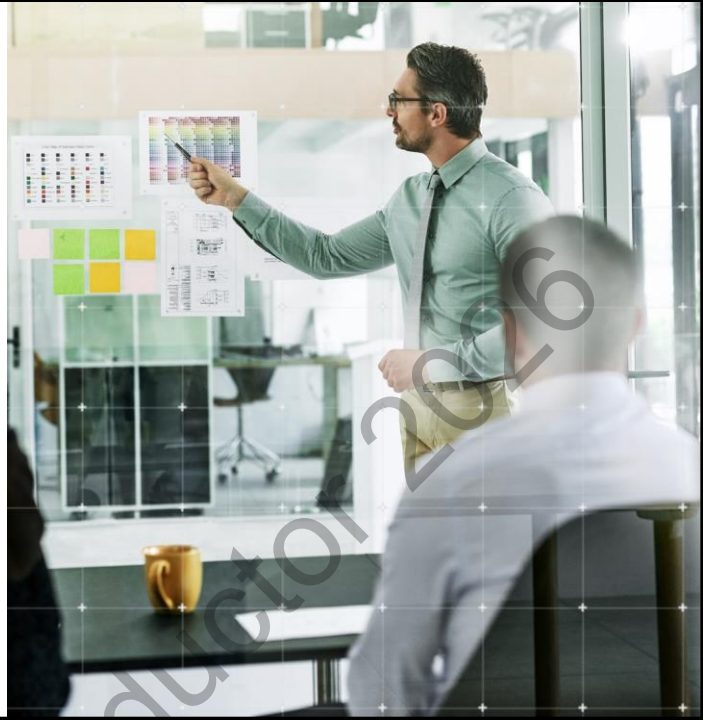
- i.e. TCU\_ERRSTATUS and not in TCU\_FMU\_ERRSTATUS

19 2662 rev 51588

19

## Module agenda

- Introduction
- FuSa protection mechanisms in MMU-720AE
  - I/O Interface Protection
  - Logic Duplication
  - RAM Protection
- **Fault Management Unit**
- FuSa Configuration Parameters



20 2662 rev 51588

20

## Fault Management Unit (FMU)

MMU-720AE contains Fault Management Units (FMU) that records and controls the reporting of faults

- FMU is present in the TCU and TBU blocks
- Components such as ADBs and LPDs report their faults to the TCU

FMU implements the following functionality:

- Enable and disable the Safety Mechanisms (SMs)
- Receive and store fault information from enabled SMs
- Enable and disable the assertion of interrupts when faults are detected
- Allow software error injection with SMs

21 2662 rev 51588

21

## Accessing the FMU

The FMU registers for both the TCU and TBUs can be accessed through the APB bus in the TCU

- These registers require Root access if your system is configured with `TCUCFG_LEGACY_TZ_EN = 0` and the input `legacy_tz_en = 0`
- Otherwise they are Secure if using the legacy TrustZone requirements

The TCU and each TBU have a separate set of interrupts

- Interrupt generation is enabled individually at each TCU and TBU `FMU_ERRCTLR` register

The TCU FMU registers start at offset `0x6_0000` from the TCU base address

The TBU FMU registers start at offset `0x1_0000` from each TBU base address

22 2662 rev 51588

22

## Enabling and disabling SMs

MMU-720AE provides a `TCU_FMU_SMEN` and a `TBU_FMU_SMEN` configuration register that control the error reporting behavior for each individual SM in that block

All SMs in MMU-720AE are enabled at reset

If you do not require checking from a particular SM, use software to write to the `TCU_FMU_SMEN` or `TBU_FMU_SMEN` register to disable that SM

- Next slides show the SMs that can be disabled

All SM errors, except for those from the legacy RAM ECC checkers, are reported as critical errors and uncorrected errors to the FMU

23 2662 rev 51588

23

## TCU\_FMU\_SMEN

| Bits | Name              | Reset Value | Type | Function                                                                |
|------|-------------------|-------------|------|-------------------------------------------------------------------------|
| 31:8 | -                 | All zeros   | RO   | RES0                                                                    |
| 7    | QTW_IN            | 1'b1        | RW   | Enables checking on QTW Inputs                                          |
| 6    | PTW_IN            | 1'b1        | RW   | Enables checking on PTW inputs                                          |
| 5    | MSI_IN            | 1'b1        | RW   | Enables checking on MSI Inputs                                          |
| 4    | APB_IN            | 1'b1        | RW   | Enables checking on APB inputs                                          |
| 3    | F_CH_IN           | 1'b1        | RW   | Enables checking on the F Channel                                       |
| 2    | ASYNC_IN          | 1'b1        | RW   | Enables checking on the asynchronous input checkers                     |
| 1    | LOCKSTEP_SYNC_OUT | 1'b1        | RW   | Enables checking on the synchronous lockstep checker (i.e. noram_chker) |
| 0    | LOCKSTEP_RAM_OUT  | 1'b1        | RW   | Enables checking on the RAM lockstep checker                            |

24 2662 rev 51588

24

## TBU\_FMU\_SMEN

| Bits | Name              | Reset Value | Type | Function                                                                |
|------|-------------------|-------------|------|-------------------------------------------------------------------------|
| 32:5 | -                 | All zeros   | RO   | RES0                                                                    |
| 4    | F_CH_IN           | 1'b1        | RW   | Enables checking on the F-Channel                                       |
| 3    | ACELITE_IN        | 1'b1        | RW   | Enables checking on the ACE-LITE interface                              |
| 2    | ASYNC_IN          | 1'b1        | RW   | Enables checking on the asynchronous input checkers                     |
| 1    | LOCKSTEP_SYNC_OUT | 1'b1        | RW   | Enables checking on the synchronous lockstep checker (i.e. noram_chker) |
| 0    | LOCKSTEP_RAM_OUT  | 1'b1        | RW   | Enables checking on the RAM lockstep checker                            |

25 2662 rev 51588

25

## FMU Interrupt Handling (1)

The FMU implements different types of external interrupts to report errors to the rest of the system

### Error Recovery Interrupt (ERI)

- Used to report Uncorrected Errors (UEs) and Deferred Errors (DEs) to the rest of the system
- You can use ERIs for error recovery, fault handling, or both

### Fault Handling Interrupt (FHI)

- Used to report UEs and Corrected Errors (CEs) to the rest of the system
- You can use FHIs for fault handling

### Critical Error Interrupt (CRI)

- Used to report critical error conditions to the rest of the system
- You can use CRIs for error recovery

You can individually enable each type of interrupt signaling by programming the FMU\_ERRCTLR registers

26 2662 rev 51588

## FMU Interrupt Handling (2)

MMU-720AE has three types of interrupts that are Edge triggered and four types that are Level sensitive

- Same set of interrupts on TCU and TBU blocks

| Edge-triggered signal     | Level-sensitive signal          | MSI | Description                                                        |
|---------------------------|---------------------------------|-----|--------------------------------------------------------------------|
| ras_et_fhi/ras_et_fhi_chk | ras_lt_fhi/ras_lt_fhi_chk       | ✗   | Fault Handling Interrupt (corrected error)                         |
| ras_et_eri/ras_et_eri_chk | ras_lt_eri/ras_lt_eri_chk       | ✗   | Error Recovery Interrupt (recoverable uncorrected error)           |
| ras_et_cri/ras_et_cri_chk | ras_lt_cri/ras_lt_cri_chk       | ✗   | Critical Error Interrupt (uncontainable uncorrected error)         |
| -                         | ras_lt_irpt_v/ras_lt_irpt_v_chk | ✗   | TCU_FMU_ERRGSR or TBU_FMU_ERRGSR register has a valid error record |

27 2662 rev 51588

## FMU Interrupt Handling (3)

The interrupt handling routine is similar to that of Arm Neoverse MMU S3, except that there are now multiple sources of errors

When software receives a RAS interrupt, it first reads the Global Status Register in the block that issued the interrupt

- This is either TCU\_FMU\_ERRGSR or TBU\_FMU\_ERRGSR

Software can then determine whether the fault originated in the RAMs or the MMU-720AE SMs

If the fault originated in the RAMs, software can read the TCU\_ERRSTATUS or TBU\_ERRSTATUS registers to determine what fault occurred

If the fault was reported by the SMs, software can read the TCU\_FMU\_ERRSTATUS or TBU\_FMU\_ERRSTATUS register to handle the error

28 2662 rev 51588

## FMU Error Injection

To inject an error into the FMU, write to the TCU\_FMU\_ERRGEN / TBU\_FMU\_ERRGEN register

- When you write to the FMU\_ERRGEN register you inject a single error only to that respective block
- It is not necessary to clear the error

The specific error to inject is written in the IERR field

- Refer to TRM for valid error codes
- Must also set the fields Status Register Valid (V), Uncorrected Error (UE), and Critical Error (CI) to 1 for error injection to take effect

Valid writes to TCU\_FMU\_ERRGEN are reported in the TCU\_FMU\_ERRSTATUS register

- Same applies to TBU\_FMU\_ERRGEN and TBU\_FMU\_ERRSTATUS
- Corresponding interrupts will be triggered, if enabled
- Software can then read the FMU registers to determine what error occurred

29 2662 rev 51588

## FMU warm reset

A warm reset clears the functional state but preserves the FMU error record registers by asserting only the functional resets

TCU and TBU blocks each contain a **fmurstdisable** pin to prevent reset from affecting the FMU registers

- Warm reset is performed by asserting fmurstdisable and then asserting resetn and resetnchk
- The fmurstdisable pin must be asserted and stable prior to entering reset

To verify that the FMU registers are preserved when asserting fmurstdisable :

1. Perform a reset using the resetn and resetnchk pins
2. Inject an error into the FMU registers
3. Read back the error status from the FMU and confirm that the injected error is present
4. Perform a reset using the resetn and resetnchk pins
5. Read back the error status from the FMU and confirm that the injected error is still present

30 2662 rev 51588

30

## Module agenda

- Introduction
- FuSa protection mechanisms in MMU-720AE
  - I/O Interface Protection
  - Logic Duplication
  - RAM Protection
- Fault Management Unit
- **FuSa Configuration Parameters**

31 2662 rev 51588



31

## FuSa specific configuration parameters (1)

MMU-720AE adds a parameter for Lock-Step Delay: **FUSA\_TEMPORAL\_DELAY**

- This is applicable to both the TCU and TBUs

| Parameter Name      | Values | Description                                                                                                                                                                                                                                                                             |
|---------------------|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FUSA_TEMPORAL_DELAY | 0,2    | Use the FUSA_TEMPORAL_DELAY parameter to delay the lockstep and shadow copies of the logic by N cycles, where N is the value of the parameter. This value must be the same across all SMMU components.<br><br>All components except the switches use the FUSA_TEMPORAL_DELAY parameter. |

32 2662 rev 51588

32

## FuSa specific configuration parameters (2)

MMU-720AE adds parameters for F-Channel reporting: **TCUCFG\_FUSA\_FCHAN\_COUNT** and **TBUCFG\_FUSA\_FCHAN\_COUNT**

| Parameter Name          | Values | Description                                                                                                                                                                                                                                 |
|-------------------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| TCUCFG_FUSA_FCHAN_COUNT | 1 - 96 | The number of F-Channels required. We use F-Channels as fault indicators from an ADB or LPD to a TCU. Set this value to 2 * Number of ADBs + the number of LPDs in the system. Set this value to 1 if you are not using F-Channel reporting |
| TBUCFG_FUSA_FCHAN_COUNT | 1-16   | Set this value to 1 as F-Channel reporting is not expected to be used on the TBU                                                                                                                                                            |

Note: It is recommended that **TCUCFG\_PARITY\_ON** and **TBUCFG\_PARITY\_ON** parameters are disabled because full duplication covers these parameters

33 2662 rev 51588

33

Thank You!  
Danke!  
Merci!  
谢谢!  
ありがとう!  
Gracias!  
Kiitos!

arm

arm

Glossary

© 2023 Arm

1

Arm Glossary

Module 128 is the ARM Glossary  
This is not a PowerPoint presentation  
This Presentation is included as a placeholder

- pdf files should not match

2 0128 rev 44195

2



# Arm Customer Services & Support

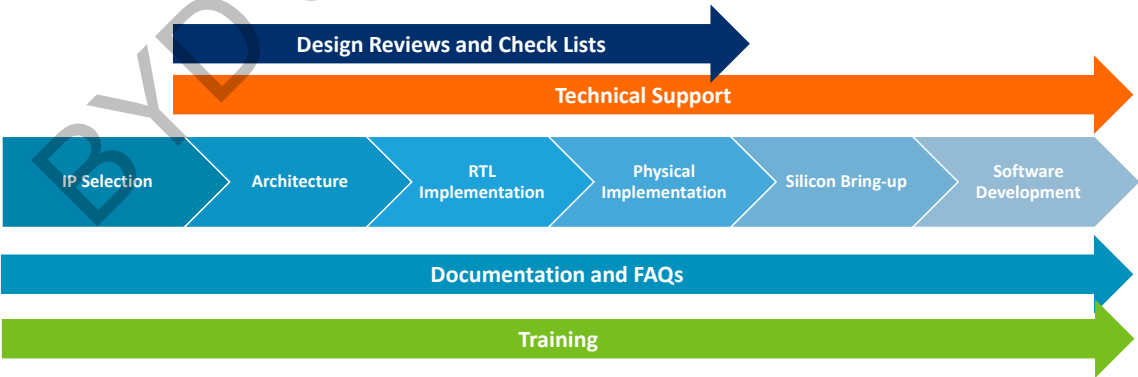
© 2024 Arm

1

## Arm Support Services

*Providing the help you need, when you need it*

We have a range of services available to help **reduce risk** and **shorten time-to-market**



2 0454 rev 50905

More info on [arm.com/support](https://arm.com/support)

2

# Support and Training – project assistance from Arm experts

Support & Training included:

|                                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Arm Developer</b><br><br>Arm's developer website includes documentation, tutorials, support resources and downloads for products and technologies.                         | <b>Arm Community</b><br><br>Stay informed, learn more, and ask and answer questions through our self-service developer community. Everyone contributes to help the community grow.                     | <b>Arm IP Explorer</b><br><br>A cloud-based platform to help SoC architects and hardware developers evaluate and select the best IP for SoC design projects.                                      |
| <b>Technical Support</b><br><br>Our worldwide team of experts provide online design advice and debug assistance throughout your subscription, via the online Arm Support Hub. | <b>On Demand Training</b><br><br>Expert-led online training designed to meet your requirements and help accelerate time to market. Seats are included in your Package for anyone using Arm technology. | <b>Design Checklists</b><br><br>A quick and easy way for to verify critical aspects in your design to help prevent errors, while ensuring the system follows Arm's recommended design guidelines. |

More info on [arm.com/support](https://arm.com/support)

3 0454 rev 50905

# Global team proving local support



4 0454 rev 50905

# Technical support

Consistently  
high customer  
satisfaction 95%

Resolve technical issues quickly and efficiently

- Keep your project on track

Experienced Arm engineers answer your questions

- Access to vast amount of Arm knowledge

Consistently high customer satisfaction

- Constantly monitored through customer surveys

Worldwide support team located in the US, Europe and Asia

- 200+ Applications Engineers covering all product areas
- Global teams providing local support

Online portal to track your support enquiries

- [Arm Support Hub](#)

5 0454 rev 50905

# On Demand training

Technical training created by experienced Arm Engineers, delivered when and where you want it

- Bite sized introductory training videos for all key IP
- Full in depth recorded training courses available
- Access via PC, tablet and mobile app
- Post questions about the training and gain answers from Arm Engineers
- Secure private company access allowing you to comment and share within your own company
- Assessment and certificates built in
- Content structured into learning plans around Arm topics



6 0454 rev 50905

<https://developer.arm.com/support/training/arm-on-demand-training>

# Design Checklists

Adding more confidence to your design

## Overview

Design Checklists help partners check through some critical aspects of their design to prevent errors

The checks ensure the system is fit for purpose and follows Arm's recommended design guidelines

Design Checklists are accessible to all partners and can be used in conjunction with remote or on-site Design Reviews.

## How to access

- Log into [Arm IP Explorer](#).\*
- Click on **Explore Processors**, Explore **System IP** or Explore **Interconnects** and search for the IP you need to check
- On the right side of the screen, scroll down to **Design Review Checklist** section
- Click **Download** the PDF checklist.\*\*



## Design Checklists Available

The three areas checked are:

- System Considerations
- Configuration Parameters
- And RTL Signals

Available for the following IP:

| Cortex-A   | Cortex-R    | Cortex-M   | System IP & Interconnect |
|------------|-------------|------------|--------------------------|
| Cortex-A32 | Cortex-R5   | Cortex-M0  | CoreLink CI-700          |
| Cortex-A35 | Cortex-R8   | Cortex-M0+ | CoreLink GIC-600         |
| Cortex-A53 | Cortex-R52+ | Cortex-M3  | CoreLink GIC-700         |
| Cortex-A55 | Cortex-R82  | Cortex-M4  | CoreLink MMU-600         |
| Cortex-A73 |             | Cortex-M7  | CoreLink MMU-700         |
|            |             | Cortex-M23 | CoreLink NIC-400         |
|            |             | Cortex-M33 | CoreSight SoC-600        |
|            |             | Cortex-M52 | Neoverse CMN-700         |
|            |             | Cortex-M55 |                          |
|            |             | Cortex-M85 |                          |

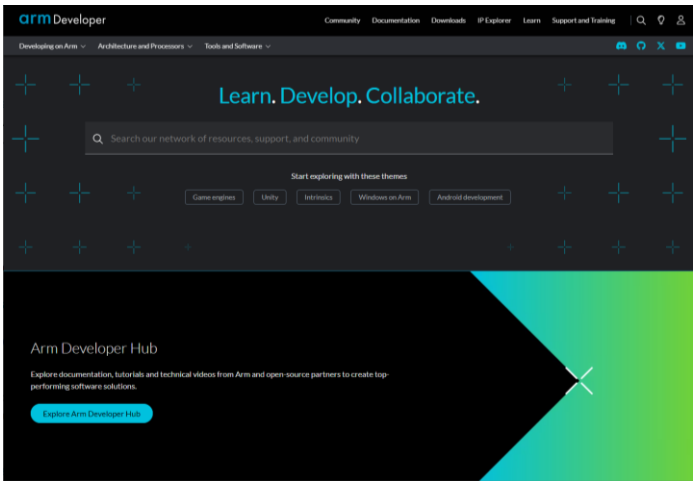
\*Registration needed; \*\*Access is based on partner IP entitlements

# Arm Developer

[www.developer.arm.com](http://www.developer.arm.com)

## One location for all technical content

- + Product information
- + Technical documents, guides, videos, and tutorials
- + Downloads
- + Developer resources
- + Training courses information and online booking

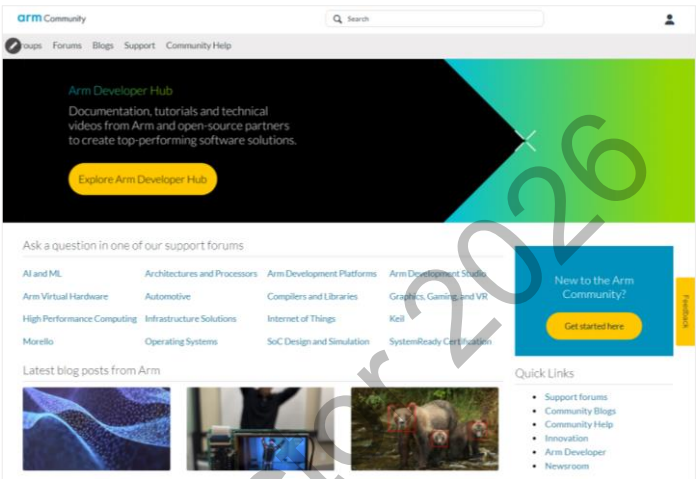


# Arm Community

www.community.arm.com

## Peer support & blogs by Arm experts

- + Discussion Forums
  - Access thousands of answers.
  - Receive support from fellow community members and Arm experts across topics such as AI and ML, Keil, and more.
- + Technical Blogs
  - Learn about Arm products, releases, and developments.
  - Access content from Arm experts, guest authors, and members of our Arm Developer Program.



9 0454 rev 50905

# Explore, Design, and Share with Arm IP Explorer

### Explore

Search Arm's full catalog of IP and view in-depth specifications



### Compare

Compare different pieces of IP based on a holistic set of attributes



### Simulate

Run simulations using industry benchmarks or your own code



### Configure

Configure your selected IP to meet specific needs



### Download

Render your configuration & download the configured RTL



### Create Concept

Visualize your design on our intelligent Canvas. Export & Share



10 0454 rev 50905



## Arm Approved Design Partners

### Network of Trusted Design Houses

#### Membership Tiers

- Design Services (Non-commercial Usage)
- SoC Services (Commercial Usage)

#### Entitlements

- IP in the Arm Flexible Access (AFA) Mainstream Package
- Software Development Tools
- Marketing Rights to the Program

#### Leverages the Flexible Access Program

- Membership Agreement
- Purchase Package Order Form

11 0454 rev 50905

## Arm Approved Training Partners

Learn Arm Technologies from Accredited Training

Global network of companies endorsed and supported by Arm that offer

- Arm Software training courses under license
- Local language versions of our software-based training courses
- Complementary training
- Public schedules

The training partners have demonstrated that they can deliver Arm architecture and technology training to a high technical level of understanding



12 0454 rev 50905

# Need help?

Tech Support

Go to [arm.com/support](https://arm.com/support)

Training

Go to [arm.com/training](https://arm.com/training)

Design Reviews

Go to [arm.com/design-reviews](https://arm.com/design-reviews)

Design Services

Go to [arm.com/arm-approved](https://arm.com/arm-approved)

Thank You!  
Danke!  
Merci!  
谢谢!  
ありがとう!  
Gracias!  
Kiitos!

arm